4章
データベースを分割する

説明してきたとおり、機能をマイクロサービスへと抽出する方法はたくさんある。ただし、部屋にいる象[†1]に対処しなくてはならない。つまり、データをどうにかする必要がある。マイクロサービスは、情報隠蔽を実践しているときに最もうまくいく。そのため、私たちは通常、データストレージや取得のメカニズムを完全にカプセル化するマイクロサービスを構築しようとする。このことから、マイクロサービスアーキテクチャに移行する場合に、その移行の力を最大限引き出したいのなら、モノリスのデータベースを分割する必要があるという結論が導かれる。

[†1] 訳注：誰もが見て見ぬふりをしている問題のこと。

しかしながら、データベースを分割することは、簡単な試みとはほど遠い。移行中のデータ同期や、論理スキーマと物理スキーマの分解、トランザクションの整合性、テーブル結合、レイテンシーなどの課題を考慮する必要がある。この章では、これらの課題を見ていきながら、私たちの助けとなるパターンを探っていく。

データベースを分割する方法に入る前に、まずは、共有データベースを管理する場合の課題と、それに対処するパターンを見ていこう。

4.1　パターン：共有データベース

1章で説明したように、結合はドメイン結合、一時的結合、実装結合の3つの観点で考えられる。この3つのうち、データベースを考えるときに最も考慮する必要があるのは実装結合だ。図4-1に示すように、複数のスキーマ間でデータベースを共有するケースが多いからだ。

1つのデータベースを複数のサービスで共有することは、目に見えて多くの問題がある。何よりも大きな問題は、何を共有して何を隠すのかを決める機会を自ら捨てるということだ。それは、情報隠蔽へと向かう動きと真っ向から反する。つまり、スキーマのどの部分を安全に変更できるかを理解するのが難しくなる。外部のサービスがデータベースにアクセスできることが分かったとしても、スキーマのどの部分が使用されているかは分からないからだ。この問題はビューを使用することである程度は緩和できるが、この後説明するように、それは完璧な解決策にはならない。

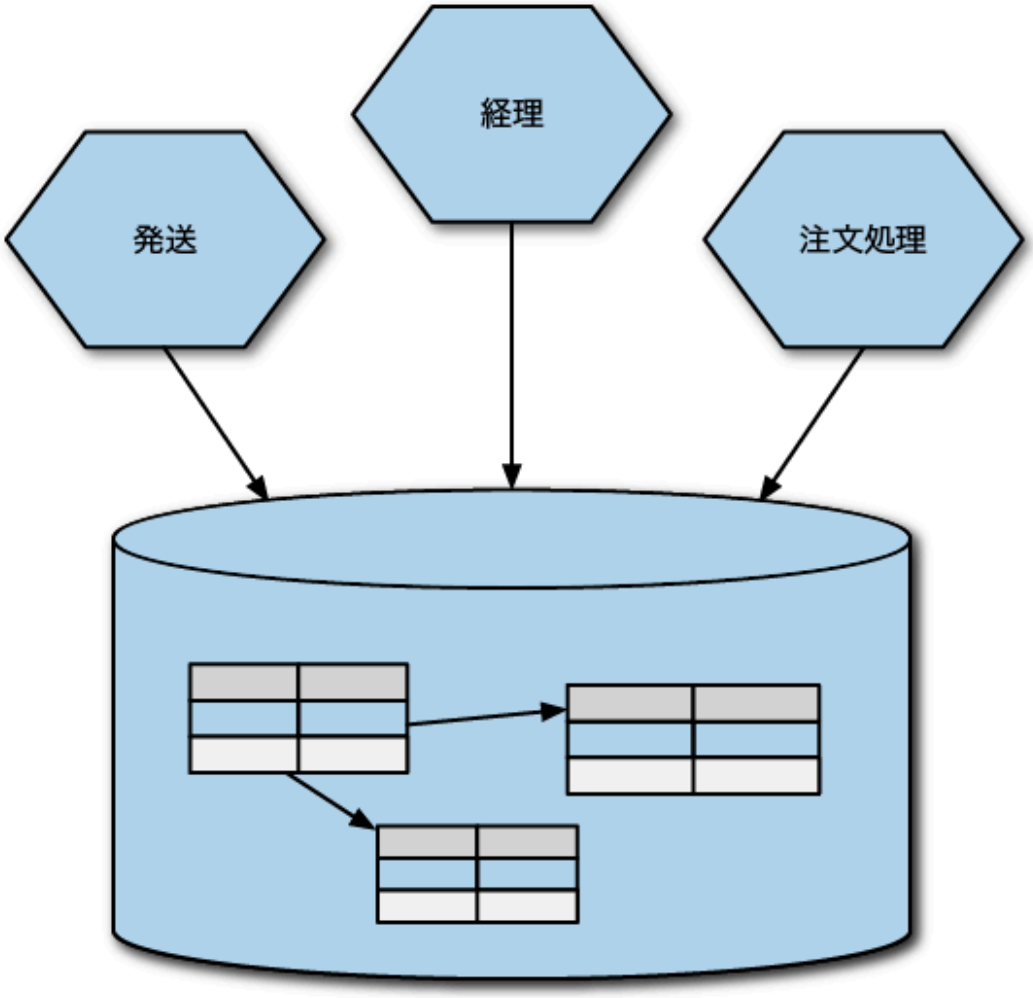


図4-1　同じデータベースに直接アクセスする複数のサービス

他には、誰がデータを「管理」するのかが不明瞭になるという問題がある。このデータを操作するビジネスロジックはどこにあるのか？　それは複数のサービスに分散しているか？　これはビジネスロジックの凝集度が欠如していることを意味する。マイクロサービスを振る舞いと状態の組み合わせだと考えるなら、マイクロサービスは1つ以上の状態マシンをカプセル化しているということになる。もし、その状態を変更する振る舞いがシステム全体に散らばってい

るとすると、状態マシンを適切に実装できているかを確認するのは厄介な問題となる。

図4-1に示すように、注文情報を直接変更できる3つのサービスがあるとする。このときに、サービス間で注文情報を変更する振る舞いに一貫性がなかったとしたら、どうなってしまうだろうか？　その振る舞いを変更する必要がある場合には、3つのサービス全てに変更を適用しなければならないだろうか？　既に述べたように、私たちが目指しているのは、ビジネス機能の凝集度を高めることだ。しかしながら、共有データベースはその逆を意味することが多い。

4.1.1　対処パターン

難しい作業にはなるだろうが、たいていの場合は、各マイクロサービスが独自に所有できるよう、データベースを分割するのが望ましい。もしそれが難しい場合には、「パターン：データベースビュー」を使用するか、「パターン：データベースをラップするサービス」を取り入れることが役に立つ。

4.1.2　使いどころ

共有データベースがマイクロサービスアーキテクチャに適している状況は2つだけだ。1つ目は、読み取り専用の静的参照データを考える場合だ。このトピックについては後で詳しく説明するが、国の通貨コード情報や郵便番号の参照テーブルなどを保持しているスキーマを考えてみてほしい。こうしたスキーマのデータ構造はとても安定しており、通常、こうしたデータの変更制御は管理タスクとして処理される。

もう一つ、複数のサービスが同じデータベースに直接アクセスするのが適切な状況がある。それは、複数のコンシューマーが使用する目的で設計・管理された定義済みエンドポイントとして、サービスがデータベースを直接公開している場合だ。この考え方については、「パターン：サービスのインターフェイスとしてのデータベース」でさらに詳しく説明する。

4.2　しかし、できない！

新しいサービスにはそれぞれ独立したスキーマを持たせるのが理想ではある。しかし、既存のモノリシックなシステムから始める場合には、そうなってはいない。どんな場合でもこれらのスキーマを分割しなくてはならないのだろう

か？　ほとんどの状況ではこれが適切な行動だと確信しているが、最初からそれが可能とは限らない。

この後で説明するように、時には作業に時間がかかりすぎることや、システムの特にデリケートな部分を変更しなければならないこともある。そのような場合には、さまざまな対処パターンが使用できる。そうしたパターンを使用すれば、少なくとも、事態が悪化するのを防ぐことができ、将来的なより良い状態に向けての賢明な足がかりとなるだろう。

スキーマとデータベース

私は過去に「データベース」と「スキーマ」という用語を同じ意味で使うという罪を犯してきた。これらの用語には曖昧さがあるため、混乱を招く場合がある。技術的に言えば、スキーマとは、データを保持する、論理的に分離されたテーブルの集合だ。そして、複数のスキーマを単一のデータベースエンジン上でホストできる。これを図4-2に示す。文脈によって、人々が「データベース」と言うときに指しているのがスキーマのこともあれば、データベースエンジンのこともある（「データベースがダウンしています！」）。

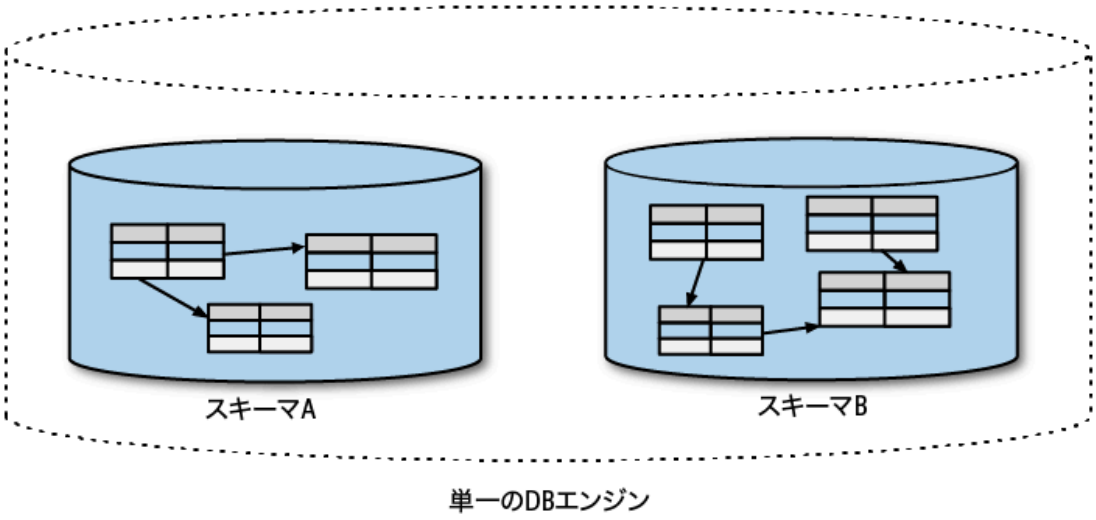


図4-2　データベースエンジンの1インスタンスは複数のスキーマをホストでき、それぞれが論理的な分離を提供する

この章では主に論理的なデータベースの概念に焦点を当てている。また、この文脈では、一般的に「データベース」という用語は論理的に分離されたスキーマに関連付けて使われるため、この章でもそうした意図で「データベース」という用語を用いる。したがって、「データベース」という記載がでてきたら、「論理的に分離されたスキーマ」のことと思ってほしい。明記されていない限り、この章の図ではデータベースエンジンの概念を省略して記載している。

特にクラウドプロバイダーが提供するデータベースを扱う場合、NoSQLデータベース製品がその種類によってスキーマの概念を持っていたり持っていなかったりすることは注意が必要だ。たとえば、AWS上のDynamoDBはテーブルという概念しかなく、ロールベースのアクセス制御を使用して、誰がデータを見たり変更したりできるかを制限している。このような状況ではスキーマをどのように考えるかが課題となるだろう。

現在のシステムで今すぐには対処できないような問題にも遭遇するはずだ。その場合には、チームメンバーと一緒にその問題に対処しよう。そうすれば、たとえどうしたらいいかをあなたが分かっていない場合でも、それがあなたの解決したい問題だということが他のメンバーに伝わる。そうしたら、**その時点で**正しいと考えている対処を始めるようにしよう。時間がたって新しいスキルや経験を積めば、最初は乗り越えられないと思われていた問題も対処しやすくなるはずだ。

4.3　パターン：データベースビュー

複数のサービス向けに単一のデータソースが必要な状況では、結合に関する懸念を軽減するためにビューを使用できる。ビューを使うと、サービスは元のスキーマの限定的な射影をスキーマとして提示できる。この射影によって、サービスに公開するデータを制限でき、サービスがアクセスすべきでない情報を隠蔽できる。

4.3.1　パブリックな契約としてのデータベース

3章で、現在は廃業した投資銀行向けに既存のクレジットデリバティブシステムをリプラットフォームする支援を行った経験について話した。このときには、システム利用者であるトレーダーにより迅速なフィードバックを提供するため、システムのスループットを向上させる必要があった。いくつかの分析を行った結果、処理のボトルネックはデータベースへの書き込みであることが分

かった。そして、技術的検証の結果、スキーマを再構築すれば、システムの書き込みパフォーマンスを大幅に向上させられることに気付いた。

この時点で、私たちの管轄外にある複数のアプリケーションがデータベースへの読み取りアクセスや、場合によっては書き込みアクセスまで行っていることが分かった。しかし、残念ながら、これらの外部システムは全て同じ認証情報でデータベースにアクセスしていた。そのため、どのシステムが何にアクセスしているのかを知るのは不可能だった。20以上のアプリケーションが接続していると推定されたが、これはネットワーク呼び出しに対するいくつかの基本的な解析から導き出されたものだった[†2]。

[†2] ネットワーク解析に頼って誰がデータベースを使っているのかを判断していると、困ったことになる。

それぞれのアクター（人間や外部システムなど）が異なるクレデンシャルのセットを持っていれば、特定の当事者にアクセスを制限したり、クレデンシャルの取り消しやローテーションの影響を軽減したり、各アクターが何をしているのかを理解したりするのがずっと容易になる。異なるクレデンシャルのセットを管理することは、特にサービスごとに管理するクレデンシャルのセットが複数あるマイクロサービスシステムでは、苦痛になることがある。私はこの問題を解決するために専用のシークレットストアを使用するのが好みだ。HashiCorpのVault[†3]は、アクターごとの短命で範囲が限定されたクレデンシャルを、データベースのようなもののために生成できるので、この分野では優れたツールだ。

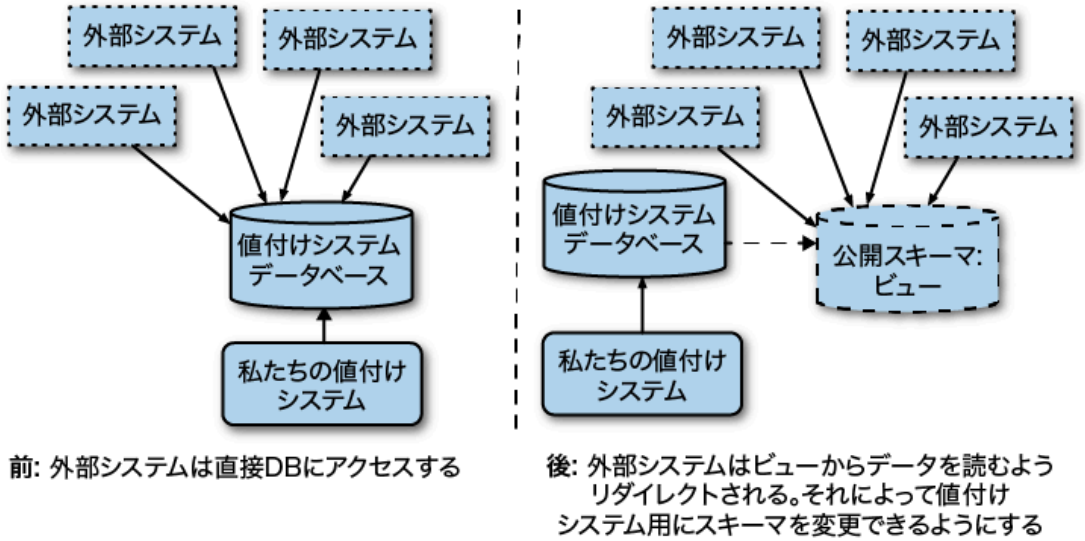
[†3] https://www.vaultproject.io

このため、誰なのかも分からないまま、私たちはこれらのユーザーに連絡を取らなければならなかった。最終的には、誰かのアイデアに従って、使われていた共有アカウントを無効にし、苦情の連絡が来るのを待つことにした。そもそもあってはならない問題に対して、これは明らかに優れた解決策ではなかったが、大体はうまくいった。しかしすぐに、これらのアプリケーションのほとんどがアクティブなメンテナンスを受けていないことが分かった。これは、新しいスキーマ設計を反映するための更新が行われる可能性がないことを意味していた[†4]。事実上、私たちのデータベーススキーマはパブリックな契約となっていたため変更は不可能だった。そのため、スキーマ構造を継続して維持しなければならなかった。

[†4] データベースを使用しているシステムの1つはPythonベースのニューラルネットだという噂だったが、誰も理解できるメンバーはおらず、「ただ動いていた」。

4.3.2　提示用のビュー

私たちは、まず外部システムがスキーマに書き込んでいる状況を解決する必要があった。幸いにも、これは簡単に解決できた。そして、データを参照したいクライアント向けに、古いスキーマのように見えるビューをホスティングする専用のスキーマを作成し、クライアントにそのスキーマを指定してもらうようにした（図4-3参照）。これによって、ビューを維持できる限り、私たちは独自のスキーマを変更できるようになった。ここでは、この実現にはたくさんのストアドプロシージャを必要としたとだけ言っておこう。



前: 外部システムは直接DBにアクセスする

後: 外部システムはビューからデータを読むようリダイレクトされる。それによって値付けシステム用にスキーマを変更できるようにする

図4-3　ビューを使用することで、基になるスキーマを変更できるようにする

投資銀行の事例では、ビューと基礎となるスキーマがかなり異なっていた。もちろん、ビューをもっとシンプルに使用することもできる。単純な例を示そう。図4-4のポイントサービスは、システムで管理するポイントカードの一覧を提供するだけのサービスだった。この情報は顧客テーブルのカラムに格納されている。そこで、顧客テーブルの他の情報を公開せずに、単一のテーブルで顧客IDとポイント会員IDのマッピングだけを公開するビューを定義した。顧客テーブルと同様、モノリスのデータベースにある他のテーブルも、ポイントサービスからは完全に隠されていた。

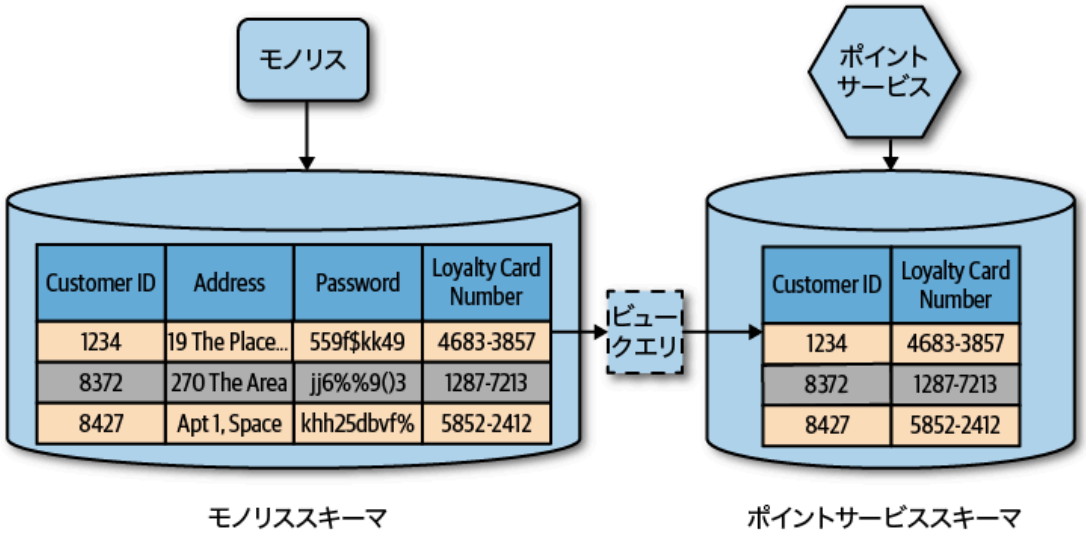


図4-4　基礎となるスキーマのサブセットを射影するデータベースビュー

情報源からの限られた情報のみを射影するビューの機能が、情報隠蔽を可能にする。それによって、何を共有し、何を隠すかが制御できる。しかし、これは完全な解決策ではない。

データベースの性質によっては、マテリアライズドビューを作成する選択肢がある。マテリアライズドビューでは、ビューの内容は事前に計算される。ビューの参照が基礎となるスキーマの参照を必要としないため、この場合パフォーマンスは向上する。トレードオフは、事前に計算されたビューがどうやって更新されるかにある。これは、ビューから「古い世代の」データセットを読み取っている可能性があることを意味する。

4.3.3　制限事項

ビューの実装方法はさまざまだが、一般的にはクエリの結果から生成される。これはビュー自体が読み取り専用であることを意味する。これはそのまま有用

性の制限になる。さらに、ビューはリレーショナルデータベースでは一般的な機能であり、比較的成熟したNoSQLデータベースの多くもサポートしている機能であるものの（たとえばCassandraやMongoDBもサポートしている）、全てのデータベース製品がビューをサポートしているわけではない。データベースエンジンがビューをサポートしていても、ソーススキーマとビューの両方を同じデータベースエンジンに入れなければならないなど、他に制限がある可能性がある。これは物理的なデプロイ時結合を増やすことになり、単一障害点を生む可能性がある。

4.3.4　所有権

基礎となるソーススキーマへの変更は、ビューの更新を要求するかもしれない。そのため、誰がビューを「所有しているか」を慎重に検討する必要がある。公開されているデータベースビューは、他のサービスインターフェイスと同じようなものであり、ソーススキーマを管理するチームが最新の状態に保つべきものだと考えたほうが良いだろう。

4.3.5　使いどころ

私は通常、既存のモノリシックスキーマを分解することが現実的ではないと考える状況においてデータベースビューを使用する。最終的な目的がサービスインターフェイスを介して情報を公開することであるならば、可能な限りビューの使用を避けるようにするのが理想だろう。そのような場合には、代わりに適切なスキーマ分解を進めるべきだ。このテクニックの制限事項は大きいかもしれない。それでも、スキーマを分解する作業は労力が大きすぎると感じているのであれば、このテクニックは正しい方向への一歩になるだろう。

4.4　パターン：データベースをラップするサービス

何かを処理するのがあまりに難しい場合には、混乱を隠してしまうのが理にかなうこともある。**データベースをラップするサービス**は、まさにそれを行うパターンだ。このパターンでは、データベースを薄いラッパーとして機能するサービスの後ろに隠し、データベースへの依存関係をサービスへの依存関係に移動する。これを図4-5に示す。

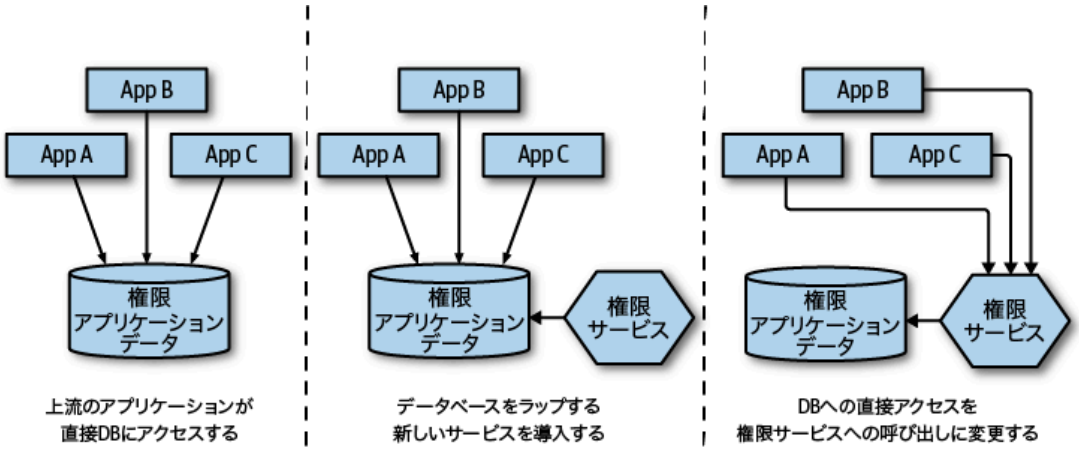


図4-5　サービスを使用してデータベースをラップする

数年前、オーストラリアにある大手銀行の一部組織に対して、プロセス改善の支援を行った時のことだ。その初日、私たちはそこで起きている課題を理解して、プロセスがどうなっているかを確かめるため、キーパーソンとの面談をいくつか設定した。そのミーティングの合間、ある人が現れて、会社のその領域の主任DBAであると自己紹介した。そして彼はこう言った。「お願いだから、どうかデータベースに物を入れるのを止めてくれ！」

コーヒーを飲みながら、そのDBAは問題点を説明してくれた。30年の歳月をかけて、銀行の業務システムは、組織の王冠をかざる宝石の一つとして形作られてきた。このシステムのより重要な役割の一つは「権限」の管理だった。銀行業務では、どの個人がどの口座にアクセスできるかを管理し、それらの口座で何をできるかを調べることは、非常に複雑だった。それらの権限がどうなっているのかを理解するために、A社、B社、C社の口座を表示することが許可されている簿記係を考えてみよう。簿記係はB社では口座間で500ドルまでの転送ができ、C社では口座間で無制限の転送と250ドルまでの引き出しを行える。これらの権限のメンテナンスと適用は、データベースのストアドプロシージャでほぼ独占的に管理されていた。全てのデータアクセスは、この権限ロジックを経由して行われていた。

銀行が規模を拡大し、ロジックと状態の量が増加するにつれて、データベースは負荷に耐えられなくなった。DBAが言うには「オラクルにできる限りの金銭を支払ってきたが、それでもまだ十分ではない」状態だった。予測される成長を考えると、ハードウェアの性能向上を考慮したとしても、最終的には組織のニーズがデータベースの能力を上回るような状況になるのではないかという懸念があった。

この問題をさらに調査する中で、スキーマの一部を分割して負荷を軽減するというアイデアについて議論した。 問題は、こうしたこと全ての真ん中にある絡み合った巣が、この権限システムであるということだった。一歩間違えれば、誰かが自分の口座へのアクセスをブロックされてしまう可能性や、さらに悪いことには、アクセスすべきではない誰かがあなたのお金にアクセスできるようになってしまう可能性があった。

私たちは、この状況を解決するための計画を立てた。まず、短期的には権限システムが変更できなくなることを受け入れた。少なくともこれ以上問題を悪化させないことが急務だったからだ。私たちは、そのために人々がより多くのデータや振る舞いを権限スキーマに入れるのを止める必要があった。いったんそれを実施できれば、権限スキーマから抽出しやすい箇所を取り除くことを検討でき、うまくいけばシステムの寿命に関する懸念を軽減できるほど負荷を減らせられると考えた。それによって、次のステップを検討するための余裕を生む計画だった。

問題のあるスキーマを「隠す」ことを可能にする新しい権限サービスの導入について、私たちは議論した。データベースは既にストアドプロシージャの形で多くの動作を実装されていた。そのため、新しいサービスを導入したとしても、そのサービスに実装できる振る舞いはほとんどなさそうだった。しかし、目標は、他のアプリケーションを作成するチームが、権限スキーマを他の誰かのものと考えて、自身のデータをローカルに保存するよう促すことにあった。そのため、図4-6に示すように、新しい権限サービスが導入された。

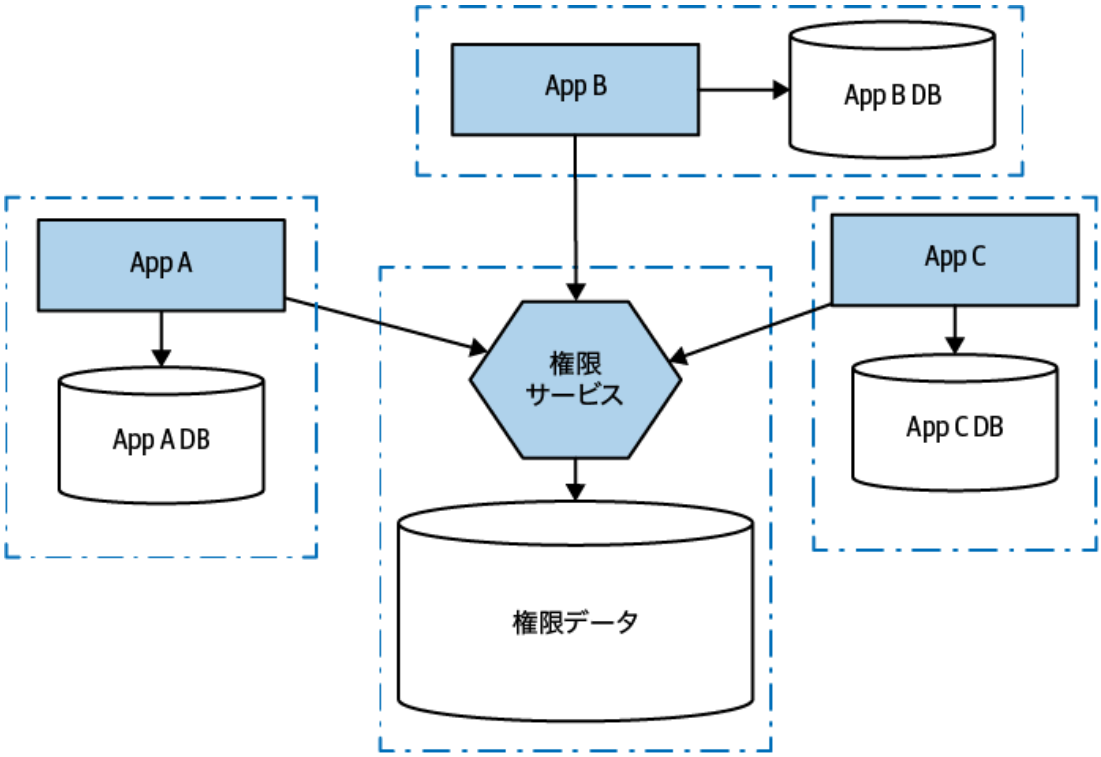


図4-6　データベースをラップするサービスパターンを使用して、中央のデータベースへの依存を軽減する

「パターン：データベースビュー」と同様に、ラッピングサービスを使用すると、何を共有し、何を隠すかを制御できる。ラッピングサービスは、状況を改善するために内側で変更を行っている間も、安定したインターフェイスをコンシューマーに提供する。

4.4.1　使いどころ

このパターンは、基礎となるスキーマを分解するのがあまりにも厳しい場合にとてもうまく機能する。スキーマの周りに明示的なラッパーを配置し、そのラッパーを介してのみデータにアクセス可能だと明確にすることで、少なくともデータベースがこれ以上大きくなるのを止められる。何が「あなたのもの」で何が「他人のもの」なのかが明確に区別される。このアプローチは、基礎となるスキーマとサービス層の両方の所有権を同じチームに持たせる場合にも最適だ。サービスAPIは、API層がどのように変化するかの適切な監視を受けながら、管理されたインターフェイスとして適切に利用される必要がある。このアプローチは、上流のアプリケーションにとっても、下流のスキーマをどのように使用しているかをより簡単に理解できるというメリットがある。それにより、テスト目的のスタブなどのアクティビティがより管理しやすくなるからだ。

このパターンには、「パターン：データベースビュー」よりも多くの利点がある。まず、既存のテーブル構造にマッピングされるビューを提示するということに制約されない。ラッピングサービスにコードを記述することで、基礎となるデータに対してより洗練された射影を提示することも可能だ。ラッピングサービスは、(API呼び出しを介して)書き込みを行うこともできる。もちろん、このパターンを採用すると、上流のコンシューマーはDBへの直接アクセスからAPI呼び出しへと移行しなければならないため、変更が必要になる。

理想的には、このパターンを使用することで、より根本的な変更への足がかりが得られ、API層の下にあるスキーマを分解する時間が生まれるだろう。根本的な問題に対処するのではなく、問題を包帯で覆っているだけだとも言える。しかし、段階的な改善を行うという精神からすると、このパターンには多くの利点があると考える。

4.5　パターン：サービスのインターフェイスとしてのデータベース

クライアントは読み取り専用のデータベースを必要とすることがある。それは、大量のデータを参照したり取得したりする必要があるからかもしれないし、外部関係者がSQLエンドポイントを必要とするツールチェーンを既に使用しているからかもしれない（ビジネスメトリクスのインサイトを得るためによく使用されるTableauのようなツールを考えるとよいだろう）。このような状況では、サービスがデータベースで管理しているデータをクライアントが閲覧できるようにすることは理にかなってはいる。しかし、公開するデータベースとサービス境界の内側で使用するデータベースを分離するように注意しなければならない。

私が見てきたアプローチの1つは、読み取り専用のエンドポイントとして公開するように設計された専用のデータベースを作成し、基礎となるデータベースのデータが変更されたときに、その変更を読み取り専用のデータベース側に反映するというものだ。サービスがイベントのストリームと同期APIをそれぞれエンドポイントとして公開するのと同じように、外部のコンシューマーにデータベースを公開することも可能だ。図4-7は、APIを介して読み取りと書き込みのエンドポイントを公開し、読み取り専用のインターフェイスとしてデータベースも公開している注文サービスの例を示している。マッピングエンジンは、内部データベースの変更を受けて、外部データベースでどのような変更を行う必要があるかを計算する。

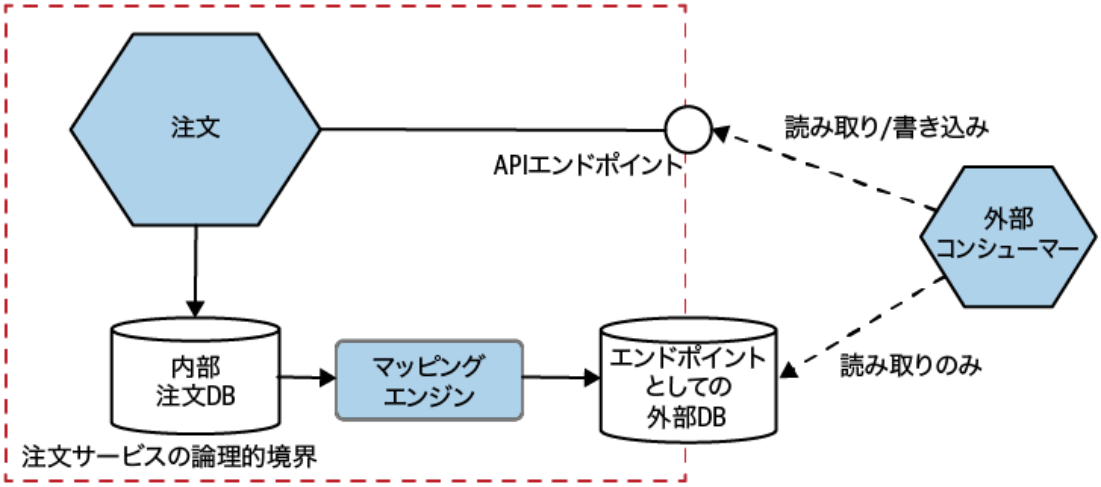


図4-7　専用のデータベースをエンドポイントとして公開することで、内部データベースを隠したままにできる

マッピングエンジンは、変更を完全に無視したり、変更を直接公開したり、またはその中間のようなことを行える。重要なのは、マッピングエンジンが内部データベースと外部データベースの間の抽象レイヤーとして機能するということだ。内部データベースの構造が変更された場合、公開されているデータベースが一貫性を保つようにマッピングエンジンを変更する必要がある。マッピングエンジンを使うと、実装の仕方にもよるが、事実上全てのケースで内部データベースへの書き込みが反映されるまでに遅延が生じる。そのため、公開されたデータベースからデータを読み込んでいるクライアントは、古いデータを見ている可能性があることを理解する必要があり、外部データベースの最終更新日に関する情報をプログラム的に公開することが適切な場合もある。

レポーティングデータベースパターン

このパターンは、Martin Fowlerが既に**レポーティングデータベース**パターン[†5]として文書化しているものだ。では、なぜここで違う名前を使ったのだろうか？　それは、より多くの人と話をするうちに、レポーティングはこのパターンの一般的な応用ではあるものの、このテクニックを使う理由はそれだけではないことに気が付いたからだ。クライアントがアドホッククエリを定義できるようにする機能は、従来のバッチ指向のワークフローよりも広い範囲をカバーしている。そこで、このパターンはおそらくこれらのレポーティングのユースケースをサポートするために最も広く使われているものの、より広い適用範囲があるかもしれないという事実を反映するために、ここでは別の名前を採用することにした。

[†5] https://martinfowler.com/bliki/ReportingDatabase.html（日本語訳：https://bliki-ja.github.io/ReportingDatabase/）[11]

4.5.1　マッピングエンジンを実装する

このパターンの細部は、更新方法、つまりマッピングエンジンをどのように実装するかという点にある。既に説明した「パターン：変更データキャプチャ」によるシステムは、ここでの優れた選択肢となるだろう。実際、変更データキャプチャは最新のビューを提供しながら、最も堅牢なソリューションでもある可能性が高い。他にはバッチ処理でデータをコピーするという選択肢もある。しかし、このアプローチは内部データベースと外部データベースの間に大きなずれが生じる可能性があり、スキーマによってはコピーする必要があるデータを決定するのが困難な場合があるため、問題となる。第三の選択肢は、問題のサービスから発生したイベントを待ち受け、それを使って外部データベースを更新することだ。

以前であれば、私はバッチジョブを使ってこれを処理していた。しかし今なら、Debezium[†6]のような専用の変更データキャプチャシステムを利用するだろう。私は、バッチプロセスが実行されなかったり、実行に時間がかかりすぎたりすることに何度も悩まされてきた。世界はバッチジョブから離れ、より速くデータを求めるようになってきており、バッチはリアルタイムへの道を歩んでいる。これを処理するために変更データキャプチャシステムを導入することは理にかなっている。特に、サービス境界の外部にイベントを公開するためにその使用を検討しているなら、なおさらだ。

[†6] https://github.com/debezium/debezium

4.5.2　ビューとの比較

このパターンは、「パターン：データベースビュー」よりも洗練されている。データベースビューは通常、特定の技術スタックに結び付けられている。たとえば、Oracleデータベースのビューを表示する場合、基礎となるデータベースとビューをホストするスキーマの両方がOracle上で実行される。しかし、このパターンでは、公開するデータベースは全く異なる技術スタックになる可能性

がある。サービスの内部でCassandraを使用し、従来のSQLデータベースをパブリックエンドポイントとして表示できる。

データベースビューよりも柔軟性がある代わりに、このパターンには追加コストがかかる。コンシューマーのニーズがデータベースビューで満たされるのであれば、最初の実装は少なく済む可能性が高い。ただ、このインターフェイスをどのように進化させるかに制限がかかる可能性があることに注意する必要がある。データベースビューの使用から始めて、後でレポーティング専用のデータベースへの移行を検討することも可能だ。

4.5.3　使いどころ

エンドポイントとして公開されるデータベースは読み取り専用なので、読み取り専用のアクセスを必要とするクライアントにのみ有用だ。これはレポーティングのユースケースに非常に適している。たとえば、あるサービスが保持する大量のデータをクライアントが結合する必要がある場合などだ。このアイデアを拡張して、データベースのデータをより大きなデータウェアハウスにインポートし、複数のサービスからデータを参照することも可能だ。このアイデアについては、『マイクロサービスアーキテクチャ』[25]の5章で詳しく説明している。

この外部データベースの射影を適切に最新の状態に保つために必要な作業を過小評価してはいけない。現在のサービスがどのように実装されているかにもよるが、これは複雑な作業になる可能性がある。

4.6　所有権を移す

ここまで、私たちはまだ根本的な問題に取り組んではいない。共有された巨大なデータベースにさまざまな包帯を巻いただけだ。巨大なモノリシックデータベースからデータを取り出すというトリッキーな作業を考える前に、問題のデータが実際にどこにあるべきかを考える必要がある。モノリスからサービスを分離する際には、データの一部は一緒に分離すべきであり、一部はそのままにしておくべきだ。

1つ以上の集約に関連するロジックをマイクロサービスにカプセル化する場合には、その状態と関連データの管理をマイクロサービス独自のスキーマに移す必要がある。その一方で、マイクロサービスが引き続きモノリスによって所有される集約と対話しなくてはならない場合には、適切に定義されたインターフ

ェイスを介してそれを行えるようにする必要がある。ここでは、それら2つの選択肢を見ていこう。

4.6.1　パターン：集約を公開するモノリス

図4-8に示す新しい請求書サービスは、請求書の管理に直接関係のないさまざまな情報にアクセスする必要がある。少なくとも、承認ワークフローを管理するには、現在の従業員に関する情報が必要だ。このデータは、現在のところ全てモノリスのデータベース内にある。従業員に関する情報をモノリス自体のサービスエンドポイント（APIやイベントストリーム）を介して公開することで、私たちは請求書サービスが必要とする情報を明確にしている。

マイクロサービスは振る舞いと状態の組み合わせだ。ドメイン集約を管理する1つ以上の状態マシンを含むものとしてマイクロサービスを捉えるという考えは既に説明した。モノリスから集約を公開する場合についても同じように考えたい。モノリスは依然として何が許容される状態の変化で何がそうでないかという概念をまだ「所有」している。私たちはモノリスをデータベースの単なるラッパーのように扱いたいわけではない。

このパターンでは、単にデータを公開するだけでなく、集約の現在の状態を参照したり、新たな状態遷移を要求したりする操作を外部に公開する。さらに、サービス境界の外部に公開する集約の状態を限定したり、外部から要求できる状態遷移操作を制限したりといったことも可能だ。

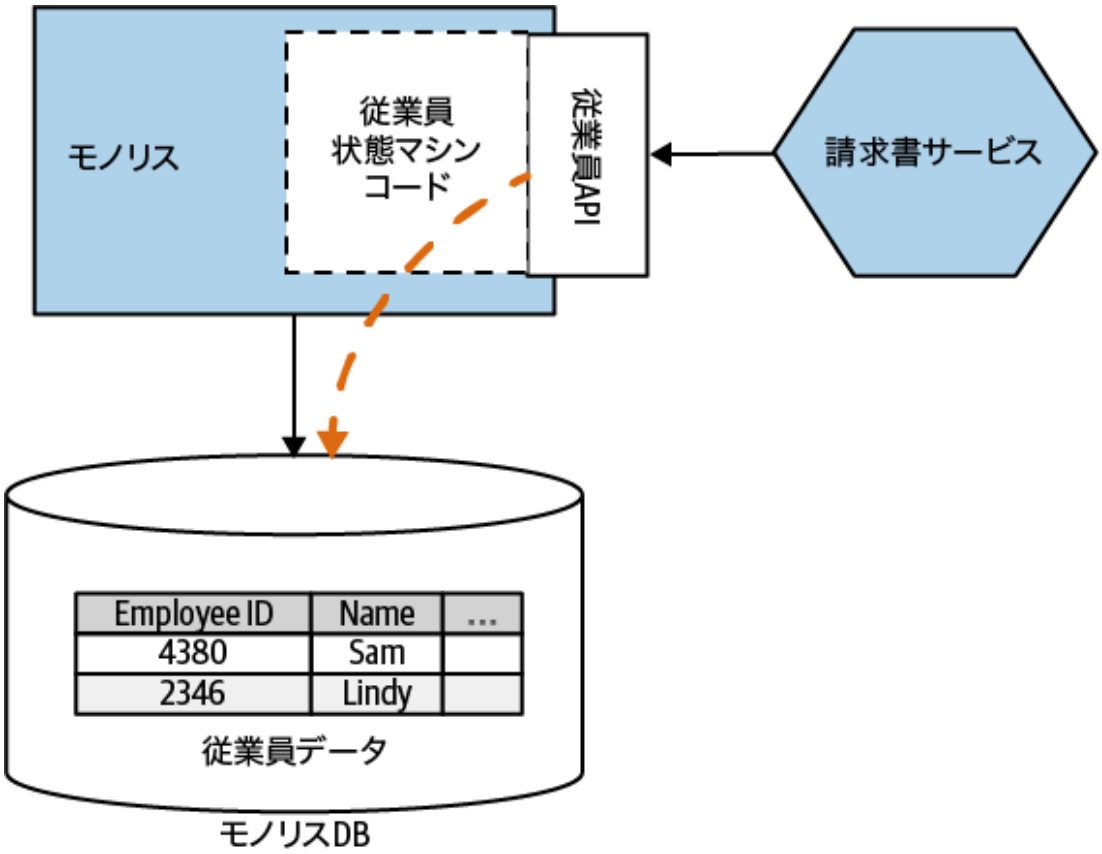


図4-8　適切なサービスインターフェイスを介してモノリスから情報を公開し、新しいマイクロサービスがそれにアクセスできるようにする

より多くのサービスへの道筋として

請求書サービスのニーズを明らかにし、必要な情報を適切に定義されたインターフェイスで明示的に公開することで、将来のサービス境界を発見できる可能性がある。この例では、図4-9で示すように、次のステップとして従業員サービスを抽出している。従業員データに関するAPIを公開したことで、新しい従業員サービスに対するコンシューマーのニーズが何であるかを理解するのに大きく前進した。

もちろん、モノリスから従業員サービスを抽出したとしても、モノリスが引き続きその従業員データを必要としている場合には、その新しいサービスを使用するようにモノリスの側を変更する必要があるだろう。

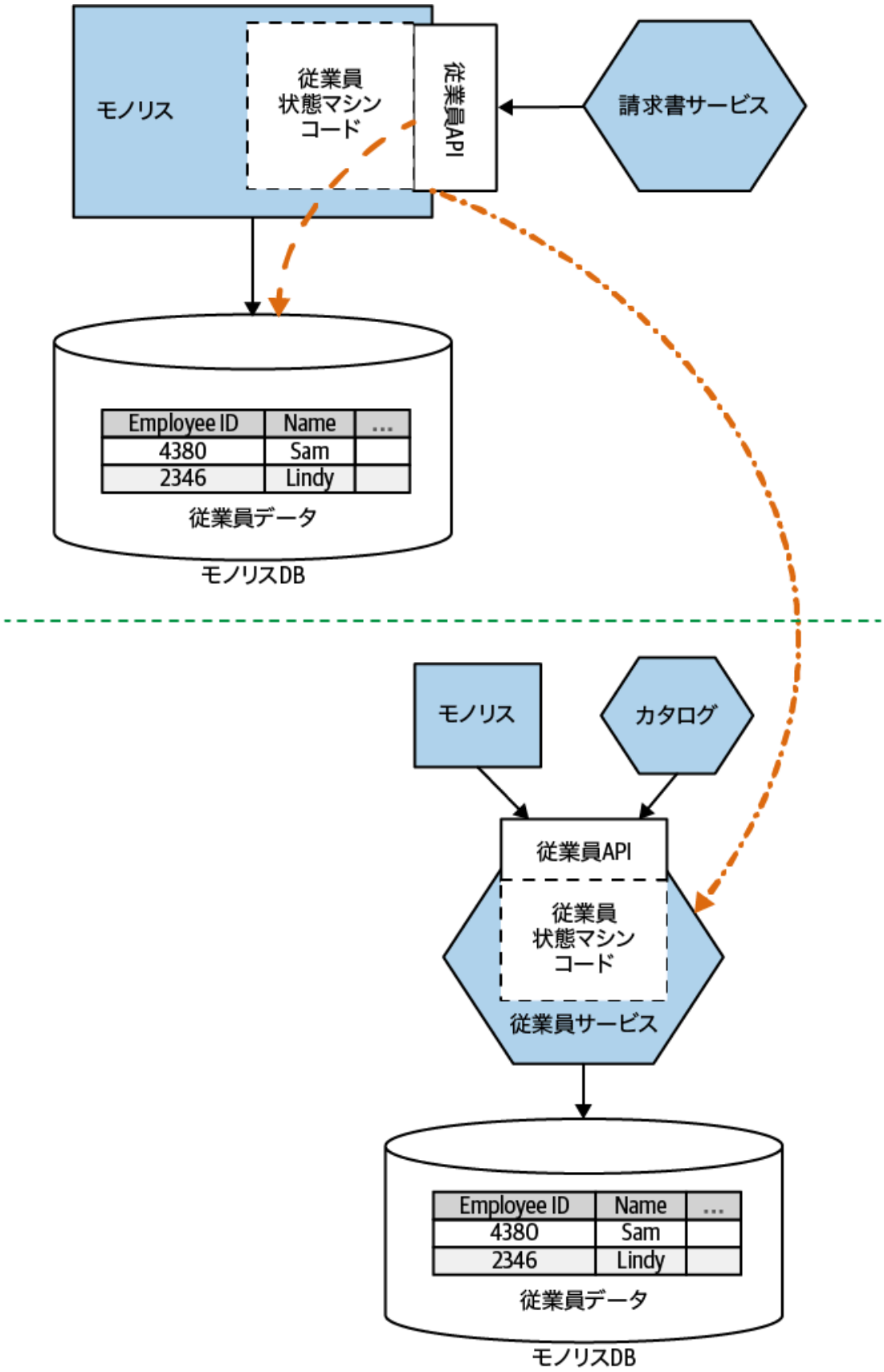


図4-9　既存のエンドポイントのスコープを使用して、新しい従業員サービスの抽出を行う

使いどころ

アクセスしたいデータが依然としてデータベースに「所有」されている場合には、新しいサービスが必要とするアクセスを可能にするために、このパターンがうまく機能する。サービスを抽出する場合に、新しいサービスがモノリスにコールバックして必要なデータにアクセスできるようにすることは、モノリスのデータベースに直接アクセスするよりも若干手間がかかる。しかし、長期的にはその方がはるかに良いアイデアだ。新しいエンドポイントを公開するようにモノリスを変更できない場合にのみ、私は「パターン：データベースビュー」の使用を検討する。そのような場合には、モノリスのデータベース上のビューを使う方がうまく機能する。あるいは、既に説明した「パターン：変更データキャプチャ」や「パターン：データベースをラップするサービス」を作成して、必要な従業員情報を公開することも可能だ。

4.6.2　パターン：データ所有権の変更

「パターン：集約を公開するモノリス」では、請求書サービスがモノリスの従業員データにアクセスする場合のように、新しいサービスが他の機能の所有するデータにアクセスする必要がある場合にどうなるかを検討した。では、モノリスにあるデータが、新たに抽出したサービスによって管理されるべきデータだった場合にはどうなるだろうか？

図4-10に、必要となる変更の概要を示す。まず、データのライフサイクルを管理する必要があるため、請求書関連のデータをモノリスから新しい請求書サービスに移す必要がある。次に、モノリスを変更し、請求書サービスを請求書関連データの真の情報源として扱い、請求書サービスのエンドポイントを呼び出してデータを読み取ったり変更を要求したりするように変更する必要がある。

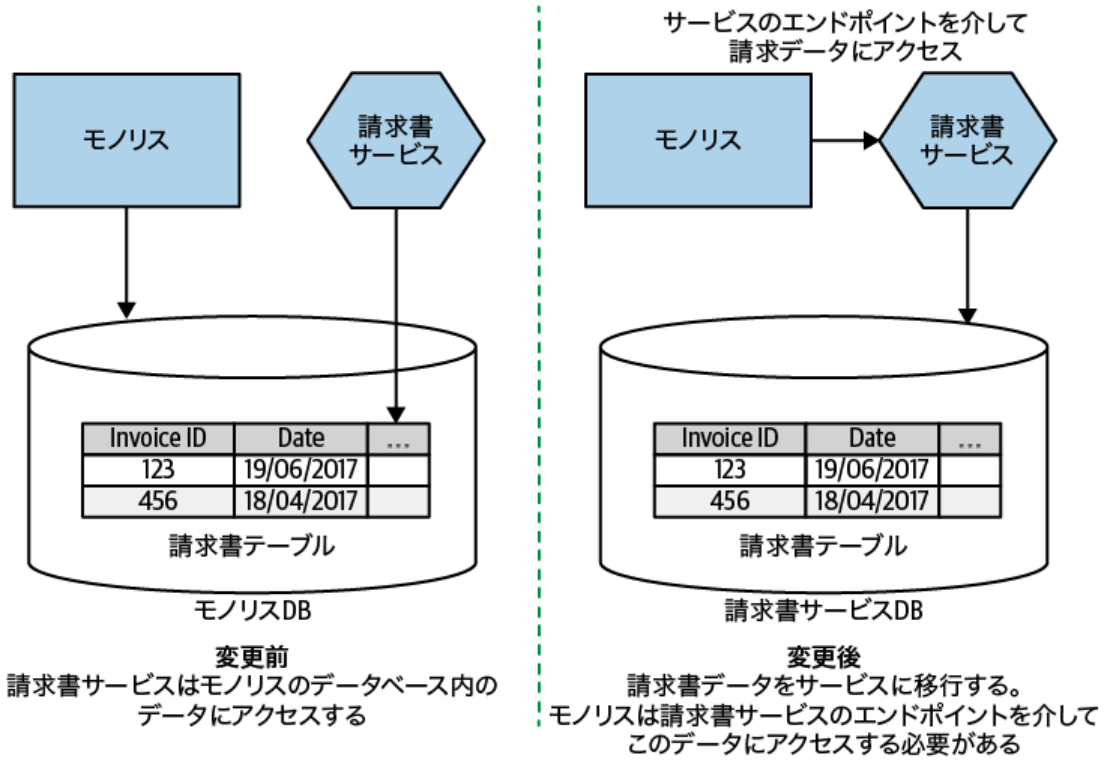


図4-10　新しい請求書サービスは関連データの所有権を持つ

しかし、既存のモノリシックデータベースから請求書データを取り外すことは複雑な問題となるだろう。この章で後ほど説明する外部キー制約の解除やトランザクション境界の解除などの影響を考慮しなければならないかもしれない。請求書関連データへの読み取りアクセスだけを必要とするようにモノリスを変更できるなら、図4-11に示すように、請求書サービスのデータベースからビューを射影することも考えられるかもしれない。しかし、この場合にはデータベースビューの持つ全ての制約が適用されることになる。そのため、モノリスを変更して新しい請求書管理サービスを直接呼び出すようにするのが望ましいだろう。

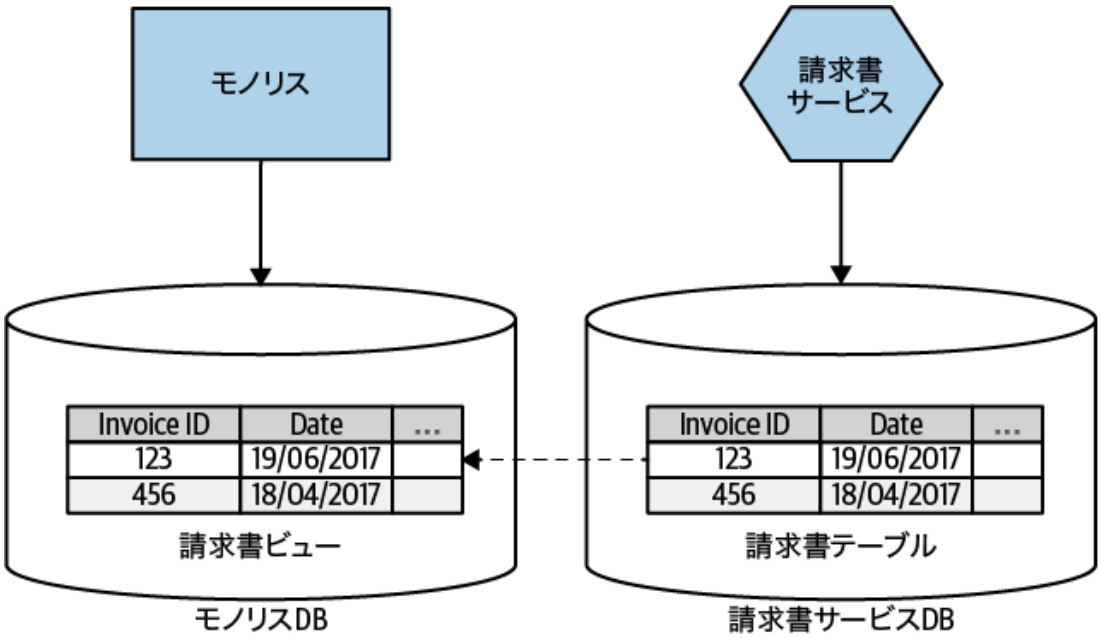


図4-11　請求書データをビューとしてモノリスに射影する

使いどころ

こちらの方が「パターン：集約を公開するモノリス」よりももう少し分かりやすい。新たに抽出したサービスがデータを変更するビジネスロジックをカプセル化している場合には、そのデータは新しいサービスの制御下にあるべきだ。そうすると、そのデータは現在の場所から新しいサービスに移す必要がある。もちろん、既存のデータベースからデータを移すプロセスは単純ではない。実際、この章の残りの部分はそのことに焦点を当てている。

4.7　データ同期

3章で説明したように、ストラングラーパターンなどの利点の1つは、新しいサービスに切り替えて問題が発生した場合、また元に戻せるところにある。問題は、そのサービスがモノリスとの間で同期を保つ必要のあるデータを管理している場合に発生する。

この例を図4-12に示す。新しい請求書サービスへの切り替えを進めているところだが、新しいサービスとモノリス内にある既存機能のコードの両方で、請求関連のデータを管理している。実装を切り替える能力を維持するには、両方のコードが同じデータを参照でき、このデータが一貫した方法で維持されるようになっている必要がある。

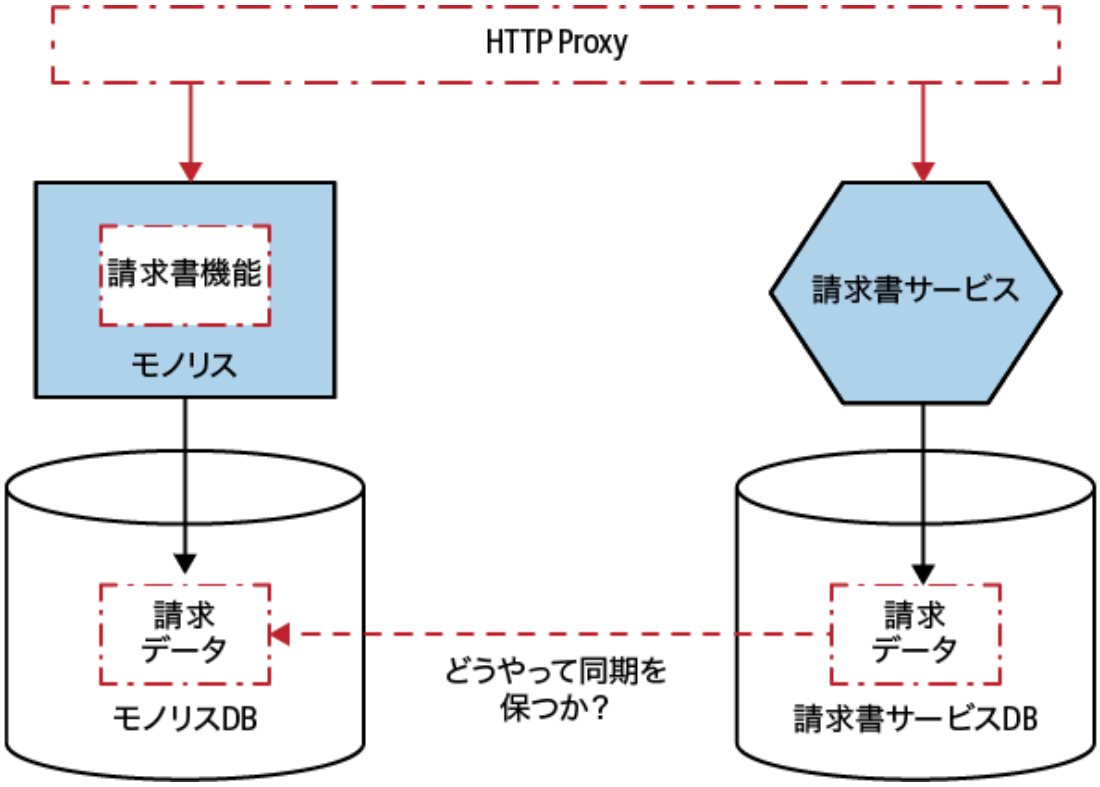


図4-12　ストラングラーパターンを使用して新しい請求書サービスに移行したいが、サービスは状態を管理している

こうした場合にはどのような選択肢があるだろうか？　まず、2つのビュー間でデータの一貫性を保つ必要があるかどうかを考える必要がある。どちらかのコードセットが請求データの完全に一貫したビューを常に表示する必要がある場合には、データを1箇所で確実に保持するのが最も簡単なアプローチだ。これは、「パターン：データベースビュー」で説明したように、新しい請求書サービスがモノリスからデータを直接読み取るという方法につながるだろう。切り替えが成功したことを確認したら、「パターン：データ所有権の変更」で説明したような、データの移行が可能となる。しかし、共有データベースの使用については懸念しすぎることはない。データベースの共有は、より完璧な抽出の一環として、短い間行う手段としてだけ検討すべきだ。共有データベースを長く放置しておくと、長期的な痛みにつながる可能性があるからだ。

もし、アプリケーションコードとデータを同時に移行するような大規模な切り替えを行うのであれば（私ならこれは避けたい）、新しいマイクロサービスに切り替える前に、バッチ処理を使ってデータをコピーできる。請求関連のデータが新しいサービスにコピーされたら、トラフィックを流し始められる。しかし、既存のモノリシックシステムの機能を使用する必要がある場合はどうなるだろう？　マイクロサービスのスキーマで変更されたデータは、モノリシックデータベースの状態には反映されないため、状態が失われることになる。

他には、コードを介して2つのデータベースを同期させておくというアプローチも考えられるだろう。つまり、モノリスと新しい請求書サービスのどちらかが両方のデータベースに書き込みを行うようにするということだ。これを行うには、やや慎重な検討が必要となる。

4.8　パターン：アプリケーションでのデータ同期

ある場所から別の場所にデータを移す作業は、どんなに恵まれた状況であっても複雑になる。さらに、そのデータの価値が高いほど、それは困難となる。たとえば診療記録の管理について考え始めたなら、データの移行方法をどうするかを慎重に考えることはずっと重要になる。

数年前のことだ。コンサルタント会社のTrifork社は、デンマーク市民の診療記録を統合するプロジェクトに参加していた[†7]。このシステムの初期バージョンは、MySQLのデータベースにデータを保存していた。しかし、時がたつにつれて、そのままではシステムが直面する課題に適さない可能性があることが明らかとなってきた。そこで、代替のデータベースとしてRiakを採用することが決定された。Riakを採用することで、システムが予想される負荷を処理するための拡張性が向上するだけでなく、回復性も向上することが期待されていた。

[†7] Kresten Krab Thorup "Riak on Drugs (and the Other Way Around)" (http://bit.ly/2m1CvLP). [33]

既存システムは1つのデータベースにデータを保存していたが、システムをオフラインにできる時間には制約があり、データが失われないことが極めて重要だった。そのため、データを新しいデータベースに移すだけでなく、移行を検証する仕組みを構築することと途中で素早くロールバックできる仕組みを持つことが必要だった。

そこで、アプリケーションによって2つのデータソース間での同期を実行することが決定された。その方法は次のようなものだった。最初のうちはMySQLのデータベースを真の情報源のままとするものの、MySQLとRiakのデータが同期されることをアプリケーションが一定期間保証する。そして、一定期間が

経過した後、MySQLを引退させる前に、Riakをアプリケーションの真の情報源とする。このプロセスをもう少し詳しく見ていこう。

4.8.1　ステップ1：データを一括で同期する

最初のステップは、新しいデータベースにデータのコピーがある状態にすることだ。診療記録プロジェクトでは、新しいRiakデータベースに旧システムからデータのバッチ移行を行った。バッチインポートが行われている間、既存のシステムは動き続けていた。そのため、インポート用のデータは、既存のMySQLシステムから取得したデータのスナップショットだった（図4-13）。ただ、バッチインポートが終了したときにソース元のデータが変更されている可能性があるので、これは課題だった。しかし、元システムをオフラインにするのは現実的ではなかったため、このケースでは仕方のないことだった。

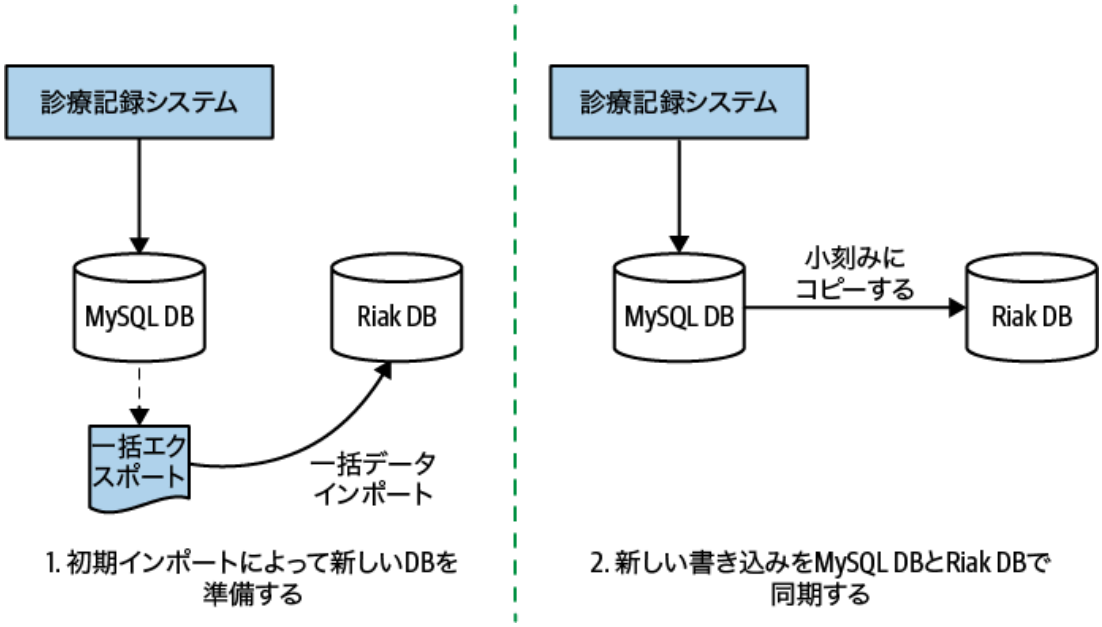


図4-13　アプリケーションベースの同期用に新しいデータストアを準備する

バッチインポートが完了すると、インポート後の変更を適用できるように、「パターン：変更データキャプチャ」の処理が実装された。これによって、Riakとの同期が可能になった。ここまで到達したら、次はアプリケーションの新しいバージョンをデプロイする番だ。

4.8.2　ステップ2：同期のために書き込み、古いスキーマから読み取る

2つのデータベースが同期されるようになったので、全てのデータを両方のデータベースに書き込む新しいバージョンのアプリケーションがデプロイされた

（図4-14）。この段階では、アプリケーションが両方のソースに正しく書き込みを行い、Riakが許容範囲内の振る舞いをしていることを確認するのがゴールだった。引き続き全てのデータをMySQLから読み取ることで、たとえRiakが突然落ちたとしてもMySQLデータベースからデータを取得できることが保証された。

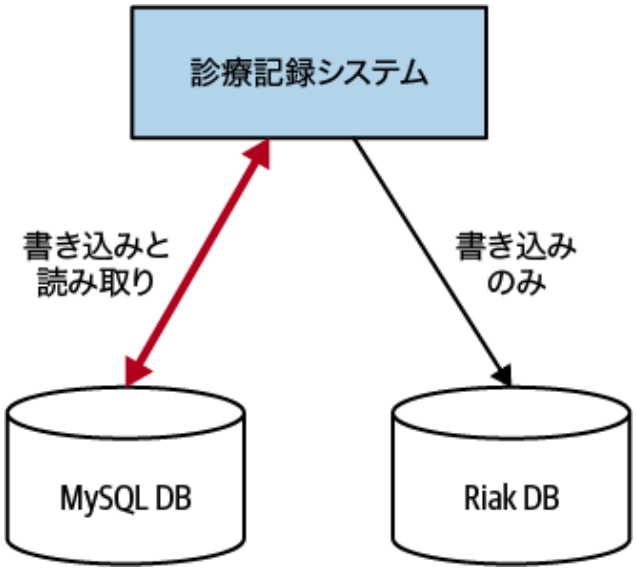


図4-14　アプリケーションは両方のデータベースを同期させておくが、片方は書き込みにのみ使用する

新しいRiakシステムに十分な信頼が築かれると、移行は次のステップへと進んだ。

4.8.3　ステップ3：同期のために書き込み、新しいスキーマから読み取る

ここまでで、Riakへの書き込みが正常に動作することを検証できた。最後のステップは、読み取りも動作するようにすることだ。アプリケーションに簡単な変更を加え、Riakを真の情報源とするようにする（図4-15）。しかしながら、依然として**両方の**データベースに書き込んでいるので、もし問題があればフォールバックするという選択肢は残されている。

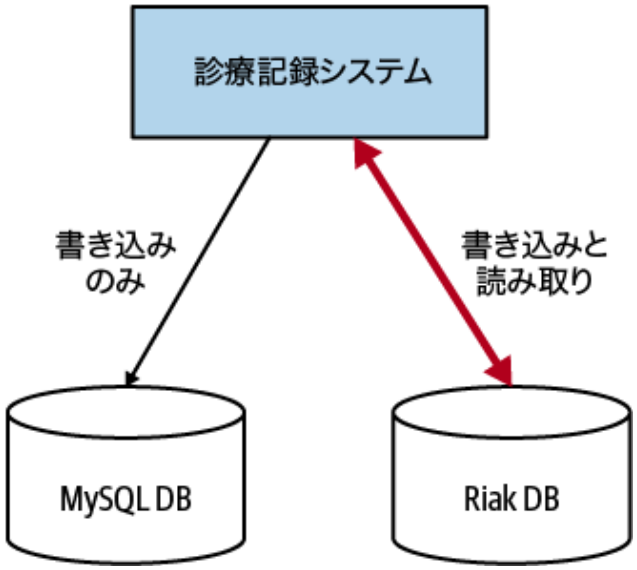


図4-15　新しいデータベースが真の情報源になったが、古いデータベースも同期が保たれている

こうして新しいシステムが十分に馴染んだら、古いスキーマを安全に削除できる。

4.8.4　このパターンを使うとき

デンマークの診療記録システムの例は、1つのアプリケーションでの対応だった。本書で扱ってきたマイクロサービスに分割しようとしている状況で、このパターンは本当に役に立つのだろうか？　まず考えられるのは、アプリケーションコードを分割する**前に**スキーマを分割したい場合に、このパターンはとても合っているかもしれないということだ。図4-16は、まさにそのような状況を示している。

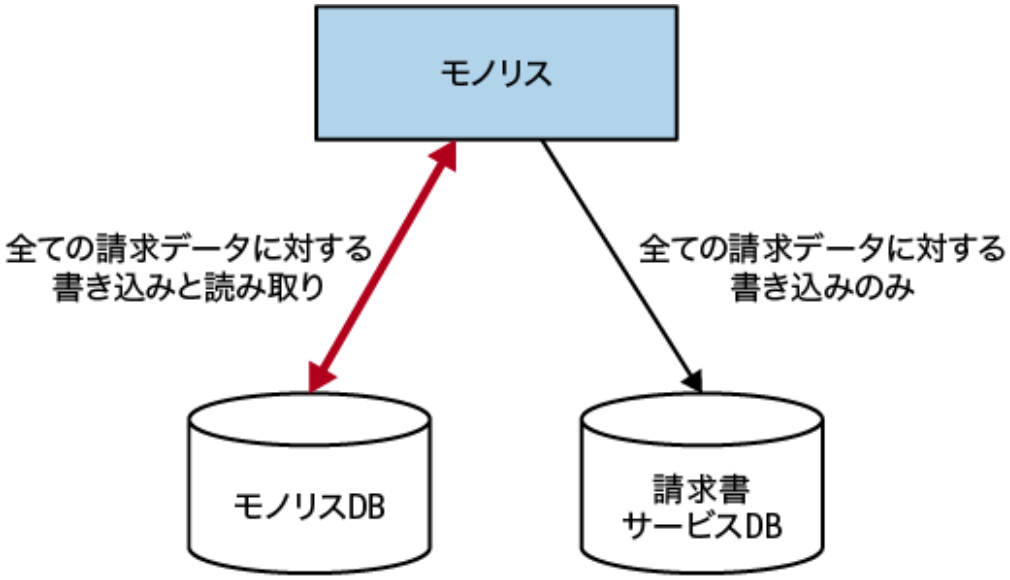


図4-16　2つのスキーマを同期させるモノリスの例

もしこの例が正しく実装されているなら、両方のデータソースは常に同期されているはずで、それはロールバックシナリオなどでの素早いデータソースの切り替えが必要な状況で大きなメリットをもたらす。デンマークの診療記録システムの例では、システムをわずかでもオフラインにはしておけなかったため、このパターンの使用は賢明な判断だった。

4.8.5　使いどころ

モノリスとマイクロサービスの両方がデータにアクセスしている場合に、このパターンの利用が考えられるが、これは極めて複雑になる。図4-17はそうした状況を示している。このパターンが機能するには、モノリスとマイクロサービスの両方がデータベース間の適切な同期を保証する必要がある。もしどちらか一方が間違えると、トラブルを生む可能性がある。ある時点で請求書サービスとモノリスの請求書機能のどちらが書き込みを行っているかがはっきりしている場合には、この複雑さはずっと軽減される。しかし、カナリアリリースなどを行っていて、モノリスとマイクロサービスのどちらもがリクエストを受けうる状況においては、このパターンはお勧めできない。結果として同期が厄介なことになるからだ。

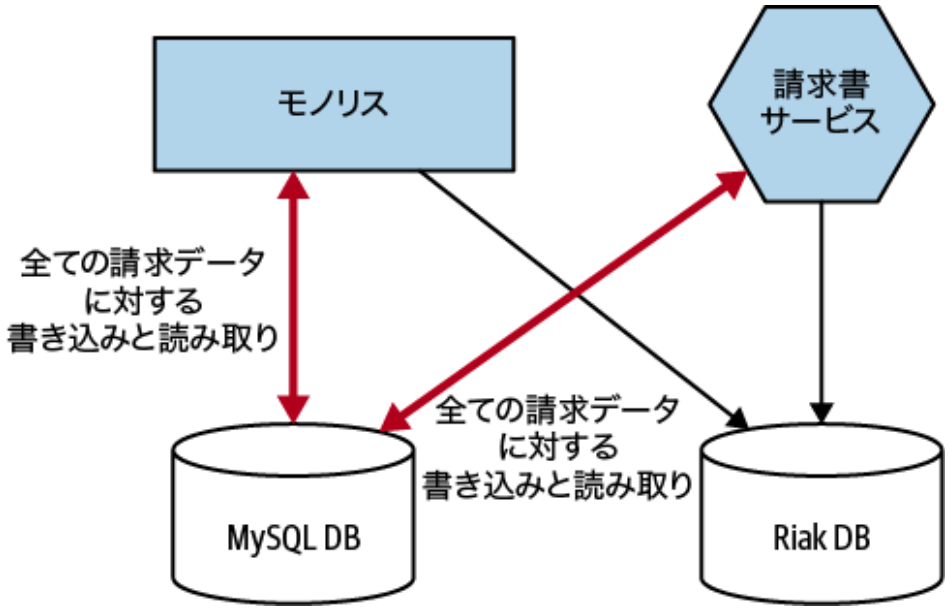


図4-17　モノリスとマイクロサービスの両方が同じ2つのスキーマを同期させようとしている例

4.9　パターン：トレーサー書き込み

トレーサー書き込みは、「パターン：アプリケーションでのデータ同期」の一種だ。図4-18にこのパターンの概要を示す。トレーサー書き込みパターンで

は、データの真の情報源を段階的に移行する。そして、その間は2つの真の情報源が存在することを許容する。まず、対象のデータを提供するのに適切な新しいサービスを特定する。そして、既存のシステムが引き続きそのデータを記録することを維持しつつ、データを変更する際には、サービスインターフェイスを通して、このデータが新しいサービスに書き込まれることも保証するようにする。既存のコードを変更して新しいサービスを参照し始め、全ての機能が新しいサービスを真の情報源として使用するようになれば、古い真の情報源は破棄可能となる。この間、2つの真の情報源の間でデータをどのように同期するかは、慎重に検討する必要がある。

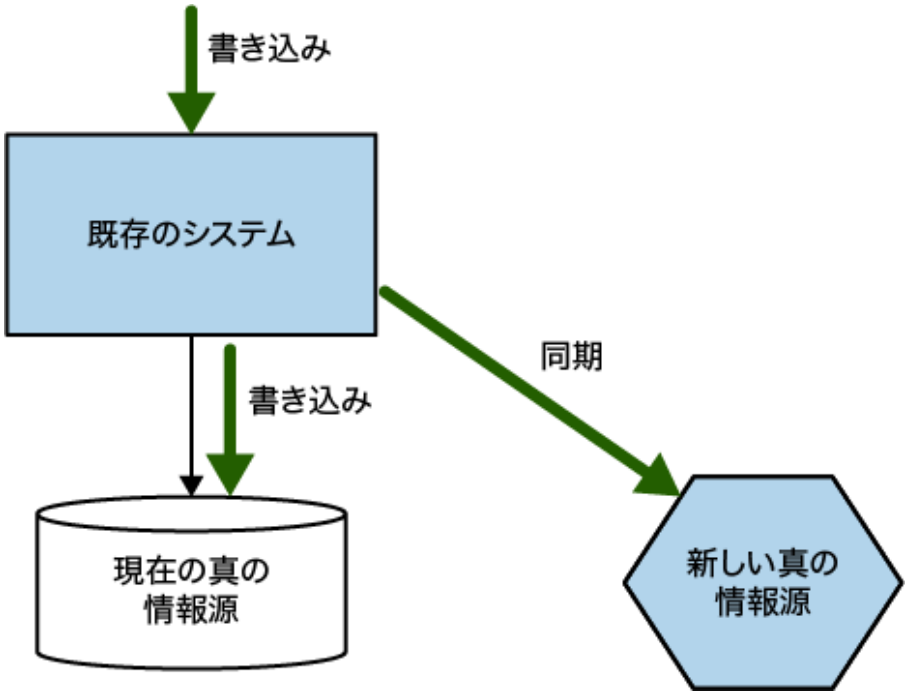


図4-18　トレーサー書き込みでは、移行中に2つの真の情報源に対応する。これにより、あるシステムから別のシステムへデータを段階的に移行可能となる

真の情報源を一つにしておきたいというのは、完全に合理的な欲求だ。その方が、データの一貫性を確保し、データへのアクセスを制御し、メンテナンスコストを削減できるからだ。しかし、データに対する真の情報源は一つしか認めないといった主張は、データの居場所を変えることが一度の大きな切り替えになるという状況へと私たちを追い込んでしまう。リリース前はモノリスを真の情報源とし、リリース後は新しいサービスを真の情報源としなければならないからだ。問題は、この切り替えの間にさまざまなことがうまくいかなくなる可能性があるということだ。トレーサー書き込みのようなパターンは、複数の真の情報源を持つことを許容するのと引き換えに、段階的な切り替えを可能にし、各リリースの影響を軽減する。

このパターンがトレーサー書き込みと呼ばれる理由は、小さなデータセットを同期することから始めて、これを段階的に増やしていき、同時に新しいデータソースのコンシューマー数を増やすことができるためだ。図4-19で示す例では、請求書関連のデータがモノリスから新しい請求書サービスに移行されている。この例では、まず基本的な請求データを同期し、次に請求書の送付先情報を移行し、最後に支払い記録を同期している。

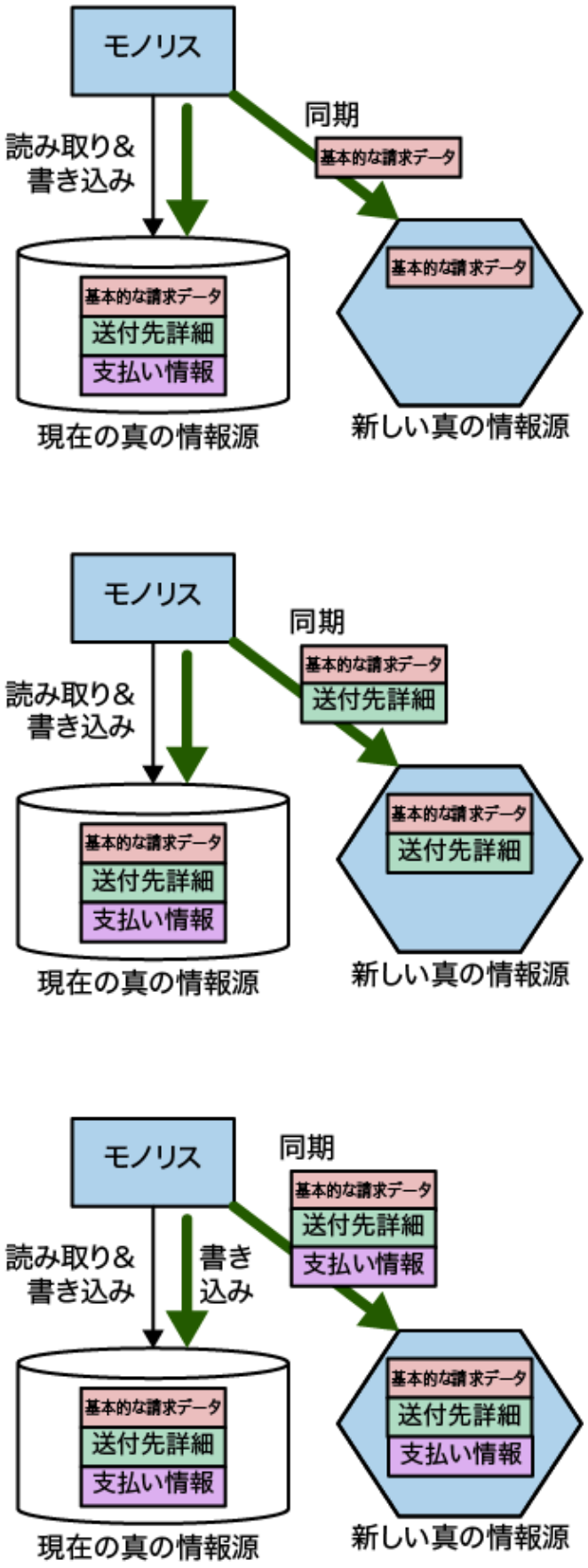


図4-19　請求書関連の情報をモノリスから請求書サービスに段階的に移す

請求書関連の情報を必要とする他のサービスは、どのような情報を必要とするかによって、モノリスと新しいサービスのどちらから情報を入手するかを選択しなければならない。モノリス内でしか利用できない情報が必要な場合は、そ

のデータとそれをサポートする機能が移行されるまで待つ必要がある。データと機能が新しいマイクロサービスで利用できるようになれば、コンシューマーは新しい真の情報源に切り替えられる。

この例におけるゴールは、モノリス自体を含め、全てのコンシューマーが請求書サービスを使用するようになることだ。図4-20は、移行中のいくつかの段階を示している。最初は、基本的な請求情報だけを両方の真の情報源に書き込んでいる。この情報が適切に同期されていることを確認すると、モノリスは新しいサービスからデータの読み取りを開始できる。より多くのデータが同期されるほど、モノリスは新しいサービスをより多くのデータの真の情報源として使用できるようになる。全てのデータが同期され、古い真の情報源を使用していた最後のコンシューマーが切り替わったら、データの同期を止められる。

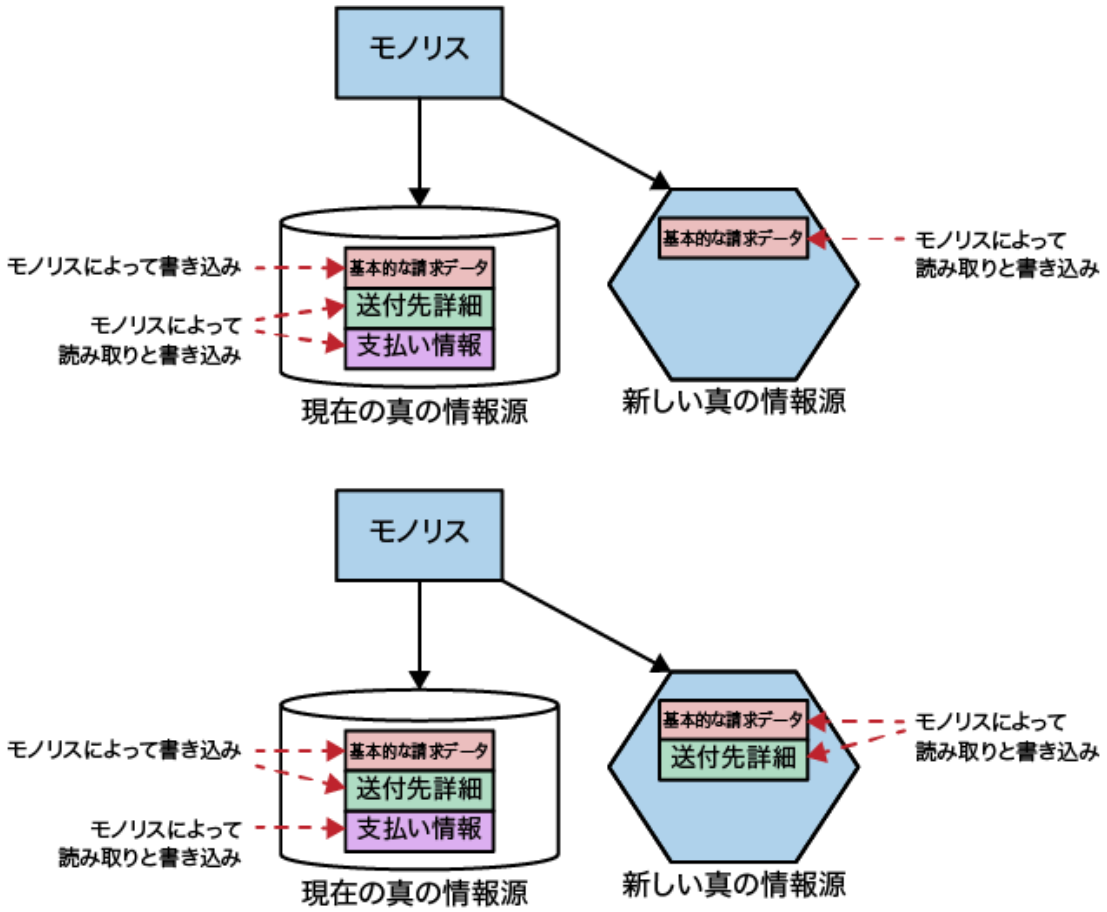


図4-20　トレーサー書き込みの中で段階的に古い真の情報源を廃止する

4.9.1　データ同期

トレーサー書き込みパターンでは、データが重複しているあらゆる状況で頭を悩ます問題、すなわち不整合に対処しなければならない。これを解決するには、次のような選択肢がある。

一方のソースに書き込む
全ての書き込み要求は、片方の真の情報源に送られる。書き込みが行われた後で、データはもう一方の真の情報源に同期される。

両方のソースに書き込む
上流のクライアントによって行われた全ての書き込み要求は、両方の真の情報源に送られる。これは、クライアント自身が両方の真の情報源の呼び出しを確実に行うか、仲介者を介して各下流サービスに要求をブロードキャストすることで行われる。

どちらかのソースに書き込む
クライアントはどちらかの真の情報源に書き込み要求を送信できる。その裏ではシステム間で双方向にデータが同期される。

両方のソースに書き込み要求を送る選択肢、あるいは一方のソースに書き込み要求を送って何らかのバックグラウンド同期に頼る選択肢は、うまくいく解決策のように思える。実際この後で触れる事例では、これら両方のテクニックが使われている。しかしながら、どちらかのソースに書き込むという第三の選択肢は、たしかに技術的には選択肢の一つではあるものの、避けるべきだろう。この選択肢は双方向同期が必要となり、そして双方向同期は実現が非常に困難な可能性があるからだ。

いずれの場合においても、真の情報源の間でデータが整合するまでには、多少の遅延が発生することになる。この不整合が生じている間隔がどれくらいになるかは、いくつかの要因に左右される。たとえば、片方の更新をもう片方にコピーするために毎晩バッチ処理を行っている場合には、同期される側の真の情報源には最大で24時間古いデータが含まれている可能性がある。変更データキャプチャなどを使用して、あるシステムから別のシステムへの更新を逐次行っている場合には、不整合が生じている間隔は数秒以下で済むかもしれない。

どれだけ長い間不整合が生じていたとしても、最終的には上記のような同期によって**結果整合性**が得られる。つまり、両方の真の情報源が同じデータを持つようになる。データ同期に関しては、不整合が生じている間隔がどれくらいであれば適切かを理解し、それに基づいて同期をどのように実装するかを決定する必要がある。

2つの真の情報源を管理する際には、同期が意図した通りに機能していることを確認する何らかの調停プロセスを用意することが重要だ。それは、各データベースに対していくつかのSQLクエリを実行するような単純なものかもしれない。仮に、同期が期待通りに動作しているかを確認しないと、2つのシステム間で不整合が発生してしまい、手遅れになるまで気付かないことがあり得る。うまく動作していることが確認できるまでの間は、新しい真の情報源をコンシューマーがいない状態で動かすことは、とても賢明なやり方だ。これについては、この後Squareが行った事例を紹介する。

4.9.2　事例：Squareの注文

このパターンはもともとSquareに所属する開発者Derek Hammerが共有してくれたもので、それ以来、私はこのパターンが実際に使われている他の事例を見つけてきた[†8]。彼は、Squareにおけるテイクアウト食品の宅配注文に関連するドメインの一部を解きほぐすのにこのパターンをどう役立てたかを詳しく説明してくれた。初期のシステムでは、複数のワークフローを管理するために1つの注文概念が使われていた。注文概念を共有していたのは、顧客が食品を注文するワークフロー、レストランが食品を準備するワークフロー、配送ドライバーが食品をピックアップして顧客に届けるワークフローだった。これらのステークホルダーのニーズは異なっていた。いずれのステークホルダーも注文という同じ用語を扱っているものの、それぞれにとって注文という用語が意味するものは異なっていた。顧客にとって、注文とは配達を希望したものであり、お金を払う必要があるものだった。レストランにとって、注文とは調理して受け渡す必要があるものだった。そして、配送ドライバーにとっては、レストランから適切なタイミングで顧客に運ぶ必要があるものだった。このように、それぞれニーズが異なっていたにもかかわらず、注文に関するコードとデータは全て一緒に束ねられていた。

[†8] Sangeeta HandaはQCon SFカンファレンスでNetflixがデータ移行の一部としてこのパターンをどのように使用したかを共有し、Daniel Bryantはその後、このことについての素晴らしい記事 “Building Resilience in Netflix Production Data Migrations: Sangeeta Handa at QCon SF”（http://bit.ly/2m1EwHT）[7]を書いてくれた。

これら全てのワークフローにおける注文の概念を1つにまとめることは、先に紹介した**デリバリー衝突**の原因となっていた。コードベースの同じ部分を変更しなくてはならないため、異なる開発者が異なるユースケースに合わせて変更を加えようとするとお互いの邪魔になってしまうからだ。Squareでは、注文を分割して、各ワークフローに対する変更を個別に行えるようにしたいと考えていたが、それと同時にスケーリングと堅牢性のニーズも満たしたいと考えていた。

新しいサービスの作成

最初のステップは、レストランと配送ドライバーに関連付けられた注文データを管理する新しいフルフィルメント[†9]サービスを作成することだった。これを図4-21に示す。このサービスは、フルフィルメントにまつわる注文データ部分の新しい真の情報源となるものだ。最初、このサービスはフルフィルメント関連のエンティティを作成できる機能だけを公開した。新しいサービスが稼働すると、フルフィルメント関連のデータはバックグラウンドのワーカーを使って既存のシステムから新しいサービスへとコピーされた。このバックグラウンドワーカーは、データベースに直接挿入するのではなく、フルフィルメントサービスが公開しているAPIを単に利用するものだった。そうすることで、バックグラウンドワーカーがデータベースに直接アクセスする必要性は回避された。

[†9] 訳注：注文が行われてからエンドユーザーに商品が届くまで必要な業務全般のこと。

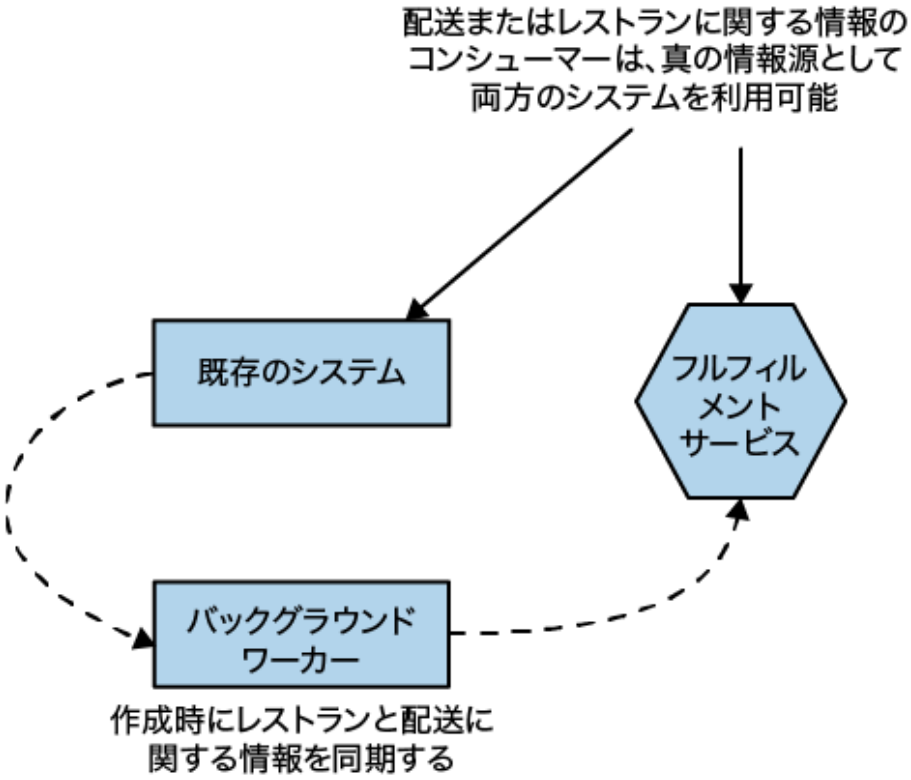


図4-21　新しいフルフィルメントサービスは、フルフィルメント関連データを既存のシステムからコピーするために使用された

バックグラウンドワーカーでは、フィーチャーフラグによってコピープロセスを有効にするか無効にするかが制御されていた。それにより、本番環境でワーカーが問題を起こした場合にもプロセスを簡単に停止できるようになっていた。同期が正しく機能していることを十分な時間をかけて確認した後で、このシステムは本番環境で実行された。そして、バックグラウンドワーカーが期待通りに動作していることが確認できた後で、フィーチャーフラグは削除された。

データの同期

この種の同期における課題の1つには、それが一方通行だということがある。既存のシステムに変更を加えた結果、フルフィルメント関連のデータはAPIを介して新しいフルフィルメントサービスに書き込まれることになった。その状態でデータを同期するため、Squareは両方のシステムに全ての更新を送ることにした。これを図4-22に示す。しかし、実際には両方のシステムに全ての更新が必要な訳ではなかった。Derekの説明によると、フルフィルメントサービスが扱っているのは注文概念の一部にすぎないため、配送やレストラン客が関心を持つ注文に関する変更だけをコピーする必要があった。

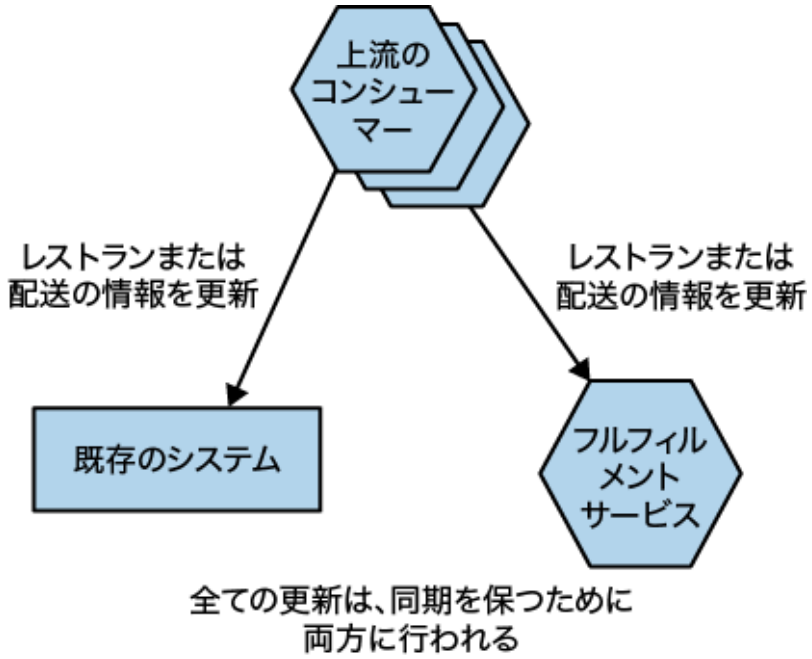


図4-22　全てのコンシューマーが両方のサービスの適切なAPIを呼び出すことで、その後の更新は同期された

レストランの情報や配送の情報を扱うコードは、既存システムと新しいサービスの2つのAPI呼び出しを行うように変更する必要があった。また、これらの上流のクライアントは、一方への書き込みが成功しても他方への書き込みが失敗した場合には、エラー状態を処理する必要があった。2つの下流システム（既存の注文システムと新しいフルフィルメントサービス）に対するこれらの変更は、アトミックな方法では行われなかった。つまり、一方のシステムで変更が反映されていても、もう一方のシステムではまだ変更が反映されていない時間がわずかながら存在する可能性があった。両方に変更が適用されるまでは、2つのシステム間で不整合が生じていることになる。これは先ほど説明した**結果整合性**の一形態だ。

システム間に不整合が生じているわずかな時間の存在は、このユースケースでは問題にならなかった。データは2つのシステム間で十分に迅速に同期されていたため、取り立ててシステムのユーザーには影響を与えなかったからだ。

もしSquareが注文の更新を管理するためにAPI呼び出しではなくイベント駆動システムを利用していたら、代替的な実装が検討できただろう。図4-23には、注文状態の変更をトリガーとするメッセージの単一ストリームがある。既存のシステムと新しいフルフィルメントサービスの両方が同じメッセージを受信する。上流のクライアントは、これらのメッセージを受け取る複数のコンシューマーがいることを知る必要はない。これは、パブリッシュ／サブスクライブ型のブローカーを使用することで処理が可能だ。

こうしたユースケースを満たすためにSquareのアーキテクチャをイベント駆動に改修するのは大変な作業だ。ただ、もし既にイベント駆動のシステムを利用しているのであれば、同期プロセスを管理することが簡単になるかもしれない。しかし、そうしたアーキテクチャが見せるのはあくまで結果整合的な振る舞いだという点には注意する必要がある。既存のシステムとフルフィルメントサービスの両方が同じイベントを同時に処理することは保証できないためだ。

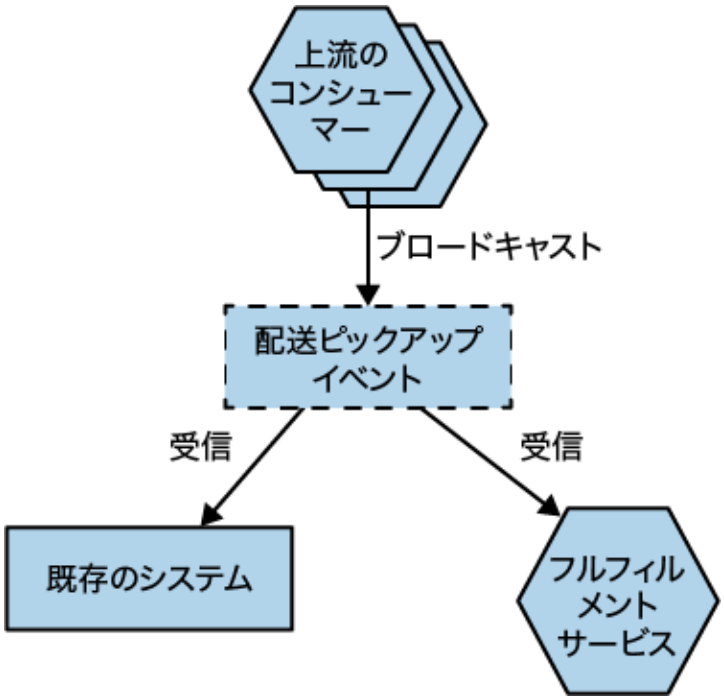


図4-23　代替的な同期化アプローチとしては、両方の真の情報源が同じイベントをサブスクライブすることが考えられる

コンシューマーの移行

新しいフルフィルメントサービスが、レストランと配送ドライバーそれぞれのワークフローに必要な全ての情報を保持するようになった。これで、ワークフローを管理するコード側を、新しいサービスを使用するように切り替えていく準備が整った。ワークフローを管理するコード側の移行が始まると、新しいサービスにはこれらのコンシューマーのニーズをサポートするためのより多くの機能が追加されていく。フルフィルメントサービスは、最初はバックグラウンドワーカーが新しいレコードを作成できるようにするためのAPIを実装していただけだった。しかし、新しいコンシューマーの移行が進むにつれて、そのニーズをサポートするために、フルフィルメントサービスには新しい機能が追加されていった。

コンシューマーが新しい真の情報源を使用するように変更しながらデータを段階的に移行していくこのアプローチは、Squareではとても効果的であること

が証明された。Derekによると、全てのコンシューマーが乗り換えるまでに、ほとんど何のトラブルも起こらなかったとのことだ。それらは定期的なリリースのたびに行われるありふれた小さな変更だった（これが本書で段階的な移行パターンを強く提唱しているもう一つの理由だ！）。

ドメイン駆動設計の観点からは、配送ドライバー、顧客、レストランに関連付けられた機能は、それぞれが異なる境界づけられたコンテキストを表していると言える。その観点から見れば、理想的にはこのフルフィルメントサービスをさらに分割して、さらにレストラン用と配送ドライバー用の2つのサービスに分けることを検討した方がいいとDerekは提案した。そのように、さらなる分解の余地はあるものの、この移行は非常に成功しているように見えた。

Squareでは、重複したデータはそのまま残された。レストランと配送に関連する注文情報を既存のシステムに残しておいたおかげで、フルフィルメントサービスが利用できないときでも、その情報を参照できた。もちろん、そのためには同期を維持する必要がある。これは時間が経過するとともに見直されるかもしれない。フルフィルメントサービスの利用可能性に十分な信頼が得られれば、バックグラウンドワーカーとコンシューマーが2つの更新呼び出しを行う必要がなくなり、アーキテクチャを整理できるだろう。

4.9.3　使いどころ

このパターンにおけるほとんどの作業は、同期の実装になるだろう。双方向同期を避け、代わりにここで説明したよりシンプルな選択肢を使用できれば、このパターンの実装はずっと簡単になる。既にイベント駆動型システムを使用している場合や、変更データキャプチャのパイプラインが利用可能な場合には、同期を行うための構成要素の多くは既にそろっているかもしれない。

システム間の不整合をどのくらいの期間許容できるかについては、慎重に検討する必要がある。あるユースケースでは気にしないかもしれないが、他のユースケースではほぼ即時にレプリケーションしたいかもしれない。矛盾を許容できる期間が短ければ短いほど、このパターンの実装は難しくなる。

4.10　データベースを分割する

データベースを使って複数のサービスを統合する手法の課題については、既に多く述べてきた。お分かりかもしれないが、私はこのやり方のファンではない。したがって、データベースをきれいに分割できるよう、データベースにも

接合部を見つける必要があると考えている。しかし、データベースは扱いにくい厄介な代物だ。このアプローチの説明例に入る前に、論理的な分離と物理的なデプロイがどのように関連しているかを簡単に説明しよう。

4.10.1　物理的なデータベースの分離と論理的なデータベースの分離

この文脈でデータベースを分割するという話をするときに、私たちが実現しようとしているのは主に論理的な分離だ。データベースエンジンは、図4-24に示すように、単一のデータベースエンジンで、論理的に分離されたスキーマを1つ以上ホストすることが可能だ。

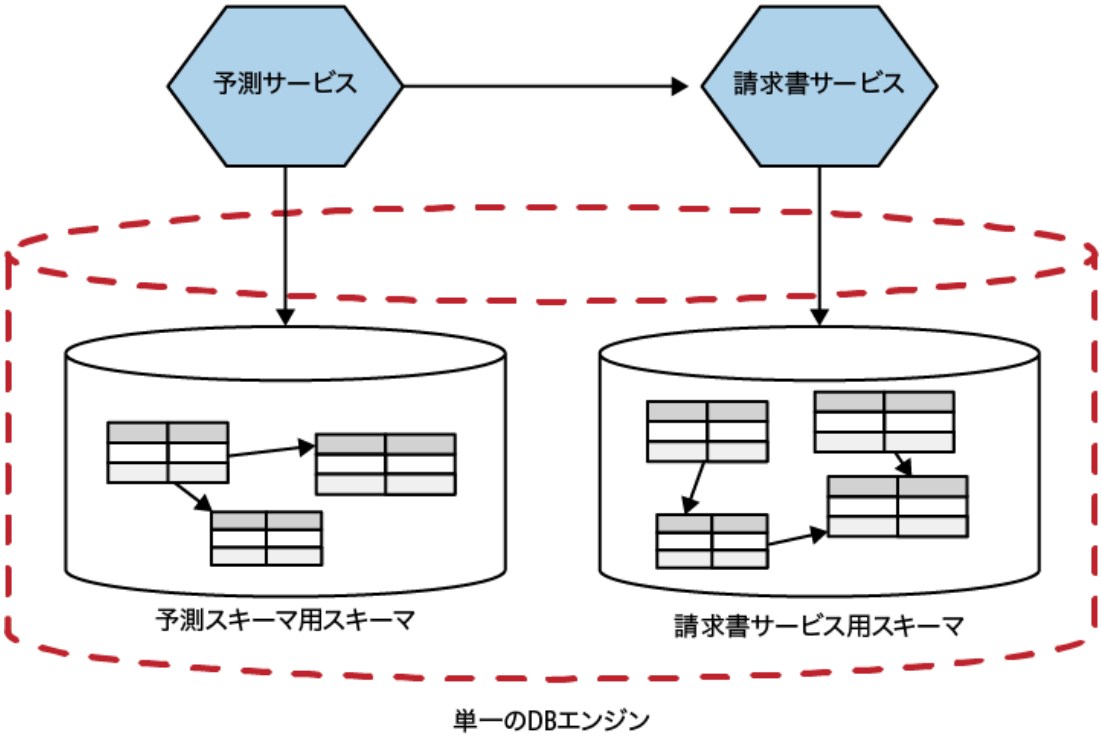


図4-24　2つのサービスが別々の論理スキーマを使用している。論理スキーマはどちらも同じ物理データベースエンジン上で動作している

分離をさらに進めると、それぞれの論理スキーマを別々のデータベースエンジン上に置くことで物理的な分離も行える。これを図4-25に示す。

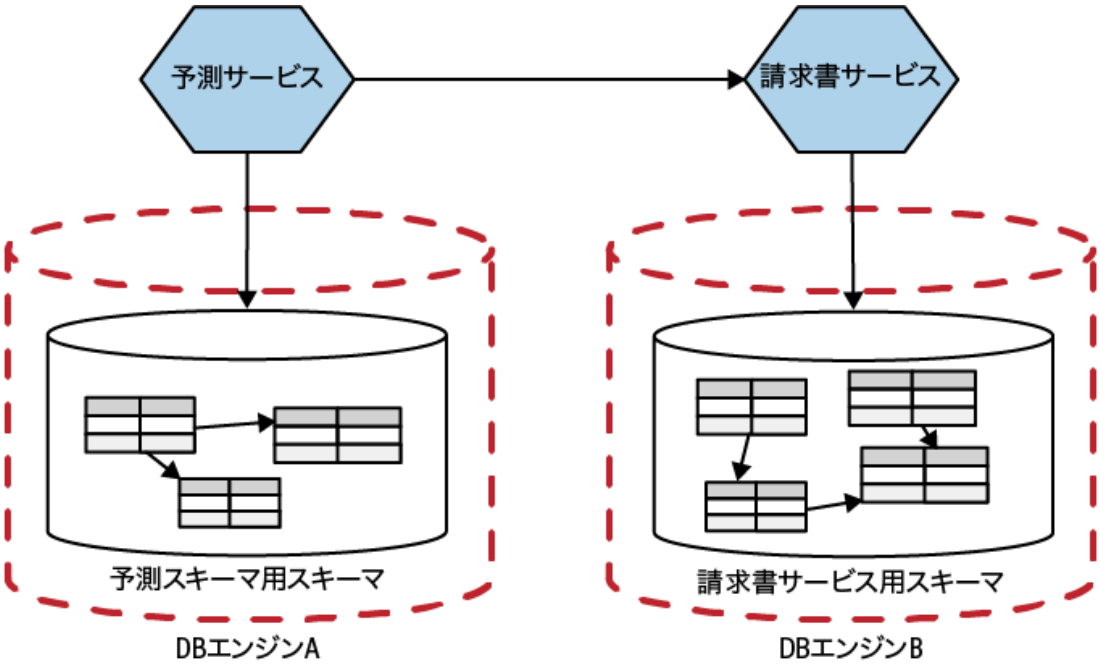


図4-25　2つのサービスが別々の論理スキーマを使用しており、それぞれが独自の物理データベースエンジン上で動作している

1つのデータベースエンジン上にあるスキーマを、なぜ論理的に分解したいのだろう？　論理的な分解と物理的な分解は、基本的に異なる目的を実現するものだ。**論理的分解**は、独立した変更と情報隠蔽を容易にする。そして、**物理的分解**は、システムの堅牢性を向上させ、リソースの競合を排除してスループットやレイテンシーを改善する。

図4-24に示すように、データベーススキーマを論理的には分解しつつも同一の物理データベースエンジン上で保持する場合には、それが潜在的な単一障害点となる。そのデータベースエンジンがダウンすると、両方のサービスが影響を受けることになるからだ。しかし、世の中はそんなに単純ではない。多くのデータベースエンジンは、マルチプライマリデータベースモードやウォームスタンバイ機能など、単一障害点を回避するための仕組みを備えている。実際には、組織内での高耐障害性データベースクラスタの構築には多大な努力が払われているかもしれないし、複数のクラスタを持つには時間、労力、コスト（厄介なライセンス料も膨れ上がる！）がかかるため、 データベースを物理的に分解することを正当化するのは難しいかもしれない。

もう一つ考えなくてはならない点は、データベースビューを公開したい場合には、複数のスキーマが同じデータベースエンジンを共有している必要があるかもしれないということだ。つまり、ソースとなるデータベースと、ビューをホストしているスキーマの両方を同じデータベースエンジン上に配置する必要があるかもしれない。

もちろん、物理的なデータベースエンジンを分けた上で個々のサービスを動かすという選択肢もあるが、いずれにせよ、それらのスキーマは論理的に分解しておく必要がある。

4.11　データベースとコードのどちらを最初に分割するか

ここまでは共有データベースでの作業に役立つパターンを説明してきたが、うまくいけば、より結合の少ないモデルへと進んでいける。データベース分解に関するパターンはこの後で詳しく見ていくが、その前に、まず順番について議論する必要がある。マイクロサービスの抽出は、アプリケーションコードがそれ自身のサービスで実行され、サービスの制御するデータが論理的に分離されたそれ自身のデータベースに抽出されるまでは「完了」しない。しかし、本書は主に段階的な変更を可能にすることについて書いているため、この抽出がどのようにして順序付けられるべきかを少し探る必要がある。これには、次のような選択肢がある。

- 最初にデータベースを分割し、次にコードを分割する。
- 最初にコードを分割し、次にデータベースを分割する。
- 両方を一度に分割する。

それぞれの選択肢には長所と短所がある。これらの選択肢を、あなたが取るアプローチに応じて、役立つかもしれないいくつかのパターンと一緒に見ていこう。

4.11.1　データベースを最初に分割する

スキーマが分かれていると、1つのアクションを実行するためにデータベースを呼び出す回数が増える可能性がある。以前は1つのSELECT文で全てのデータを取得できていたかもしれないことが、2つの場所からデータを引き出してメモリ内で結合する必要が生じるかもしれない。また、2つのスキーマに移行するとトランザクション整合性が崩れ、アプリケーションに大きな影響を与える可能性もある。章の後半では、分散トランザクションやサーガについてと、それらがどのように役立つかについて説明する。図4-26のように、スキーマを分割してもアプリケーションコードを一緒にしておくことで、間違った道に進んでいると気付いた場合に、変更を元に戻したり、サービスの利用者に影響を与えずに微調整を続けたりできる。DBの分離に意味があると分かったら、次に

アプリケーションコードを2つのサービスに分割することを検討できるだろう。

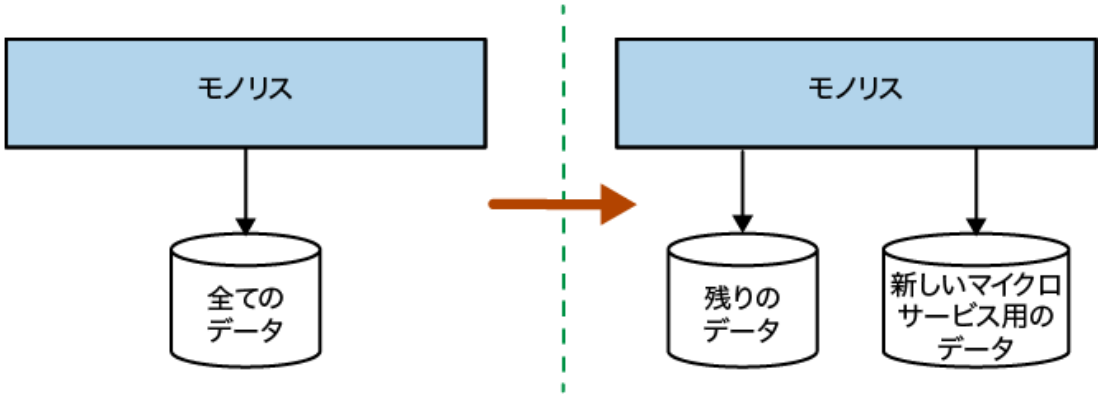


図4-26　最初にスキーマを分割することで、パフォーマンスとトランザクション整合性に関する問題を早期に発見できる可能性がある

逆に、このアプローチからは短期的な利益はあまり得られそうにない。モノリシックなコードのデプロイがまだ残っているからだ。共有データベースの痛みは時間がたつにつれて感じるものだ。だから、短期的な利益は十分に得られないとしても、長期的な見返りを得るために、分割の時点では時間と労力を費やすことになる。このような理由から、パフォーマンスやデータの一貫性の問題を特に懸念している場合にのみ、私はこのルートを採用することにしている。また、モノリス自体が商用ソフトウェアのようなブラックボックスシステムである場合、この選択肢は利用できないことも考慮する必要がある。

ツールに関する注意

データベースの変更を困難にする理由の1つには、変更を簡単に行えるツールが限られていることがある。コードでは、IDEにリファクタリングツールが組み込まれている。そして、基本的に変更するシステムはステートレスであるという利点もある。データベースでは、変更しているものに状態があり、そしてIDEのリファクタリング機能のような優れたツールもない。

何年も前に、このツールのギャップが2人の同僚Nick AshleyとGraham Tackleyと私を駆り立て、DBDeployというオープンソースツールを作成した。現在は廃れてしまったが（オープンソースツールを作ることとメンテナンスをすることは全く違うんだ）、このツールは、スキーマ上で確定的に実行できるSQLスクリプトを使って変更を取り込めるようにすることで機能した。各スキーマには特別なテーブルがあり、どのスキーマスクリプトが適用されたかを追跡するために使用されていた。

DBDeployの目標は、スキーマへの段階的な変更を可能にし、各変更のバージョンを管理し、変更を複数のスキーマで異なる時間に実行できるようにすることだった（開発者スキーマ、テストスキーマ、本番スキーマを考えてみてほしい）。

現在では、FlywayDB[†10]や似たような機能を提供するものを人々に紹介している。しかし、どのようなツールを選ぶにしても、バージョン管理可能な差分スクリプトで各変更をキャプチャできるようにすることを強くお勧めする。

[†10] https://flywaydb.org

パターン：境界づけられたコンテキストごとのリポジトリ

Hibernateのようなフレームワークに支えられたリポジトリ層[†11]を使ってコードをデータベースに結びつけることで、オブジェクトやデータ構造とデータベースを簡単にマッピングできるようにするのが一般的なプラクティスになっている。全てのデータアクセス用に単一のリポジトリ層を持つのではなく、図4-27のように、境界づけられたコンテキストの境界に沿ってリポジトリを分解することには価値がある。

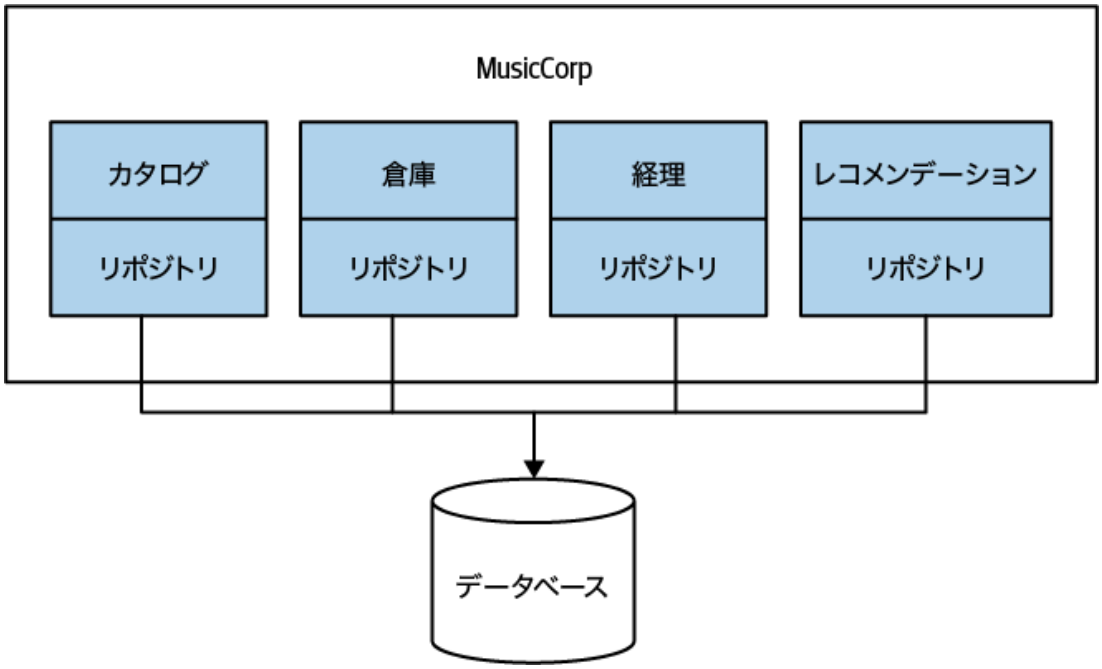


図4-27　リポジトリ層を分割する

[†11] 訳注：データの操作に関連するロジックをビジネスロジックから切り離し、抽象化したレイヤー。

データベースに対応づけられたコードを特定のコンテキストのコード中に配置しておくと、データベースのどの部分がどのコードで使われているかを理解しやすくなる。Hibernateであれば、境界づけられたコンテキストごとにマッピングファイルを用意することで、これがとても明確になる。そのようにして、それぞれの境界づけられたコンテキストがスキーマのどのテーブルにアクセスしているかを明らかにしておくことは、将来に分解を行う際どのテーブルを移動させる必要があるかを理解する助けとなる。

ただ、話はそれだけではない。たしかに、経理機能のコードが元帳テーブルを使用することや、カタログ機能のコードが品目テーブルを使用することは分かるかもしれない。しかし、データベースが元帳テーブルから品目テーブルへの外部キー関係を強制していることは明らかではない。このようなデータベースレベルの制約を確認するには、別のツールでデータを可視化する必要がある。手始めには、自由に利用できるSchemaSpy[†12]といったツールを使用するのが良いだろう。SchemaSpyは、テーブル間の関連のグラフィカルな表現を生成できるツールだ。

[†12] http://schemaspy.sourceforge.net

こうしたことは全て、最終的にサービス境界となる部分をまたぐ可能性のあるテーブル間の結合を理解するのに役立つ。しかし、そうした結合はどうやったら断ち切れるのだろうか？　また、複数の境界づけられたコンテキストが同じテーブルを使っている場合はどうなるだろうか？　このトピックについては、章の後半で詳しく説明する。

使いどころ

このパターンは、モノリスを分割する方法をよりよく理解するためにモノリスと向き合いなおそうとしている場合には、どんな状況でも本当に効果的だ。リポジトリ層をドメインの概念に沿って分解することは、マイクロサービスのた

めの接合部が、データベースだけでなく、コードにも存在している可能性を理解するのに役立つはずだ。

パターン：境界づけられたコンテキストごとのデータベース

アプリケーションの観点からデータアクセスを明確に分離したのなら、このアプローチをスキーマにまで広げるのは理にかなっている。マイクロサービスの独立デプロイ可能性という考え方の中心にあるのは、マイクロサービスが自分のデータを所有すべきだという考えだ。識別した境界づけられたコンテキストの境界でデータベースを明確に分離することで、アプリケーションコードを分離する前に、この分解を始められる。

ThoughtWorksで企業向けに収益の計算と予測を行う新しいメカニズムを実装していた時のことだ。私たちは3つの大まかな機能領域を特定した後で、これをどう分解するかについてプロジェクトの責任者であるPeter Gillard-Mossに相談を行った。彼は、機能は十分に分離されているように感じているものの、これらの機能を別々のマイクロサービスにした場合に余計な作業が発生するのではないかという懸念を表明してくれた。この時点ではチームは小さく（3人しかいなかった）、チームは新しいサービスを分割することは正当化できないと感じていた。最終的には、新しい収益機能は3つの境界づけられたコンテキスト（それぞれが個別のJARファイルになった）を含む1つのサービスというモデルに落ち着いて、実際にデプロイされた。これを図4-28に示す。

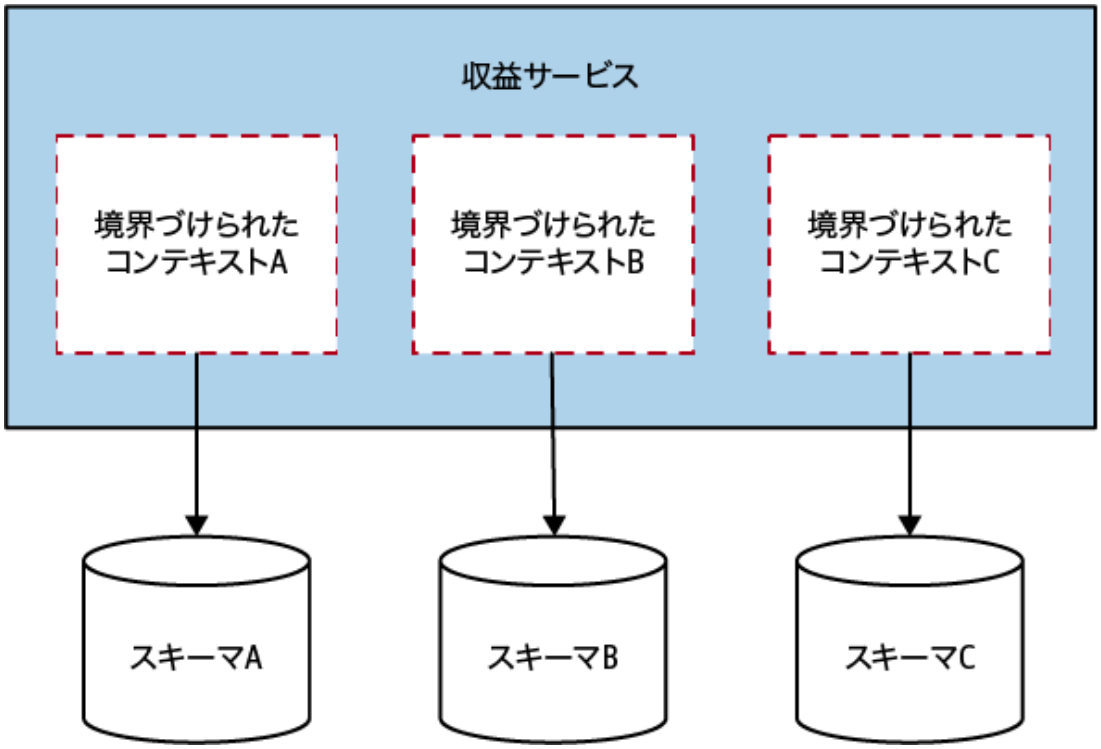


図4-28　収益サービスの各境界づけられたコンテキストは、後に分離できるように、個別のデータベーススキーマを持っていた

それぞれの境界づけられたコンテキストは、完全に分離されたデータベースを持っていた。これは、後でサービスを分離することになった場合に、それをずっと簡単にするだろうという考えからだった。しかし、後になってその必要がないことが判明した。数年後、この収益サービスは、複数の関連データベースを持つモノリス、つまりモジュラーモノリスの好例のまま残っていた。

使いどころ

モノリスにしておくなら、一見すると、分離された個々のデータベースを維持するために余計な作業をすることはあまり理にかなっていない。これは詰まるところリスクヘッジのパターンだ。単一のデータベースより少し手間はかかるものの、後々マイクロサービスに移行する際の選択肢を広げておけるからだ。マイクロサービスに移行しないとしても、データベースを支えるスキーマを明確に分離しておくことは、特にモノリス自体に多くの人が関わっている場合には本当に有効だ。

これは、（既存のシステムを再実装するのではなく）新しいシステムを構築する人たちには、ほぼ常に推奨しているパターンだ。私は、新製品開発やスタートアップなどでマイクロサービスを採用することは好まない。特にスタートアップでは、構築しているものの性質が大きく変わることがある。しかし、このパターンはよい中間点になる。サービスの分離が将来的に必要になるかもしれない箇所では、スキーマの分離を維持しよう。そうすれば、システムの複雑さを軽減しながら、結合を減らすことの恩恵を受けられる。

4.11.2　コードを最初に分割する

図4-29に示すように、ほとんどのチームが最初にコードを分割し、次にデータベースを分割している。そうしたチームは、データベースを分離して分解を完了するための自信を与える短期的な改善を、（うまくいけば）新しいサービスから得ている。

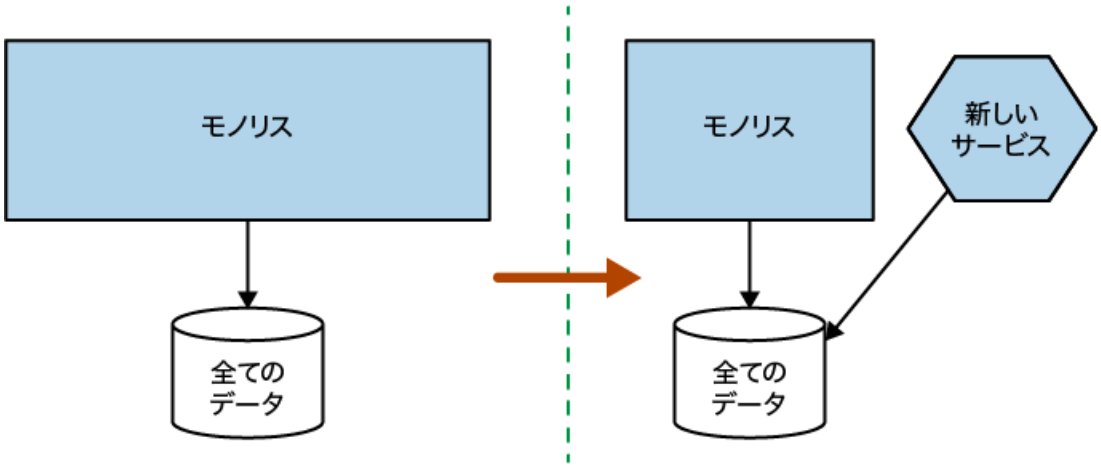


図4-29　最初にアプリケーション層を分割することで、単一の共有スキーマが得られる

アプリケーション層を分割することで、新しいサービスがどのようなデータを必要としているのかがとても理解しやすくなる。また、独立してデプロイ可能なコード成果物が早い段階で手に入るという利点もある。このアプローチで常に懸念となるのは、ここまで進んだところで、共有データベースを使用したままになってしまうことだ。このアプローチを取るのであれば、次のことを覚えておこう。データ層への分離を完了させないと、将来に向けての問題を抱え込むことになる。こうした罠に陥ったチームを過去に見てきたが、もちろん正しい対処を行った組織の事例も聞いている。JustSocial社は、マイクロサービス移行の中でこのアプローチを使用した組織の1つだ。データの結合操作をアプリケーション層に引き上げることで生じる、厄介な問題の発見が遅れるかもしれないというのも、このアプローチの潜在的な課題の1つだ。

この方向に進む場合には、自分自身に正直になる必要がある。マイクロサービスの所有するデータを次のステップの一部として確実に分割できると確信を持てるだろうか？

パターン：データアクセス層としてのモノリス

モノリスから直接データにアクセスするのではなく、モノリス自体にAPIを作成するモデルへと移行するのも可能だ。図4-30では、請求書サービスは顧客サービスが持つ従業員に関する情報を必要としている。そのため、請求書サービスが従業員情報にアクセスできるように、顧客サービスに従業員APIを作成している。JustSocial社のSusanne Kaiserは、同社がマイクロサービス移行の中でうまく使えたことから、このパターンを私に教えてくれた。このパターンには多くの利点があるが、驚くことにあまり知られていない。

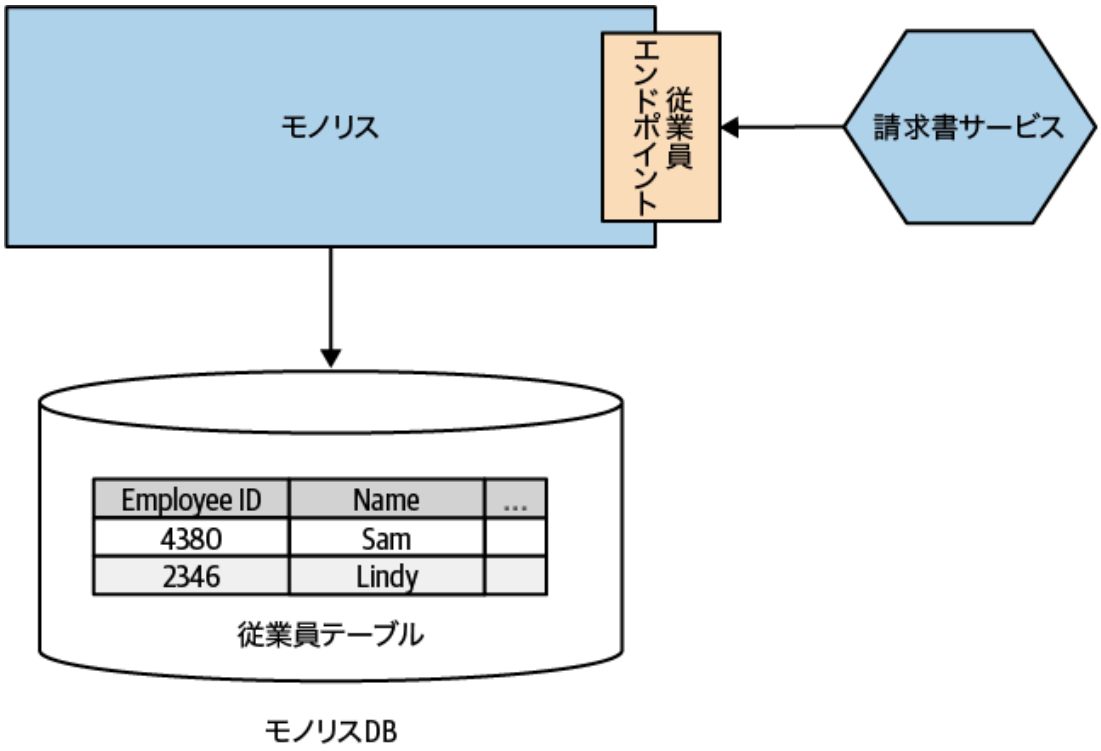


図4-30　モノリスでAPIを公開することで、サービスは直接のデータバインディングを回避できる

このパターンが広まっていない理由は、人々の心の中に「モノリスは死んだ、何の役にも立たない」という考えがあるからかもしれない。人々はそれから離れたいと思っていて、それをより便利にしようとは考えていないのだ。しかし、このパターンの利点は明らかだ。データの分解にまだ取り組まないとしても、情報を隠蔽できる。それによって、新しいサービスをモノリスから分離しやすくなる。私の場合は、モノリス内にデータをとどめておく必要があると感じた場合にこのモデルを採用する傾向にある。特に、新しいサービスが実質的にステートレスになりそうなら、このモデルはうまくいくはずだ。

このパターンは、他のサービス候補を識別する方法としても捉えられる。このアイデアを拡張すると、図4-31に示すように、従業員APIがモノリスから分離され、それ自体がマイクロサービスになることが分かる。

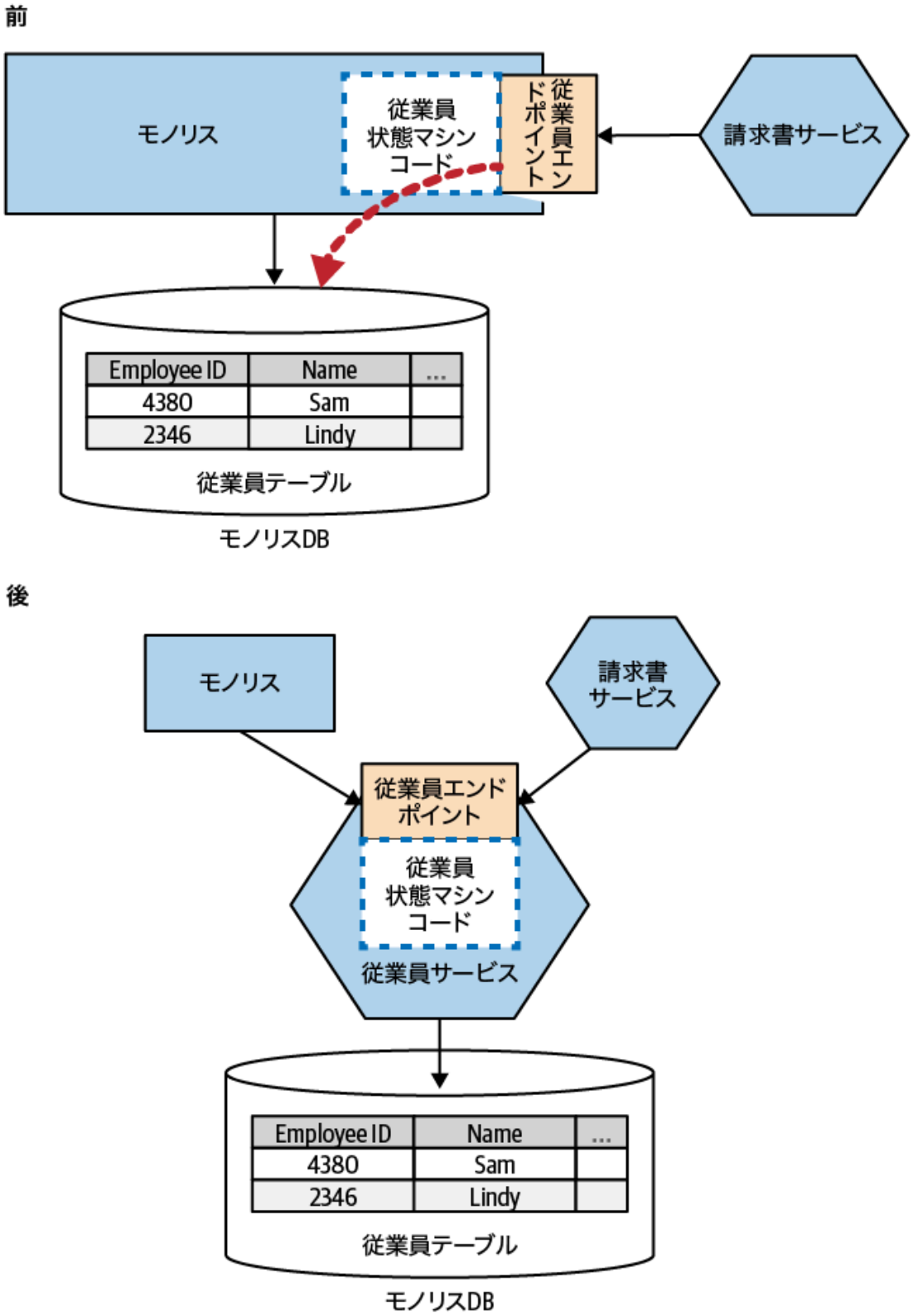


図4-31　従業員APIを使用して、モノリスから分割される従業員サービスの境界を識別する

使いどころ

このパターンは、データを管理するコードがモノリスの中に残っている場合に最適だ。前に話したように、データまで踏まえてマイクロサービスを考える一つの方法が、状態とその遷移を管理するコードのカプセル化だ。つまり、デー

タの状態遷移がモノリスによって提供されている場合には、その状態にアクセス（または変更）したいマイクロサービスは、モノリスでの状態遷移を経由する必要があるということになる。

モノリスのデータベースから参照しようとしているデータが本当はマイクロサービスによって「所有」されるべきなら、このパターンをスキップして、代わりにデータを分割することをお勧めする。

パターン：複数スキーマストレージ

既に説明したように、悪い状況はそれ以上悪化させるべきではない。あるデータベースのデータを直接利用しているからといって、マイクロサービスが保存する**新しい**データもそこに入るべきだということにはならない。図4-32には、請求書サービスの例を示している。請求のコアデータはまだモノリスにあり、現在もそこにアクセスしている。この状態で、請求書サービスにレビュー機能を追加した。これはモノリスにはない新しい機能だ。これをサポートするには、レビュー担当者を請求書ID（Invoice ID）にマッピングし、テーブルに保存する必要がある。この新しいテーブルをモノリスに入れてしまうと、モノリスのデータベースが大きくなってしまう。代わりに、ここではこの新しいデータを請求書サービスのスキーマに格納した。

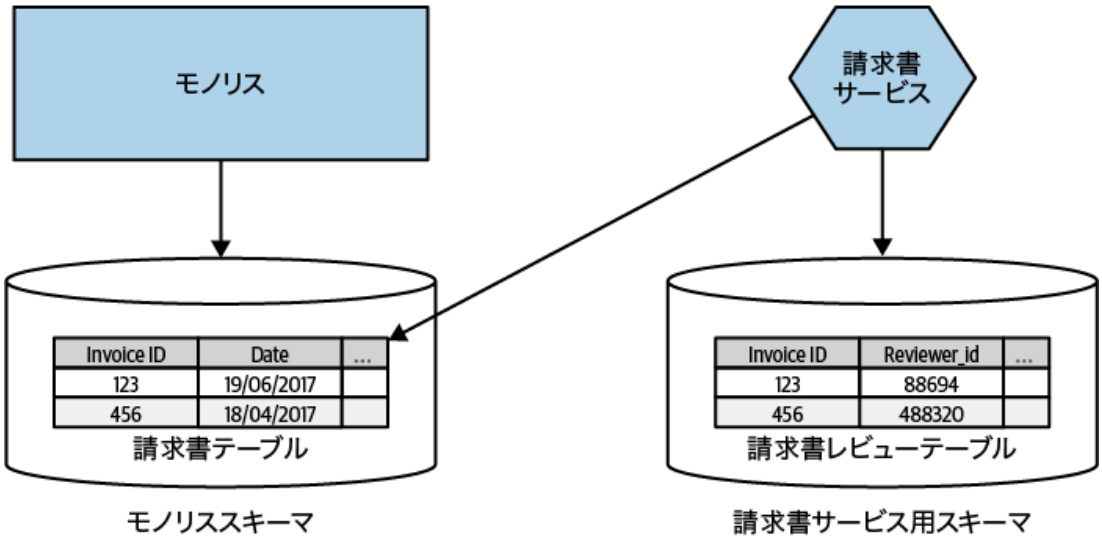


図4-32　請求書サービスは新しいデータを独自のスキーマに格納するが、モノリス内の古いデータにも直接アクセスする

この場合には、外部キー関係がスキーマ境界をまたがっている場合にどうなるかを考えなければならない。章の後半では、この問題についてさらに深く掘り下げていく。

モノリシックデータベースからデータを取り出すのは時間がかかるし、一度にできることではない。そのため、マイクロサービスが独自のローカルストレージを管理する一方で、モノリシックDB内のデータにもアクセスすることを許容しなくてはならない。モノリスから残りのデータをきれいに引っ張り出せるなら、そのときには新しいスキーマに一度にテーブルを移行できる。

使いどころ

このパターンは、新しいデータの保存を必要とするマイクロサービスに新しい機能を追加する場合にうまく機能する。モノリスが必要とするデータではないことは明らかなので（モノリスにその機能はないからだ）、最初から分離しておける。このパターンは、モノリスから独自のスキーマにデータを移動し始めるときにも意味がある。

モノリスのスキーマでアクセスしているデータが、自分のスキーマに移動する予定のないデータである場合、このパターンと合わせて「パターン：データアクセス層としてのモノリス」を使用することを強くお勧めする。

4.11.3　データベースとコードを一緒に分割する

もちろん、図4-33に示すように、一度の大きなステップで分割を行うという選択肢もある。コードとデータの両方を一度に分割するというやり方だ。

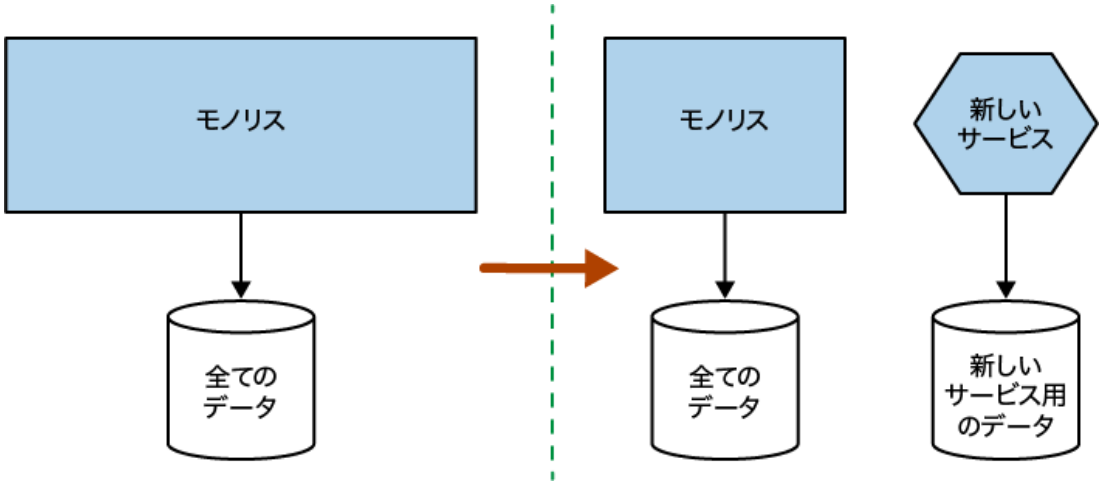


図4-33　コードとデータの両方を一度に分割する

懸念となるのは、この方法はずっと大きなステップであり、決断の効果を評価できるまでに時間がかかってしまうことだ。このアプローチを避け、最初にスキーマ層かアプリケーション層のどちらかを分割することを私としては強くお勧めする。

4.11.4　結局どちらを最初に分割すべきか？

「場合による」という答えには、ほとほとうんざりしていることだろう。言いたいことはよく分かる。それを責めることはできない。問題は、同じ状況にいる人は一人としていないということだ。そのため、十分な文脈とさまざまな長所と短所を述べることによって、判断に役立つような情報をあなたに提供しようとしている。しかし、時に人々が望むのはこれらについての少しばかり刺激的な意見であることを私は知っているので、ここでそれを述べておこう。

モノリスの変更が可能で、パフォーマンスやデータの一貫性への潜在的な影響を懸念している場合には、まずスキーマを分割することを考えよう。それ以外の場合には、コードを分割して、それを使ってデータの所有権にどのような影響を与えるかを理解しよう。しかし、自分自身で考えることも重要だ。あなたが直面している特定の状況における意思決定プロセスに影響を与えうる、あらゆる要因を考慮に入れよう。

4.12　スキーマ分割の例

ここまでは、かなり高いレベルでのスキーマ分離について見てきた。しかし、データベースの分解には複雑な課題があり、いくつかの難しい問題もある。ここでは、さらに低レベルでのデータ分解パターンを見て、その影響を探っていく。

リレーショナルデータベースとNoSQL

この章で詳しく説明するリファクタリング例の多くは、リレーショナルスキーマを扱う際に発生する課題を扱っている。この種のデータベースが持つ性質は、スキーマを分離するという点でさらなる課題を生み出す。リレーショナルデータベース以外のデータベースを使用しているケースも多いかもしれないが、以下のパターンの多くはそうした場合にも依然として適用可能だ。変更をどのように行うかについての制約は少なくなるかもしれないが、このアドバイスが引き続き役に立つことを願っている。

4.13　パターン：テーブルの分割

時には、1つのテーブル内のデータを2つ以上のサービス境界をまたいで分割する必要があり、それが興味深い状況になることがある。図4-34では、商品テー

ブルという、複数の機能に共有されているテーブルがある。このテーブルには販売中の商品だけでなく、在庫水準の情報も格納されている。

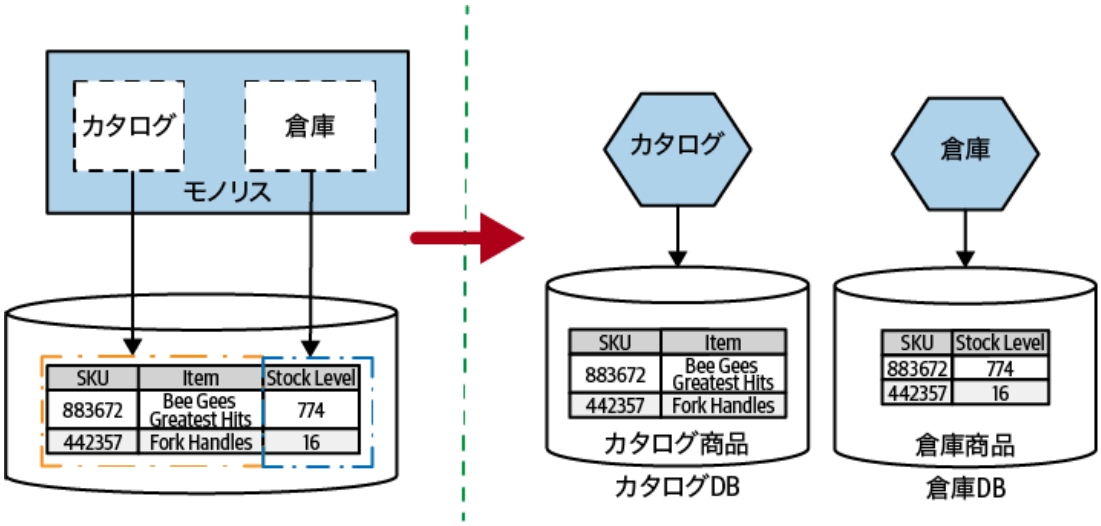


図4-34　分割されている2つの境界づけられたコンテキストの橋渡しをしている単一のテーブル

この例では、カタログと倉庫を新しいサービスとして分割したい。しかし、両方のデータがこの1つのテーブルに混在している。そこで、図4-34に示すように、このデータを2つの別々のテーブルに分割する必要がある。段階的移行の精神からすると、スキーマを分離する前に、既存のスキーマの中でテーブルを分割するのが理にかなっているかもしれない。これらのテーブルが単一のスキーマに存在するなら、倉庫商品テーブルのSKU列からカタログ商品テーブルへの外部キー関係を宣言することは妥当だろう。しかし、これらのテーブルは最終的には別々のデータベースに移動させる予定であり、データの一貫性を強制する単一のデータベースを持たなくなる。そのため、この方法ではあまり恩恵を得られないかもしれない（この考え方については後ほど詳しく説明する）。

データの所有権を列ごとに簡単に分離できるため、この例はとても簡単だ。しかし、複数のコードが同じ列を更新する場合はどうなるだろうか？　図4-35に示す、ステータス列を含む顧客テーブルを考えてみよう。

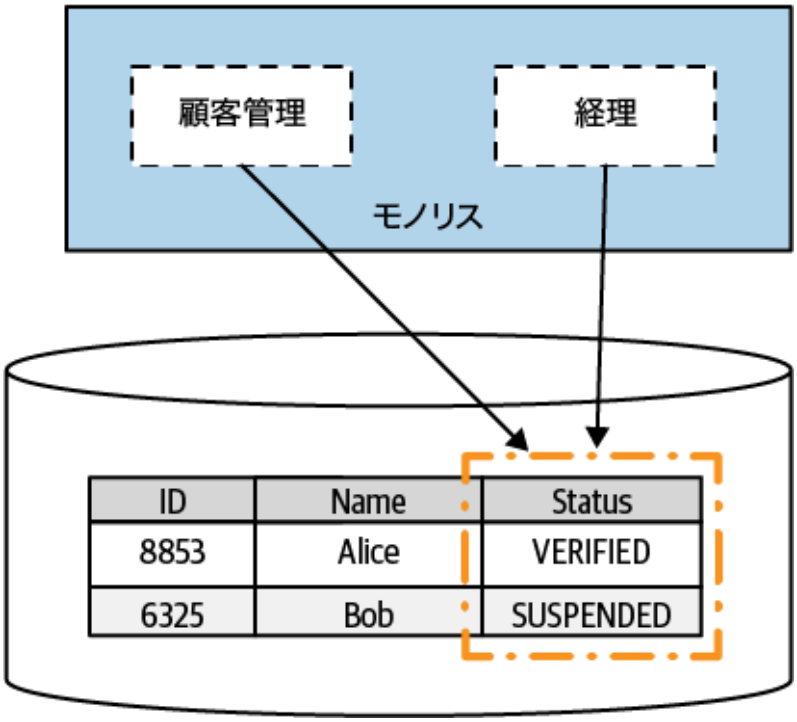


図4-35　顧客管理と経理、両方のコードが顧客テーブルのステータスを変更できる

この列は、顧客のサインアッププロセス中に更新される。特定の人が自分のメールを確認した（またはしていない）ことを示すために、この列は`NOT_VERIFIED`（認証待ち）、`VERIFIED`（認証済み）の順に値が更新される。一度`VERIFIED`に更新されると、顧客は買い物ができるようになる。経理のコードは、請求が未払いの場合に顧客を`SUSPENDED`（利用停止）にする処理を行う。この場合でも、顧客のステータスは依然として顧客ドメインモデルの一部であるべきだと思われる。そのため、この列は間もなく作成される顧客サービスによって管理されるべきだ。覚えておいてほしいのは、可能であればドメインエンティティの状態マシンは単一のサービス境界内に保持しておきたいということ、そして、ステータス列の更新は確かに顧客の状態マシンの一部のように感じられるということだ。このことは、サービスの分割が行われたときには、新しい経理サービスはこのステータスを更新するためにサービス呼び出しを行う必要があることを意味している。これを図4-36に示す。

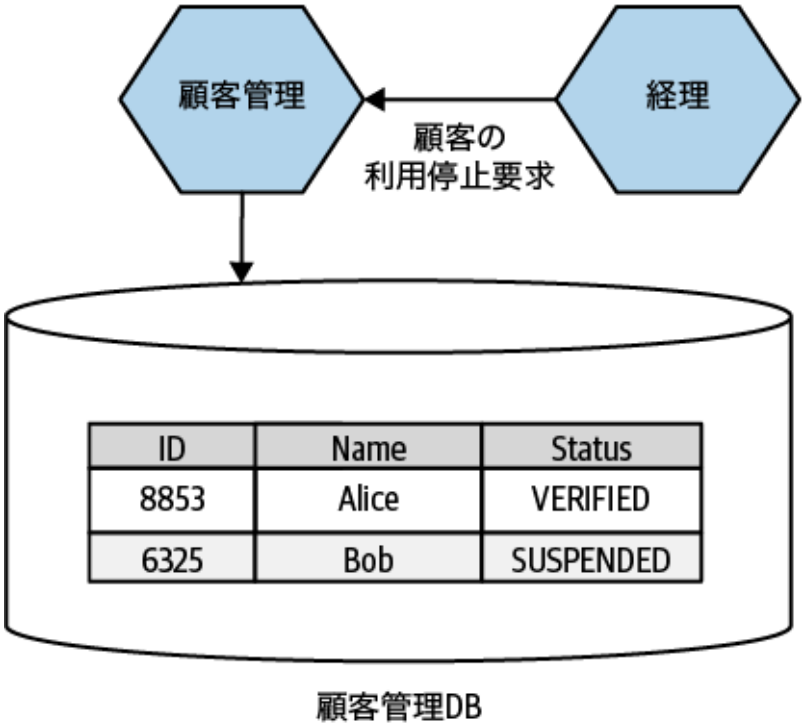


図4-36　新しい経理サービスは顧客を利用停止にするためにサービス呼び出しを行う必要がある

このようにしてテーブルを分割する際の大きな問題は、データベースのトランザクションによって与えられた安全性が失われることだ。「4.15　トランザクション」や「4.16　サーガ」など、この章の最後ではこのトピックをより深く掘り下げていく。

4.13.1　使いどころ

一見すると、このパターンはとても簡単に見える。テーブルが現在のモノリス内の2つ以上の境界づけられたコンテキストによって所有されている場合、これらの線に沿ってテーブルを分割する必要がある。しかし、そのテーブルの特定の列が、コードベースの複数の部分によって更新されているように見える場合には、誰がそのデータを「所有」すべきかを判断する必要がある。それはスコープ内にある既存のドメインの概念なのだろうか？　これを考えることは、このデータの行き先を判断するのに役立つだろう。

4.14　パターン：外部キーのコードへの移動

Music Corpで、アーティスト、曲、アルバムに関する情報を管理・公開できるカタログサービスを抽出することにしたとする。現在、モノリス内のカタログ関連のコードは、販売可能なCDの情報を保存するためにアルバムテーブルを使用している。アルバムテーブルのレコードは、最終的に全ての売り上げを追跡

する元帳テーブルで参照される。図4-37にこれを示す。元帳テーブルの行は、販売されたアルバムを示す識別子（SKU）と共にCDの販売額だけを記録している。SKUとはStock Keeping Unit（ストック・キーピング・ユニット）の略で、小売システムで商品を管理するのに使われる一般的な識別子だ。

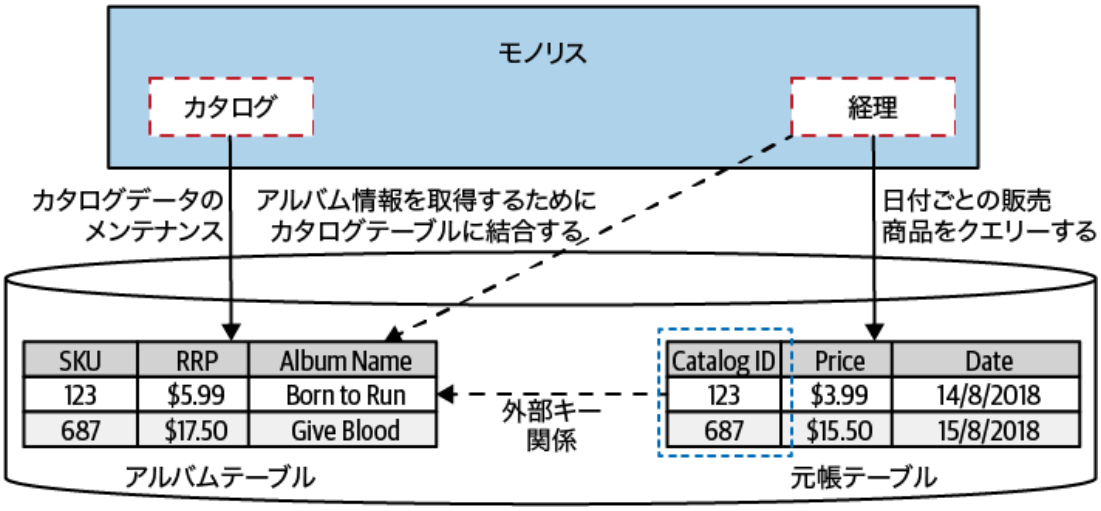


図4-37　外部キー関係

毎月末に、最も売れたCDの概要をまとめたレポートを作成する必要がある。元帳テーブルはどのSKUが最も多く売れたかを理解するのに役立つが、そのSKUに関する情報はアルバムテーブルにある。そのため、「SKU 123を400枚販売し、1,596ドルを売り上げました」と報告する代わりに、「ブルース・スプリングスティーンの『Born to Run』を400枚販売し、1,596ドルを売り上げました」というように販売品についてより多くの情報を追加したい場合には、図4-37のように、経理機能のコードによってトリガーされたデータベースクエリで元帳テーブルとアルバムテーブルの情報を結合する必要がある。

スキーマでは、外部キー関係を定義し、元帳テーブルの行がアルバムテーブルの行との関係を持っていることが分かるようにしている。つまり、元帳テーブルの行がアルバムテーブルの行を参照している場合、その行が存在することが分かる。これによって販売されたアルバムに関する情報が常に得られる。こうした外部キー関係によって、データベースエンジンはパフォーマンスを最適化でき、その結果、結合操作を可能な限り高速にできる。

ここで、カタログ機能のコードと経理機能のコードをそれぞれの対応するサービスに分割したくなったとする。アルバムテーブルと元帳テーブルは、最終的には異なるスキーマに存在することになる。このとき考えるべき重要な問題が2つある。まず、新しい経理サービスが将来このレポートを生成したい場合、データベース結合を行えないとすると、カタログ関連の情報をどのようにして

取得するのだろうか？　そして、データ不整合が新しい世界に存在する可能性があるという事実をどう考えるとよいだろうか？

4.14.1　結合の移動

最初に結合の置き換えを見ていこう。モノリシックシステムでは、アルバムテーブルの行を元帳の売上情報と結合するために、データベース結合を実行する。一つのSELECTクエリを実行して、アルバムテーブルに結合する。これは、クエリを実行して必要なデータを全て引き出すために、データベースを1回呼び出す必要がある。

新しいマイクロサービスベースの世界では、新しい経理サービスはベストセラーのレポート生成を担当するが、アルバムデータを手元に持っていない。この場合は、図4-38に示すように、新しいカタログサービスからこれを取得する必要がある。レポートを生成するとき、経理サービスは、最初に元帳テーブルを参照して先月のベストセラーSKUのリストを抽出する。この時点では、SKUのリストとそれぞれのSKUの販売部数しかない。手元にはそれしか情報がないからだ。

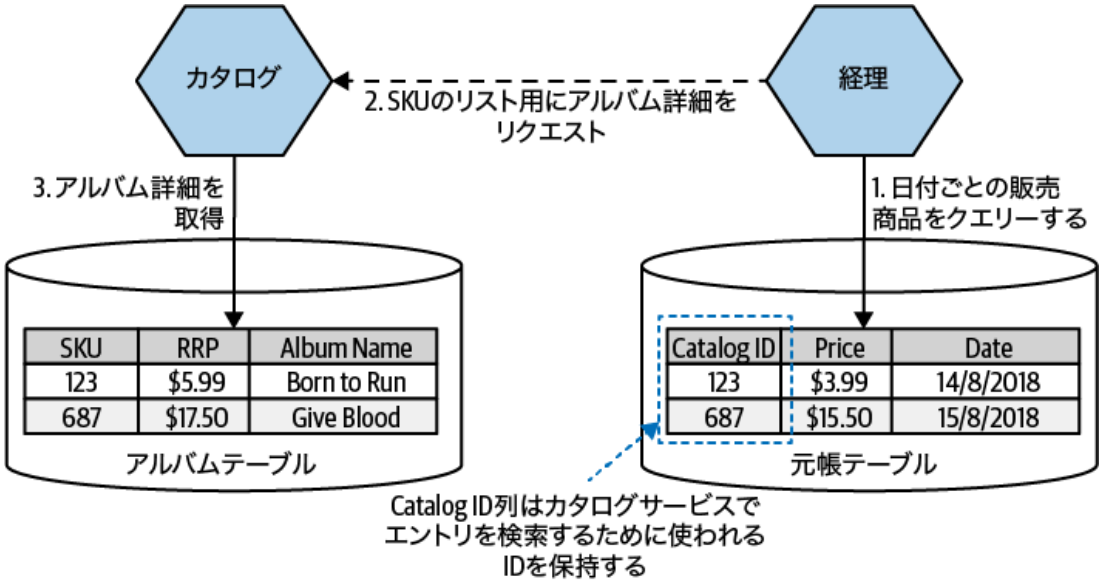


図4-38　データベースの結合操作をサービス呼び出しに置き換える

次に、カタログサービスを呼び出して、これらのSKUの情報を要求する。このリクエストにより、カタログサービスは自身のデータベース上で独自にSELECTクエリを発行する。

論理的には、結合操作は依然として行われているものの、それはデータベース内ではなく、経理サービス内で行われている。残念ながら、これはそれほど効

率的にはならない。1つのSELECTクエリを持つ世界から、元帳テーブルに対するSELECTクエリを持ち、その後にカタログサービスへのサービス呼び出しが行われ、それに従って、アルバムテーブルに対するSELECT文がトリガーされる新しい世界へと変わったのだ。

この状況で操作の全体的な待ち時間が増加しなかったとしたら、それは驚くべきことだ。レポートは月毎に生成されるため、前もってキャッシュしておける。そのため、この特定のケースでは大きな問題ではないかもしれない。しかし、この操作が頻繁に行われている場合は、問題がより大きくなる可能性がある。その場合には、カタログサービスで一括してSKUを検索できるようにするか、必要なアルバム情報をローカルにキャッシュすることで、この待ち時間の増加による影響を軽減できる。

最終的には、このレイテンシーの増加が問題かどうかは、あなただけが判断できるものだ。主要な操作に対する許容可能なレイテンシーを理解し、現在のレイテンシーがどれくらいかを測定できるようにする必要がある。Jaeger[†13]のような分散システムは、複数のサービスにまたがる操作の正確な時間計測をする機能を提供してくれるので、ここでは本当に役立つ。操作を遅くしてもまだ十分に速く、特にトレードオフとして他の恩恵を受けられる場合には、これを許容できるかもしれない。

[†13] https://www.jaegertracing.io

4.14.2　データの一貫性

厄介なのは、カタログと経理が別々のサービスであり、別々のスキーマを使用すると、データの不整合が発生する可能性があるということだ。単一のスキーマでは、アルバムテーブルの行が元帳テーブルに参照されている場合、アルバムテーブルの行を削除することはできない。スキーマがデータの一貫性を強制していたということだ。新しい世界では、そのような強制力は存在しない。この場合、どのような選択肢があるだろうか？

削除前にチェックする

最初の方法は、アルバムテーブルからレコードを削除するときに、そのレコードへの参照が既にないことを経理サービスに問い合わせて確認することだ。こ

こでの問題は、これを正しく実行できるかを保証するのが難しいということだ。たとえば、SKU683を削除したいとする。経理サービスに「SKU683を使用していますか？」と問い合わせる。すると、このレコードは使用されていないとの回答が返ってくる。その後、レコードを削除するものの、削除している間に、経理システムにSKU683への新しい参照が作成されてしまう。このようなことが起こらないようにするには、削除が完了するまでSKU683のレコードに新しい参照が作成されないようにする必要がある。そのためにはロックが必要になるだろうし、ここが分散型システムに伴う最大の課題だ。

管理しているレコードが使用されているかをチェックすることのもう一つの問題は、カタログサービスからの事実上の逆依存関係が生じることだ。そのレコードを使用している他のサービスをチェックする必要が出てくる。保有する情報を使用しているサービスが他に1つしかない場合でもそれは十分問題だが、コンシューマーが増えるにつれてこの問題はずっと深刻になる。

この操作が正しく実装されているかを確認するのが困難であることに加えてサービスの結合度も高まることから、この選択肢は検討しないことを強くお勧めする。

円滑に削除を処理する

より良い選択肢は、カタログサービスがアルバムの情報を持っていないかもしれないという事実を、経理サービスが円滑な方法で処理することだ。これは、指定されたSKUを検索できない場合、レポートに「アルバム情報が利用できません」と表示させるのと同じくらい単純なアプローチだ。

この状況では、カタログサービスは、以前は存在していたSKUを要求したということを教えることが可能だ。HTTPを使用している場合であれば、`410 GONE` レスポンスコードを使用すると良いだろう。410レスポンスコードは、一般的に使用される404とは異なる。404は要求されたリソースが見つからないことを表すが、410は要求されたリソースが利用可能であったが、もはや存在しないことを表す。この区別は、特にデータの不整合の問題を追跡する際に重要になる。HTTPベースのプロトコルを使用していない場合でも、この種のレスポンスをサポートすることでメリットが得られないかを検討するのがよいだろう。

さらに高度な機能を利用したい場合は、イベントをサブスクライブすることで、カタログのアイテムが削除されたときに経理サービスに通知されるように

もできる。カタログ削除イベントをピックアップすれば、削除されたアルバム情報をローカルデータベースにコピーすることも可能だ。これは、この特定の状況ではやりすぎのように感じるものの、他のシナリオ、特にサービスの境界を越えてカスケード削除のようなことを実行する分散状態マシンを実装したい場合に便利な方法だ。

削除を許可しない

システムにあまり多くの矛盾を導入しないようにするための一つの方法として、カタログサービスのレコードを削除できないようにする方法が考えられる。既存のシステムで商品を削除することが、それが販売されていないことを保証することに似ている場合、論理削除機能を実装できる[†14]。論理削除はステータス列を設けて、利用できないようにその行をマークするか、あるいは、専用の「墓地」テーブルに行を移動することで実現できる。そして、論理削除されている場合でも、経理サービスはそのアルバムのレコードを要求できるようにする。

[†14] このようなリレーショナルデータベースで履歴データを維持することは、特にエンティティの古いバージョンをプログラムで再構成する必要がある場合は、複雑になる。もし、この領域での要件が厳しい場合は、状態を維持するための代替手段としてイベントソーシングを検討するとよいだろう。

では、どうやって削除を処理すべきか？

私たちはモノリシックなシステムには存在し得ない障害モードを作ってしまったことになる。これを解決する方法を検討するには、ユーザーのニーズを考慮しなければならない。したがって、適切なソリューションを選択するには、あなたの直面している具体的な状況を理解する必要がある。

個人的には、こうした状況では、2つの方法を用いてこれを解決するだろう。カタログ内のアルバム情報の削除を許可しないようにする方法と、経理サービスが欠落したレコードを処理できるようにする方法だ。レコードをカタログサービスから削除できないようにすれば、それによるデータの破損は起こらないため、経理サービスからの検索が失敗することはないと主張できる。しかし、何らかの理由によってデータに破損が生じた場合には、結果としてカタログサービスが過去の状態に巻き戻され、探しているレコードが存在しなくなる可能

性がある。そうなったときに、経理サービスがボロボロになってしまうのは避けたいだろう。そんな状況は起こらないと考えるかもしれない。しかし、私は常に回復性を持つシステムの構築を模索しており、呼び出しが失敗するケースを考慮するようにしている。経理サービスでこれを円滑に処理するのは、思いの外、簡単な修正に見える。

4.14.3　使いどころ

外部キー関係を効果的に分解しようと考えたとき、最初に確認しなければならないのは、本当は1つであるべきものを2つに分割していないかということだ。集約をバラバラにしているのではないかと心配な場合には、いったん立ち止まって考え直してみてほしい。ここでの元帳とアルバムのケースでは、2つの異なる集約があり、それらの間に関係があることは明らかだ。しかし、別のケースではどうだろうか。注文テーブルと注文した商品の詳細を含む注文明細テーブルには、多くの関連する行がある。注文明細を別のサービスに分割した場合、データの整合性の問題が発生する。実際には、注文明細の各行は注文自体の一部だ。したがって、それらは1つのユニットとして見るべきであり、モノリスの外に移動させたい場合は、それらを一緒に移動させなくてはならない。

時には、モノリスのスキーマから一部を大きく削り取ることで、外部キー関係の両側を一緒に移動させられるかもしれない。その場合は、データベースの分解はずっと簡単だ。

4.14.4　例：共有静的データ

静的参照データ(まれにしか変更されないが、一般的に重要なデータ)は興味深い課題を引き起こす可能性があり、それを管理するためのさまざまなアプローチを過去に見てきた。ほとんどのケースでその方法はデータベースになる。私は、Javaプロジェクトで独自の`StringUtils`クラスを書いたのと同じくらい、国コードがデータベースに保存されているのを見てきた（図4-39参照）。

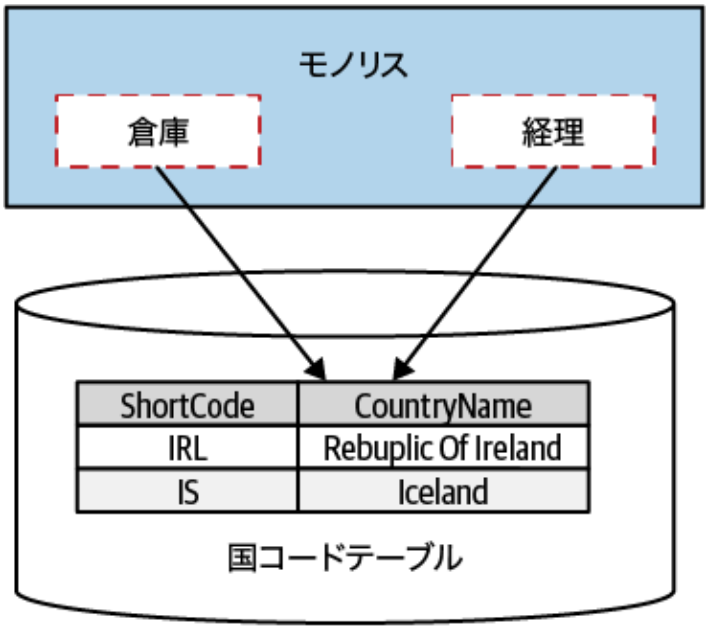


図4-39　データベースに保存されている国コード

国コードのように頻繁には変化しない少量のデータがなぜデータベースに保存されている必要があるのか、私はいつも疑問に思っていた。しかし、根本的な理由が何であれ、こうしたデータベースに保存される共有静的データの例はたくさんある。では、コードのいろいろな部分が同じ静的参照データを必要とする音楽ショップで何をすべきだろうか。ここで、多くの選択肢があることを明らかにしよう。

パターン：静的参照データの複製

図4-40に示すように、各サービスに独自のデータのコピーを持たせるのはどうだろうか。これには読者の多くが息を飲むことだろう。データが重複している？　狂ってるんじゃないか？　聞いてほしい！　私はあなたが思うほどクレイジーではない。

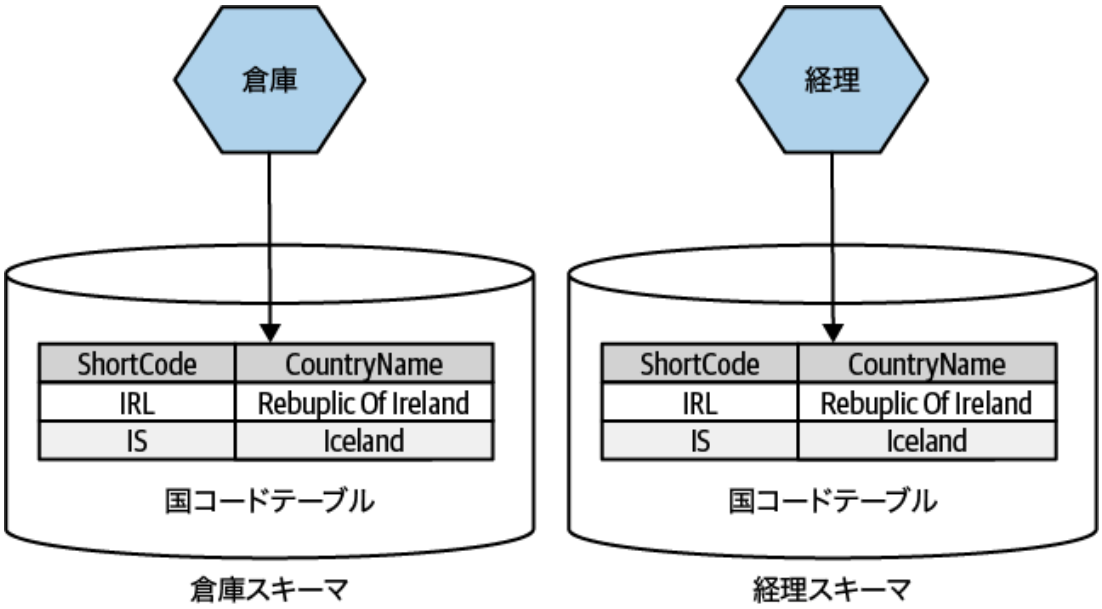


図4-40　各サービスに独自の国コードのスキーマを持つ

データの重複についての懸念は、2つのことに行き着く傾向がある。1つ目は、データを変更する必要があるたびに、複数の場所で変更しなければならないことだ。しかし、この状況において、データはどのくらいの頻度で変更されるだろうか？　最後に国が作られて正式に認められたのは2011年で、南スーダン（国コードはSSD）が作られたのが最後だ。そのため、この懸念はあまり心配する必要はないと私は考えている。もっと心配なのは、データの整合性が取れていない場合はどうなるのかということだ。たとえば、経理サービスは南スーダンという国を知っているのに、なぜか倉庫サービスのデータは古く、南スーダンについて何も知らないという場合だ。

矛盾が問題になるかどうかは、データがどのように使われているかによる。倉庫サービスがこの国コードデータを使用して、CDが製造されている場所を記録しているとする。南スーダンで製造されたCDの倉庫はなかったため、このデータがないことは問題にならない。一方、経理サービスでは、売上情報を記録するために国コード情報が必要で、南スーダンに顧客がいるため更新が必要だ。

各サービス内でローカルにしか使われていないデータであれば、不整合は問題にならない。境界づけられたコンテキストの定義を思い出してほしい。境界づけられたコンテキストでは、境界の内側に情報が隠される。一方で、データがこれらのサービス間通信の一部である場合は、異なる懸念がある。倉庫サービスと経理サービスの両方が同じように国情報を扱う必要があるのであれば、このような性質の重複は間違いなく懸念事項となる。

もちろん、何らかのバックグラウンドプロセスを使用して、これらのコピーを同期させておくことも考えられる。このような状況では、全てのコピーが一貫していることを保証はできない。しかし、バックグラウンドプロセスが十分に頻繁に（そして迅速に）実行されると仮定すれば、データの不整合の可能性を低くでき、それで十分な可能性がある。

開発者である私たちは、重複を目にしたとき、しばしば悪い反応を示す。情報の複製を管理するための余分なコストを心配し、重複したデータに微妙な差異があると、さらに心配になる。しかし、時には重複は2つの悪いことのうちのマシな方の場合もある。データの重複を受け入れることは、結合の発生を避けるという意味では、賢明なトレードオフかもしれない。

使いどころ

このパターンが使用されることは滅多にない。たいていはこの後で検討する選択肢の方を選ぶべきだろう。このパターンは、全てのサービスが全く同一のデータセットを見る必要がなく、かつ大量のデータを扱う場合には有用だ。イギリスの郵便番号ファイルのように、郵便番号から住所へのマッピングを定期的に更新するようなものはこれに適しているかもしれない。これはかなり大規模なデータセットで、コード形式で管理するのは大変だろう。もし直接このデータに結合したいのであれば、それもこの選択肢を選ぶ理由になるかもしれない。しかし正直に言うと、私自身はそれを選んだ覚えはない。

パターン：専用の参照データスキーマ

もし本当に国コードの真の情報源を一つにしたいのであれば、図4-41に示すように、このデータを専用のスキーマに移すことも可能だ。

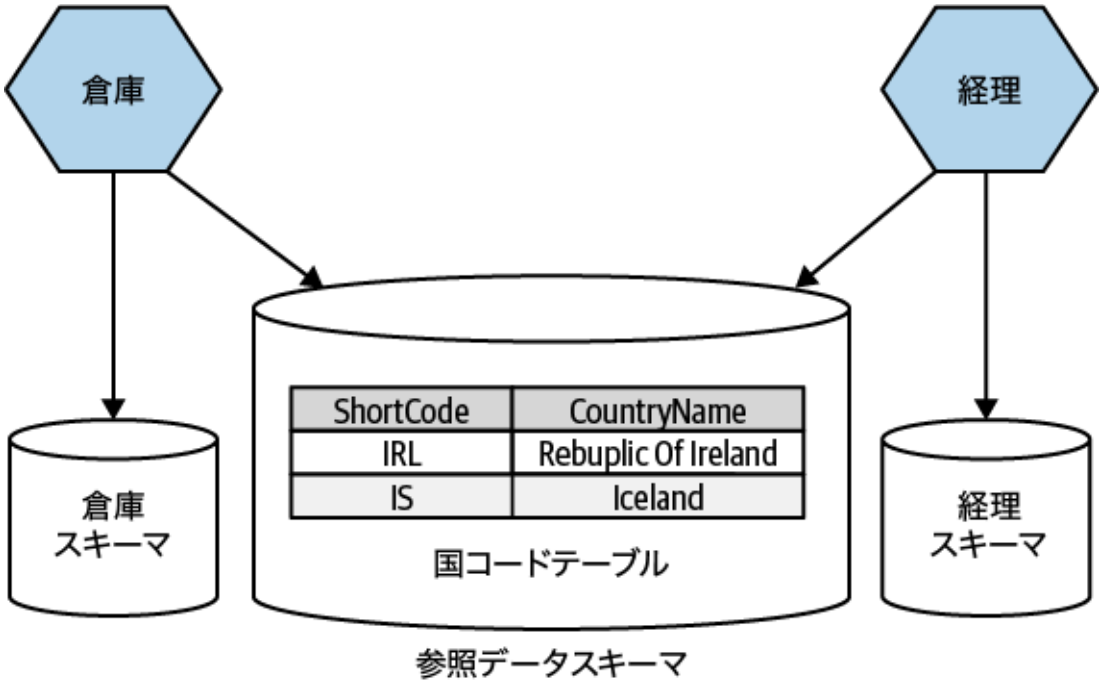


図4-41　参照データに専用の共有スキーマを使用する

この場合は、共有データベースの全ての課題を考慮しなければならない。結合と変更に関する懸念は、データの性質によっていくらかは相殺される。変更頻度が低く、単純に構造化されているので、この参照データスキーマを定義されたインターフェイスと考えると簡単だ。この状況では、参照データスキーマを独自にバージョン管理されたエンティティとして扱い、スキーマ構造がコンシューマーへのサービスインターフェイスを表していることを理解してもらう必要がある。このスキーマを壊すような変更を加えるのは、痛みを伴う可能性が高い。

スキーマでデータを保持することで、サービスがローカルデータの結合クエリの一部としてこのデータを使用する機会が増える。しかし、これを実現するには、スキーマが同じ基礎となるデータベースエンジン上にあることを確認する必要があるだろう。そのため、潜在的な単一障害点に対する懸念をさておいたとしても、論理世界から物理世界へのマッピング方法が複雑になる。

使いどころ

この選択肢には多くのメリットがある。重複に関する懸念を回避でき、データのフォーマットが変更される可能性が非常に低いため、結合に関する懸念のいくつかが軽減される。大量のデータや、クロススキーマ結合のオプションが必要な場合には、有効なアプローチだ。ただし、スキーマフォーマットの変更は、複数のサービスに大きな影響を与える可能性が高いことを覚えておいてほしい。

パターン：静的参照データ用のライブラリ

突き詰めていくと、国コードデータにはあまりエントリがない。ISO標準のリストを使用していると仮定すると、表現されているのは249カ国だけだ[†15]。これはコードの中では、おそらく単純な静的列挙型としてうまく収まるものだろう。実際、少量の静的参照データをコード形式で格納することは、私も何度も行ってきたし、マイクロサービスアーキテクチャで行われているのも見てきた。

[†15] ISOファンの皆さんにはおなじみのISO 3166-1だ。

もちろん、必要ないのであればこのデータを複製したくない。そのため、このデータを必要とするあらゆるサービスが静的にリンクできるライブラリにこのデータを配置することを検討することになる。アメリカに拠点を置くファッション小売業のStitch Fixは、静的参照データを保存するために、そうした共有ライブラリを頻繁に利用している。

Stitch Fixのエンジニアリング担当VPであるRandy Shoupによると、この手法が最もよく機能するのは、データ量が少なく、変更が頻繁に行われない、あるいは全く行われない（仮に変更がある場合には、変更について事前に多くの警告があるような）種類のデータだという。標準的な服のサイズであるXS、S、M、L、XL、またはズボンの股下の測定を考えてみてほしい。

図4-42では、国コードのマッピングを`Country`の列挙型で定義し、サービスで使用するためのライブラリにバンドルしている。

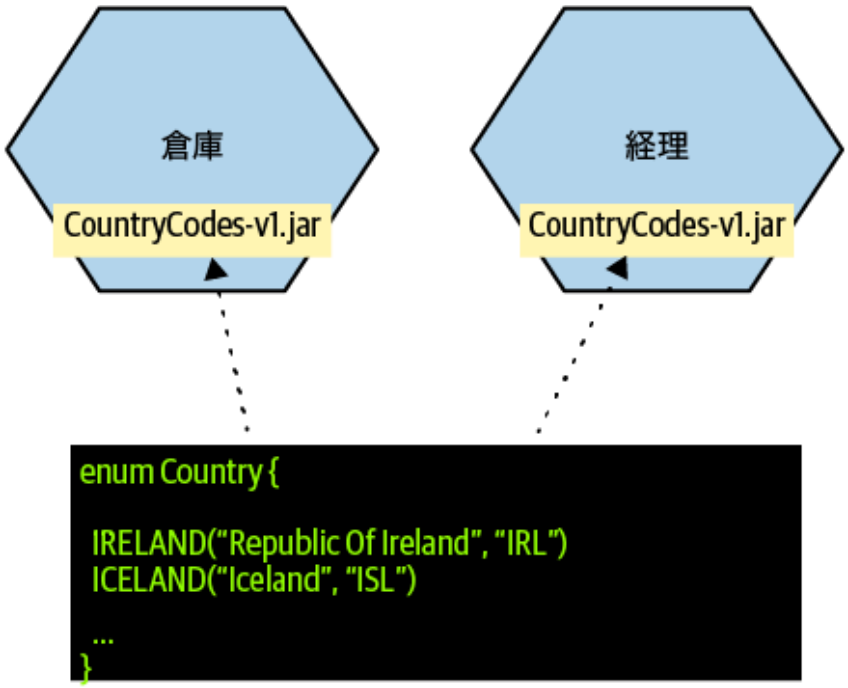


図4-42　サービス間で共有できるライブラリに参照データを格納する

これはきちんとしたソリューションだが、欠点がないわけではない。技術スタックが混在している場合には、単一の共有ライブラリを共有できない可能性がある。また、マイクロサービスの黄金律を覚えているだろうか？　マイクロサービスは独立してデプロイできなくてはならない。国コードライブラリを更新する際に、全てのサービスが新しいデータをすぐに取り込む必要があると、新しいライブラリが利用可能になった瞬間に全てのサービスを再デプロイしなくてはならない。これは典型的なロックステップリリース[†16]であり、まさにマイクロサービスアーキテクチャで避けようとしているものだ。

[†16] 訳注：2つ以上のサービスを足並みを揃えて一斉にリリースしなければならないこと。

実際には、同じデータをどこでも利用できるようにする場合には、変更を十分に通知することが助けになるかもしれない。Randyから聞いた事例を挙げよう。Stitch Fixで、データセットの一つに新しい色を追加する変更を行った時のことだ。この変更は、そのデータ型を使用している全てのサービスに公開する必要があるもので、全てのチームがデータセットの最新のバージョンを取り込んでいることを確認するのに、かなりのリードタイムを必要としたそうだ。国コードの例を考えてみると、新しい国を追加する必要がある場合には、多くの事前通知が必要になる。たとえば、南スーダンであれば、半年前に国民投票

が行われた後、2011年7月に独立した。新しい国が気まぐれで作られることは滅多にないはずだ。

マイクロサービスが共有ライブラリを使用している場合、本番環境には異なるバージョンのライブラリがデプロイされている可能性があることを受け入れる必要がある。

つまり、国コードライブラリを更新する必要がある場合、図4-43に示すように、全てのマイクロサービスが同じバージョンのライブラリを持つことが保証されているわけではないことを受け入れる必要があるということだ。これがうまくいかない場合は、この次に紹介する選択肢が役立つかもしれない。

このパターンの簡略化されたバージョンでは、問題のデータは設定ファイル（標準のプロパティファイル）か、必要に応じて、より構造化されたJSON形式で保持される。

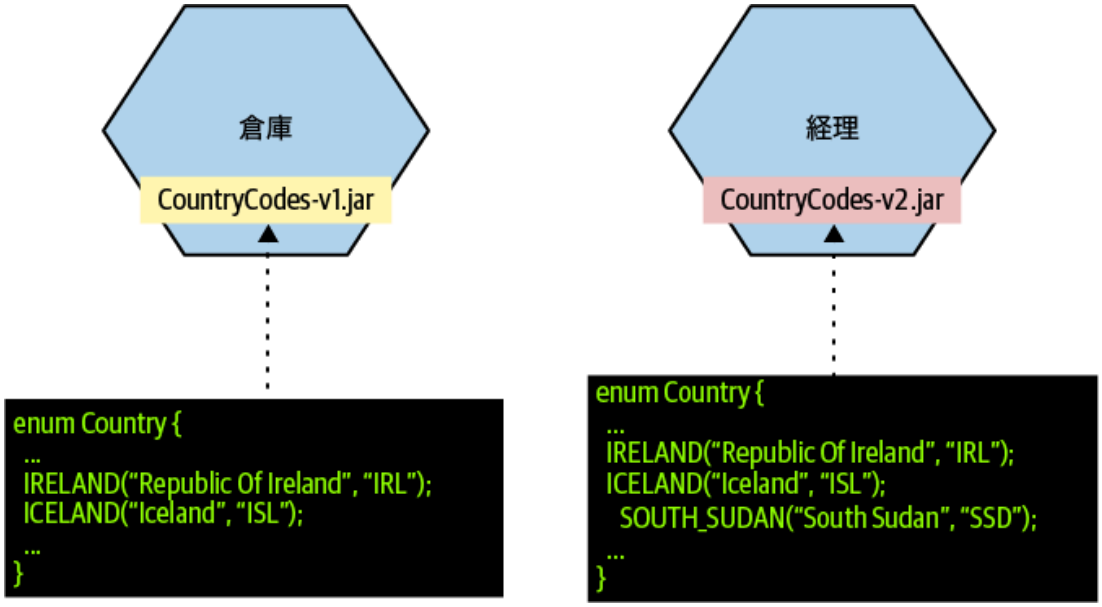


図4-43　共有参照データライブラリ間の違いが問題を引き起こす可能性がある

使いどころ

見落とされがちだが、データが多くなく、異なるサービスが異なるバージョンのデータを参照することが許容可能な場合には、これは優れた選択肢だ。どのサービスがどのバージョンのデータを持っているかの可視性は特に便利だ。

パターン：静的参照データ用のサービス

そろそろ、この話も終盤だ。これはマイクロサービスを作るという本なので、図4-44のように、国コード用のサービスを作ることを考えてみてはどうだろうか？

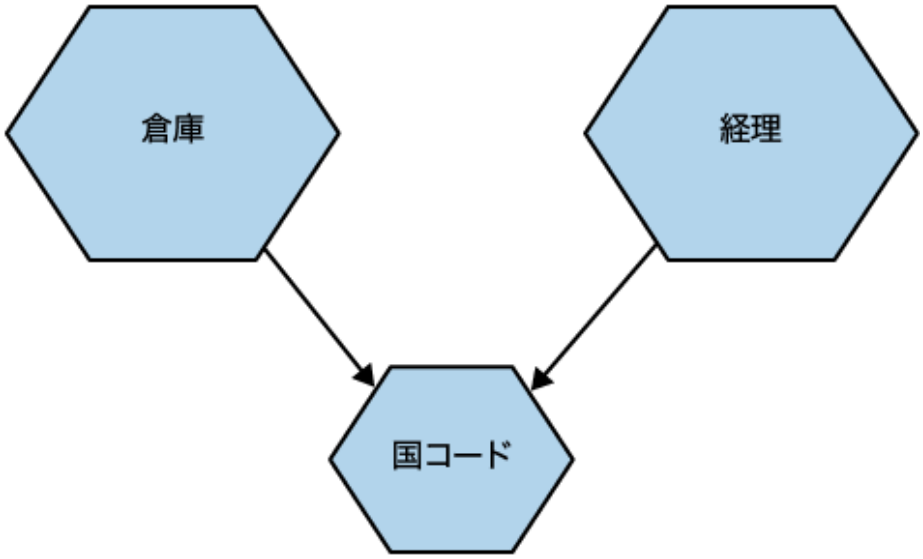


図4-44　専用サービスから国コードを提供する

私は世界中でこれを題材としたグループワークを実施してきたが、国コード用の専用サービスを作ることに関しては、たいていは参加者の間で意見が二つに分かれた。何人かの人はすぐに「それはうまくいく！」と考える。しかし、一般的には、グループの大部分は頭を振って、「それはクレイジーに見える！」という意見に立って何かを言い始める。もっと深く掘り下げてみると、そうした立場を取る人々が懸念していることの核心へと迫ることが可能だ。そうすると、「やりすぎ」という言葉が頻繁に出てくることに気付く。

もう少し掘り下げてみよう。私が人々と話をして、なぜこのアイデアに賛成する人と反対する人がいるのかを理解しようとするとき、それは一般的にいくつかのことに帰着する。マイクロサービスの作成と管理の敷居が低い環境で働いている人は、この選択肢を検討する可能性が高い。一方で、新しいサービスを

作成するのに何日も、あるいは何週間もの作業が必要な場合は、このようなサービスを作成することに抵抗を感じるのも無理はない。

元同僚で、『Infrastructure as Code』（オライリー・ジャパン）[24]の著者でもあるKief Morrisは、イギリスに拠点を置く大規模な国際銀行のプロジェクトでの経験を私に話してくれた。そこでは、ソフトウェアを最初にリリースする際には、その承認を得るのに1年近くかかったそうだ。設計の承認から導入のためのマシンのプロビジョニングに至るまで、何かを始める前には、銀行内の10以上のチームにまず相談しなければならなかった。このような経験は、残念ながら大規模な組織では珍しいことではない。

新しいソフトウェアをデプロイするのに多くの手作業や承認、さらには新しいハードウェアを調達して設定する必要がある組織では、サービスを作成するためにかかるコストが大きくなる。そのような環境では、新しく作成するサービスは厳選されている必要がある。余分な作業を正当化するために多くの価値を提供しなければならなくなるため、国コードサービスのようなものを作ることは正当化されないかもしれない。一方で、サービスのテンプレートをスピンアップして、1日以内にそれを本番環境に押し出し、全てを完了できれば、これを実行可能な選択肢として検討する可能性ははるかに高くなるだろう。

さらに良いことに、国コードサービスは、Azure Cloud FunctionsやAWS LambdaのようなFaaS（Function-as-a-Service）プラットフォームとの相性が良い。そうしたプラットフォームは機能の運用コストの低さが魅力的であり、国コードサービスのようなシンプルなサービスとの相性は抜群だ。

もう一つの懸念として挙げられているのは、国コードサービスを追加することで、ネットワークに依存した別のサービスを追加することになり、レイテンシーに影響を与える可能性があるということだ。私は、このアプローチは、この情報のための専用のデータベースを持つよりも悪くはなく、より速くすることも可能だと考えている。どうしてか？　既にはっきりしているように、このデータセットには249個のエントリしかない。私たちの国コードサービスは、これを簡単にメモリに保存して直接提供できる。国コードサービスはこれらのレコードをコードで保存するだけなので、データストアを作成する必要はない。

もちろん、このデータはクライアント側で前もってキャッシュすることもできる。結局のところ、このデータに新しいエントリを追加することはあまりない。図4-45に示すように、このデータが変更されたときにコンシューマーに知らせるためにイベントを使うことも考えられる。データが変更されると、興味

のあるコンシューマーはイベントを通じてアラートを受け取り、これを使ってローカルキャッシュを更新できる。変化の頻度が低いことを考えると、このシナリオでは伝統的なTTLベースのクライアントキャッシュで十分だと思うが、私は何年も前に汎用の参照データサービスで同様のアプローチを使用して大きな効果を得た経験がある。

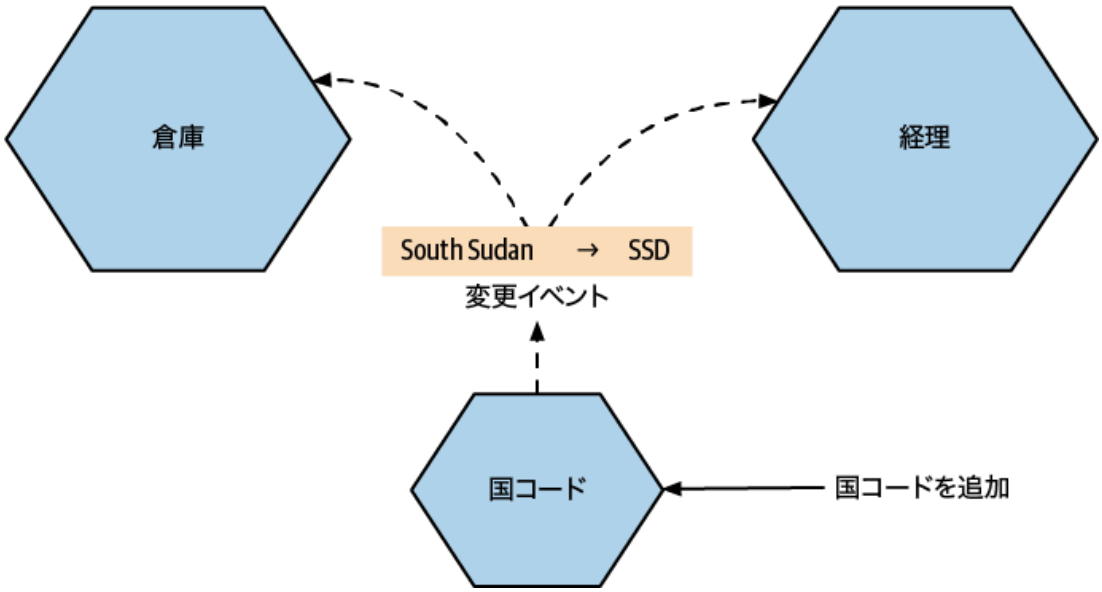


図4-45　コンシューマーがローカルキャッシュを更新できるように更新イベントの発行する

使いどころ

データ自体のライフサイクルをコードで管理している場合には、この選択肢に手を伸ばすだろう。データを更新するAPIを公開する際に、データのライフサイクルをコードで管理する場合には、専用のマイクロサービスがそれを行うのは理にかなっている。そのマイクロサービスは、そのデータの状態マシンを包含するサービスとなる。これは、データが変更されたときにイベントを発生させたい場合や、テスト目的でスタブを作成するために、より便利なインターフェイスを提供したい場合にも理にかなっている。

ここでの大きな問題は、マイクロサービスを新たに作成するのにかかるコストに常に行き着く。その作業は十分に正当化できるだろうか、それとも他のアプローチの方がより賢明な選択肢だろうか。

どうするか？

またしても、たくさんの選択肢をあげてしまった。つまるところ、どうすればいいだろうか？　いつまでもはっきりしない態度でいるわけにもいかないの

で、こうしよう。全てのサービスで常に国コードが一貫していることを保証する必要がないと仮定するなら、この情報は共有ライブラリに保存することにするだろう。国コードや服のサイズのような単純で小さいデータについては、データをローカルサービススキーマに複製するよりも、共有ライブラリで管理するほうがはるかに理にかなっている。より複雑な参照データや大規模なボリュームの場合には、これを各サービスのローカルデータベースに入れてしまうのがいいかもしれない。

サービス間でデータの一貫性を持たせる必要がある場合は、専用のサービスを作成することを検討しよう（あるいは、大規模な静的参照サービスの一部としてこのデータを提供することも考えられる）。新しいサービスを作成するための作業を正当化するのが難しい場合にのみ、この種のデータのために専用のスキーマを持つことに頼ることになるだろう。

この節の例で取り上げてきたのは、スキーマを分離するのに役立つデータベースリファクタリング手法だ。このテーマのより詳細な議論については、Scott AmblerとPramod Sadalageによる『データベース・リファクタリング』（ピアソン・エデュケーション）[1]を参照してほしい。

4.15　トランザクション

データベースを分解するときに結果として起こりうるいくつかの問題について触れてきた。参照整合性の維持が問題となり、レイテンシーが増加し、レポーティングのような活動がより複雑になる可能性がある。これらの問題のいくつかについて、さまざまな対処パターンを見てきたが、一つだけ大きな問題が残っている。トランザクションだ。

トランザクションでデータベースに変更を加えることが可能なら、システムがずっと見通しやすくなり、開発や保守も容易になる。データの安全性と一貫性の確保はデータベースに任せているので、それ以外の問題に目を向けることができる。しかし、データベース間でデータを分割してしまうと、状態の変化を

アトミックな方法で適用するという、データベーストランザクションを使用するメリットが失われてしまう。

この問題に対処する方法を探る前に、通常のデータベーストランザクションが提供するものを簡単に見ていこう。

4.15.1　ACIDトランザクション

データベーストランザクションについて話すとき、私たちは一般的にACIDトランザクションについて話している。**ACID**とは、データストレージの耐久性と一貫性を保証する信頼あるシステムを導く、データベーストランザクションの主要な特性を表す頭字語だ。それぞれの文字は、**原子性（Atomicity）**、**一貫性（Consistency）**、**分離性（Isolation）**、**永続性（Durability）**を表している。これらの特性が与えてくれるものを次に示す。

原子性
トランザクション内で完了した操作が、全て完了するか、全て失敗することを保証する。何らかの理由で変更に失敗した場合は、操作全体が中断され、変更は行われなかったかのようになる。

一貫性
データベースに変更が加えられた場合、データベースが有効で一貫性のある状態であることを担保する。

分離性
複数のトランザクションが干渉することなく同時に動作する。これは、1つのトランザクション中に行われた暫定的な状態変更を他のトランザクションからは見えないようにすることで実現される。

永続性
トランザクションが完了すると、システム障害が発生してもデータが失われることがないことを保証する。

注意すべきは、全てのデータベースがACIDトランザクションを提供しているわけではないことだ。私がこれまでに使った全てのリレーショナルデータベースシステムはACIDトランザクションを提供しているし、Neo4jのような新しいNoSQLデータベースの多くも同様にACIDトランザクションを提供している。MongoDBは長年にわたって単一のドキュメントを中心としたACIDトランザクションをサポートしていたが、複数のドキュメントをアトミックに更新したい場合に問題が発生する可能性があった[†17]。

[†17] MongoDB 4.0のリリースによって、マルチドキュメントのACIDトランザクションをサポートするようになった。ただし、私自身はMongoDBのこの機能を使ったことはない。

本書は、これらの概念を詳しく深く掘り下げるための本ではない。これらの概念をさらに掘り下げたい人は『データ指向アプリケーションデザイン』（オライリー・ジャパン）[20]を読むことをお勧めする。以下では、主に原子性に関心を持つことにする。他の特性も重要ではないというわけではないが、トランザクション境界を分割する際には、原子性が最初の問題となる傾向があるからだ。

4.15.2　ACIDだとしても原子性に欠ける？

まず明確にしたいのは、データベースを分割してもACIDスタイルのトランザクションを引き続き使用できるということだ。しかし、これらのトランザクションの範囲は、その有用性と共に縮小されている。図4-46を考えてみよう。私たちは、システムに新しい顧客を登録するプロセスを追跡している。このプロセスの最後に到達すると、顧客のステータスを「保留中」から「確認済み」に変更する。新しい顧客の登録が完了したので、登録保留テーブルから一致する行の削除も行いたい。単一のデータベースでは、これは単一のACIDデータベーストランザクションの範囲内で行われる。つまり、両方のテーブルが更新されるか、どちらも更新されないかのいずれかとなる。

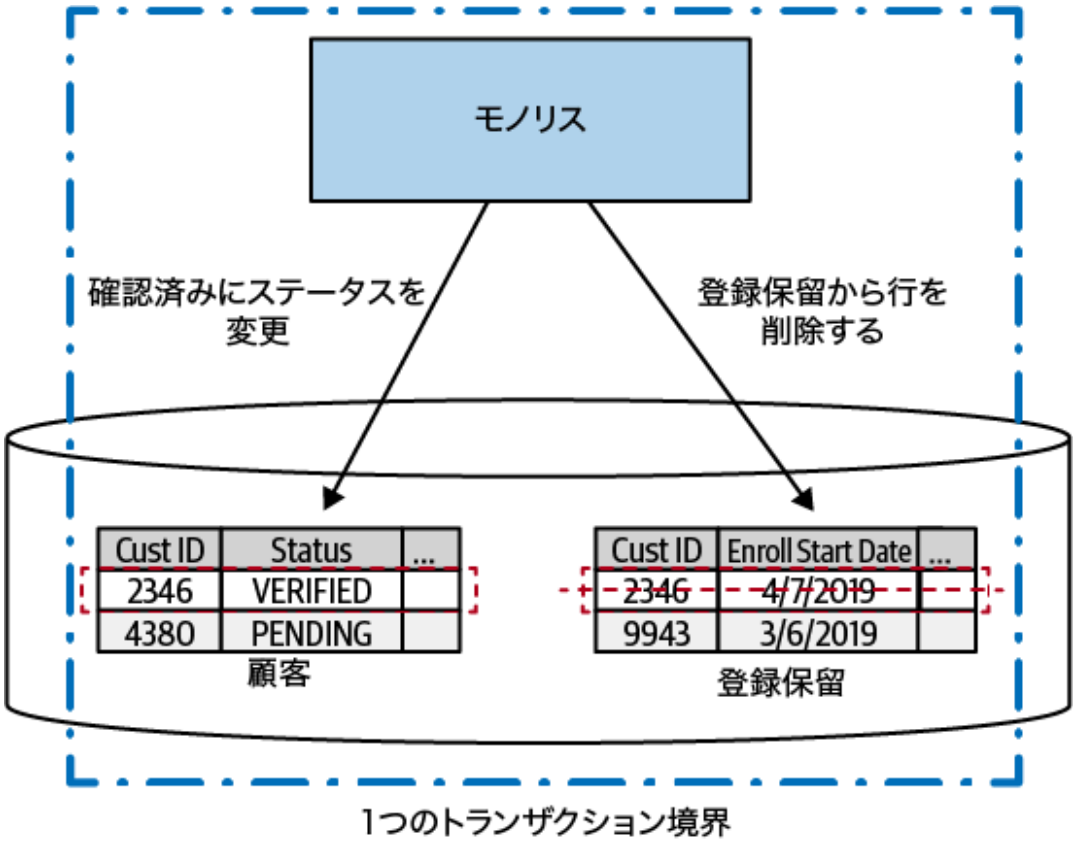


図4-46　1つのACIDトランザクションの範囲内での2つのテーブルの更新

これを図4-47と比較してみてほしい。全く同じ変更を行っているが、それぞれの変更は別のデータベースで行われている。これは考慮すべき2つのトランザクションがあり、それぞれが他のものとは独立して動作したり失敗したりする可能性があることを意味している。

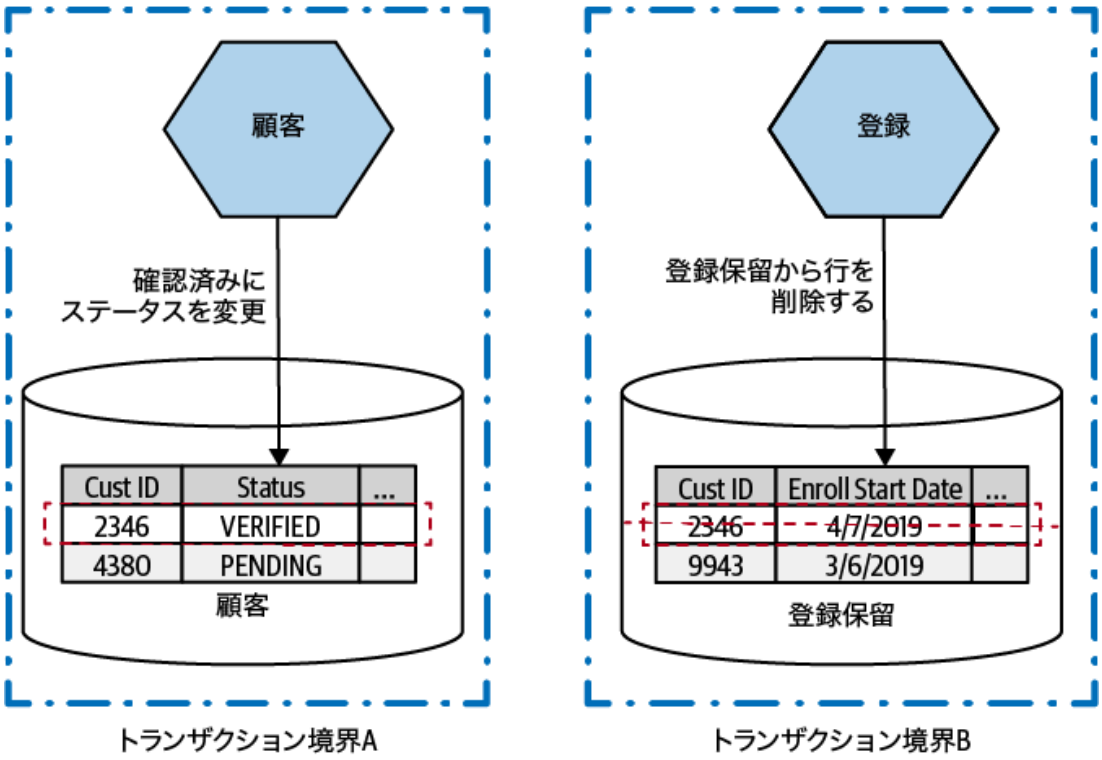


図4-47　顧客サービスと登録サービスの両方に加えられた変更は、2つの異なるトランザクションで独立して行われる

もちろん、これら2つのトランザクションを順序立てて実行することも可能だ。その場合には、登録保留テーブルから行を削除するのは、顧客テーブルの行を変更できた場合に限られる。しかし、登録保留テーブルからの削除が失敗した場合にどうすればよいのだろうか。そのロジックは自分たちで全て実装する必要がある。このようなユースケースを処理できるようにステップを並べ替えられるというのは、とても便利なアイデアだ（このアイデアは、サーガについて説明するときに戻ってくる）。しかし基本的には、この操作を2つの別々のデータベーストランザクションに分解することで、操作全体の原子性が保証されなくなったことを受け入れなければならない。

この原子性の欠如は、特に以前はこの特性に依存していたシステムを移行する場合に、重大な問題を引き起こす。この時点で、複数のサービスに同時に行われる変更を判断可能にするような他のソリューションを探し始める。通常、最初に検討されるのは分散トランザクションだ。ここでは、分散トランザクションを実装するための最も一般的なアルゴリズムの一つ、2フェーズコミットを見てみよう。

4.15.3　2フェーズコミット

2フェーズコミット（2相コミット、2PC）アルゴリズムは、全体の操作の一部として複数の分離されたプロセスを更新する必要がある場合に、分散システムにトランザクションで変更を行う能力を与えるためによく使用されている。2フェーズコミットには制限があることを前もって知っておいてもらいたい。それについては後ほど説明するが、知っておく価値のあることだ。分散トランザクション、特に2フェーズコミットは、マイクロサービスアーキテクチャに移行するチームが直面している課題を解決するための方法として頻繁に提起されている。しかし、これから見るように、これらは問題を解決しないかもしれないし、システムにさらに混乱をもたらすかもしれない。

アルゴリズムは、投票フェーズとコミットフェーズの2つのフェーズに分かれている（これが**2フェーズコミット**という名前の由来だ）。**投票フェーズ**では、中央のコーディネーターがトランザクションに参加する全てのワーカーに連絡を取り、状態の変更が可能かどうかの確認を求める。図4-48では、2つのリクエストがある。1つは顧客のステータスを「確認済み」に変更したいというもので、もう1つは登録保留テーブルから行を削除したいというものだ。全てのワ

ーカーが、要求された状態の変更が可能であることに同意した場合、アルゴリズムは次のフェーズに進む。そして、要求されたステータスの変更が何らかのローカル条件に違反しているといった理由で変更ができないと言うワーカーが中にいた場合には、操作全体が中断となる。

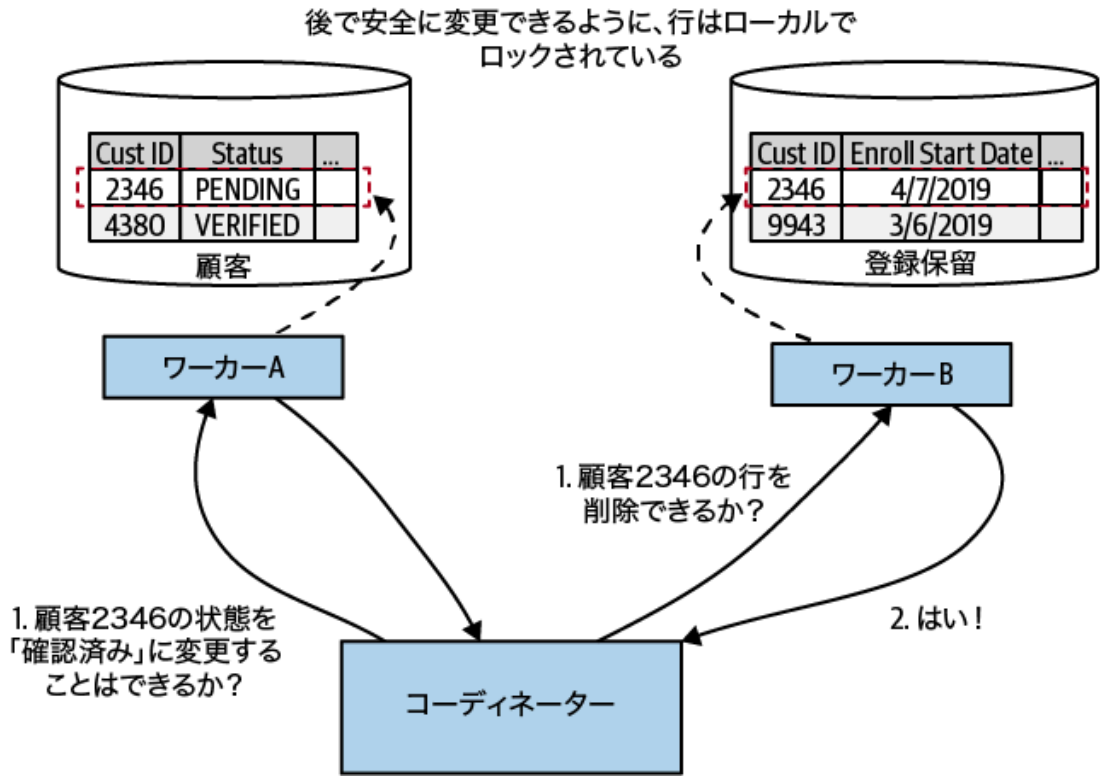


図4-48　2フェーズコミットの最初のフェーズでは、ワーカーはローカルの状態変更を実行できるかどうかを決定するために投票を行う

ワーカーが変更を行えることを示しても、即座にその変更が反映されるわけではない点は重要だ。ワーカーは、その代わりに、将来のある時点でその変更を行うことを保証している。ワーカーはどのようにしてそのような保証をするのだろうか。図4-48では、ワーカーAは顧客テーブルの行の状態を変更して特定の顧客の状態を「確認済み」に更新できると言っている。その後、別の操作で行が削除されたり、別の小さな変更が行われたりして、後になって「確認済み」への変更が無効となった場合はどうだろうか？　ワーカーAは、変更が確実に行われることを保証するために、そうした変更が行われないようにレコードをロックしなければならないだろう。

もし、コミットに投票しなかったワーカーがいた場合には、ロールバックメッセージを全ての関係者に送信して、ローカルでクリーンアップできるようにする必要がある。これにより、ワーカーは保持しているロックを解除できる。全てのワーカーが変更に同意した場合には、図4-49に示すように、コミットフェ

ーズに移行する。ここでは、変更が実際に行われ、関連するロックが解除される。

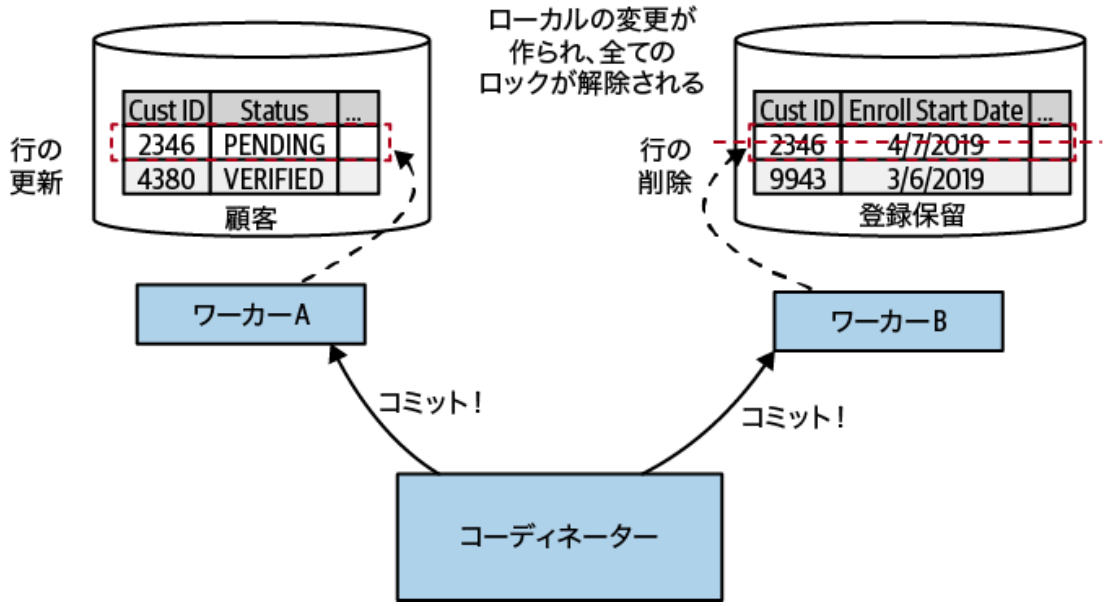


図4-49　2フェーズコミットのコミットフェーズでは、実際に変更が適用される

こうしたシステムでは、これらのコミットが正確に同じ時間に行われることは保証できない点に注意してほしい。コーディネーターは全ての参加者にコミット要求を送信する必要があり、そのメッセージは異なる時間に到着して処理される可能性がある。つまり、トランザクションコーディネーターの外部でワーカーの状態を確認できるようになっても、ワーカーAへの変更は確認できるが、ワーカーBへの変更は確認できない可能性があるということだ。コーディネーター間の待ち時間が長くなり、ワーカーが応答を処理するのが遅くなるほど、この不整合の間隔は広くなるかもしれない。ACIDの定義に戻ると、分離性はトランザクション中に中間状態を見ないことを保証する。しかし、この2フェーズコミットでは、それは失われている。

2フェーズコミットが機能するとき、その中心にあるのは分散ロックの調整だ。ワーカーは第2フェーズでのコミットを確実に実行するために、ローカルリソースをロックする必要がある。単一プロセスのシステムでロックを管理してデッドロックを回避するのは、楽しいことではない。次に、複数の参加者間でロックを調整するという課題を想像してみてほしい。これはきれいにはいかない。

2フェーズコミットに関連した障害モードはたくさんあるが、これについて調べている時間はない。あるワーカーがトランザクションを進めることに投票し

たのに、コミットを求められても応答しないという問題を考えてみよう。その場合にはどうすればよいだろうか？　これらの障害モードの中には自動的に処理できるものもあるが、システムを手動で問題を解決しなければならない状態にしてしまうものもある。

参加者の数が増え、システムの待ち時間が長くなればなるほど、2フェーズコミットではより多くの問題が発生する。特にロックの範囲が大きい場合やトランザクションの持続時間が大きい場合には、2フェーズコミットは、システムに多大なレイテンシーを生じさせる原因となる。このような理由から、2フェーズコミットは通常、非常に短い期間の操作にのみ使用される。操作が長引けば長引くほど、リソースが長い時間ロックされることになる。

4.15.4　分散トランザクションにはノーと言う

これまでに説明した理由から、マイクロサービス全体の状態変化を調整するために、2フェーズコミットのような分散トランザクションを使用するのはお勧めできない。では、他に何ができるだろうか？

第一の選択肢は、そもそもデータを分割しないということだ。本当に原子的で一貫性のある方法で管理したい状態の断片があって、ACID形式のトランザクションを使わずにうまくこれらの特性を得る方法がない場合には、その状態を単一のデータベースに残し、その状態を管理する機能を単一のサービス (またはモノリス) に残すようにしよう。モノリスをどこに分割するかを検討していて、どのような分解が簡単（または難しい）かを考えている最中なら、現状トランザクションで管理されているデータを今すぐに分割するのはあまりにも難しいと判断できるだろう。その場合には、システムの他の領域の作業を行い、その後でこのトピックに戻ってくるとよい。

しかし、本当にデータを分割する必要があっても、分散トランザクションを管理する手間をかけたくない場合はどうなるだろうか？　ロックを回避しながら複数のサービスで操作を実行するにはどうすればいいだろう？　操作に数分、数日、もしかしたら数カ月かかるかもしれない場合はどうすればいいだろうか？　このような場合には、サーガという別のアプローチが考えられる。

4.16　サーガ

2フェーズコミットとは異なり、**サーガ（saga）**は複数の状態変更を調整でき、リソースを長時間ロックする必要がないようデザインされたアルゴリズム

だ。サーガでは、関係するステップを独立して実行できる離散的な作業としてモデル化することでそれを実現する。これには、ビジネスプロセスを明示的にモデル化することを強いるという追加の効果があり、大きな利点になる。

Hector Garcia-MolinaとKenneth Salemによって最初に提案された核となるアイデア[†18]は、彼らが長命トランザクション（Long-lived Transactions、LLT）と呼ぶものの操作をどのように処理するのが最善かという課題を反映したものだった。これらのトランザクションには長い時間（数分、数時間、場合によっては数日）がかかり、そのプロセスの一部としてデータベースに変更を加える必要がある。

[†18] Hector Garcia-Molina and Kenneth Salem, "Sagas", in ACM Sigmod Record 16, no. 3 (1987): 249-259. [12]

LLTを通常のデータベーストランザクションに直接マッピングした場合、1つのデータベーストランザクションはLLTのライフサイクル全体にまたがることになる。これは、LLTが行われている間、複数の行やテーブル全体が長時間ロックされる結果となり、他のプロセスがこれらのロックされたリソースを読み取ったり変更したりしようとした場合に、重大な問題を引き起こす可能性がある。

その代わりに、先の論文の著者らは、これらのLLTを一連のトランザクションに分解し、それぞれが独立して処理できるようにすることを提案した。このアイデアは、これらの「サブ」トランザクションのそれぞれの継続時間が短くなり、LLT全体の影響を受けるデータの一部のみを変更するというものだ。その結果、ロックの範囲と継続時間が大幅に削減されるため、基礎となるデータベースでの競合がはるかに少なくなる。

サーガは、もともとは単一のデータベースに対して動作するLLTを支援するメカニズムとして想定されていた。しかし、このモデルは複数のサービスにまたがる変更を調整するためにも同様に機能する。私たちは、一つのビジネスプロセスを、単一のサーガの一部として協調するサービス群に行わせる一連の呼び出しに分割できる。

この先に進む前に、サーガは通常のデータベーストランザクションで使われるACIDの用語でいう原子性を与えないことを理解する必要がある。私たちはLLTを個々のトランザクションに分割しているが、 サーガ自体のレベルでは原子性はない。しかし、LLT内の各サブトランザクションには原子性がある。各サブトランザクションは、必要に応じてACIDトランザクションの変更に関連付けることが可能だからだ。サーガが私たちに与えてくれるのは、それがどの状態にあるかを推論するのに十分な情報だ。この影響を処理するのは私たち次第ということになる。

図4-50に示されている簡単な注文処理フローを使って、マイクロサービスアーキテクチャのコンテキストでサーガをさらに探っていこう。

ここでは、注文処理プロセスは単一のサーガとして表現され、フローの各ステップは異なるサービスによって実行される操作を表している。各サービス内では、どのような状態変化もローカルACIDトランザクション内で処理できる。たとえば、倉庫サービスを使用して在庫をチェックして予約するとき、倉庫サービスはその予約を記録する内部の予約テーブルに行を作成するかもしれない。この変更は通常のトランザクション内で処理される。

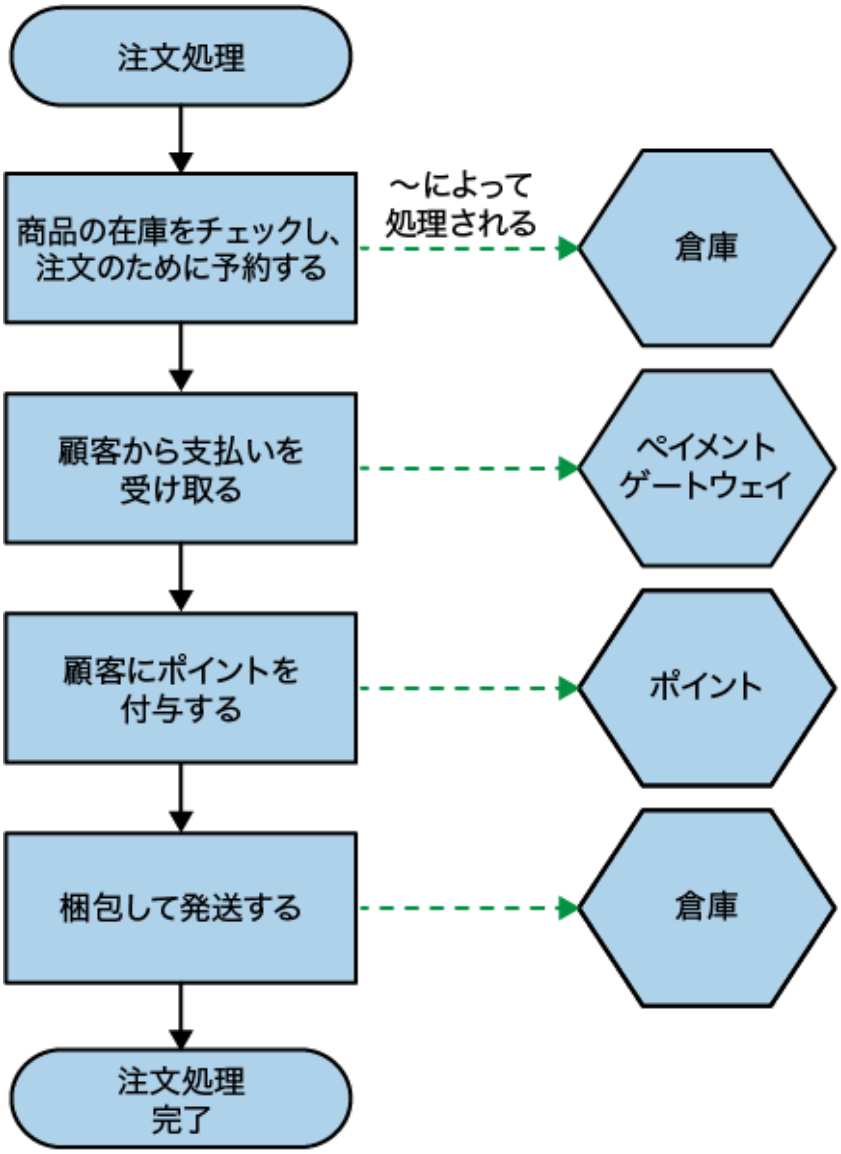


図4-50　注文処理フローの例と、その操作を実行する責務を持つサービス

4.16.1　サーガの障害モード

サーガは個々のトランザクションに分割されている。そのため、障害をどのように処理するか、より具体的には障害が起きたときにどのように回復するかを考える必要がある。サーガが提案された論文では、後方回復と前方回復という2種類の回復方法が記載されている。

後方回復には、障害を元に戻して後始末すること、つまりロールバックが含まれる。これを機能させるには、以前にコミットしたトランザクションを元に戻せる補償アクションを定義する必要がある。**前方回復**では、障害が発生したポイントからピックアップして処理を継続できる。これを実現するには、トランザクションを再試行できる必要がある。言い換えると、これはシステムがこの再試行をするのに十分な情報を保持していることを意味する。

モデル化されているビジネスプロセスの性質にもよるが、いずれの障害モードでも、後方回復、前方回復、またはその2つの組み合わせを使うことになるだろう。

サーガのロールバック

ACIDトランザクションでは、ロールバックはコミット前に起きる。ロールバック後は、何も起こらなかったかのような状態に戻り、変更は行われない。しかし、サーガの場合はそうはいかない。複数のトランザクションが関与しているため、そのうちのいくつかは、全体の操作をロールバックする前にコミットされている可能性がある。では、既にコミットされているトランザクションをロールバックするにはどうすればいいだろうか？

図4-50の注文を処理する例に戻ってみよう。潜在的な障害モードを考えてほしい。商品を梱包しようとしたところ、倉庫に商品が見つからないことに気がついた。システムは商品が存在すると考えているものの、棚にはないというシチュエーションだ。これを図4-51に示す。

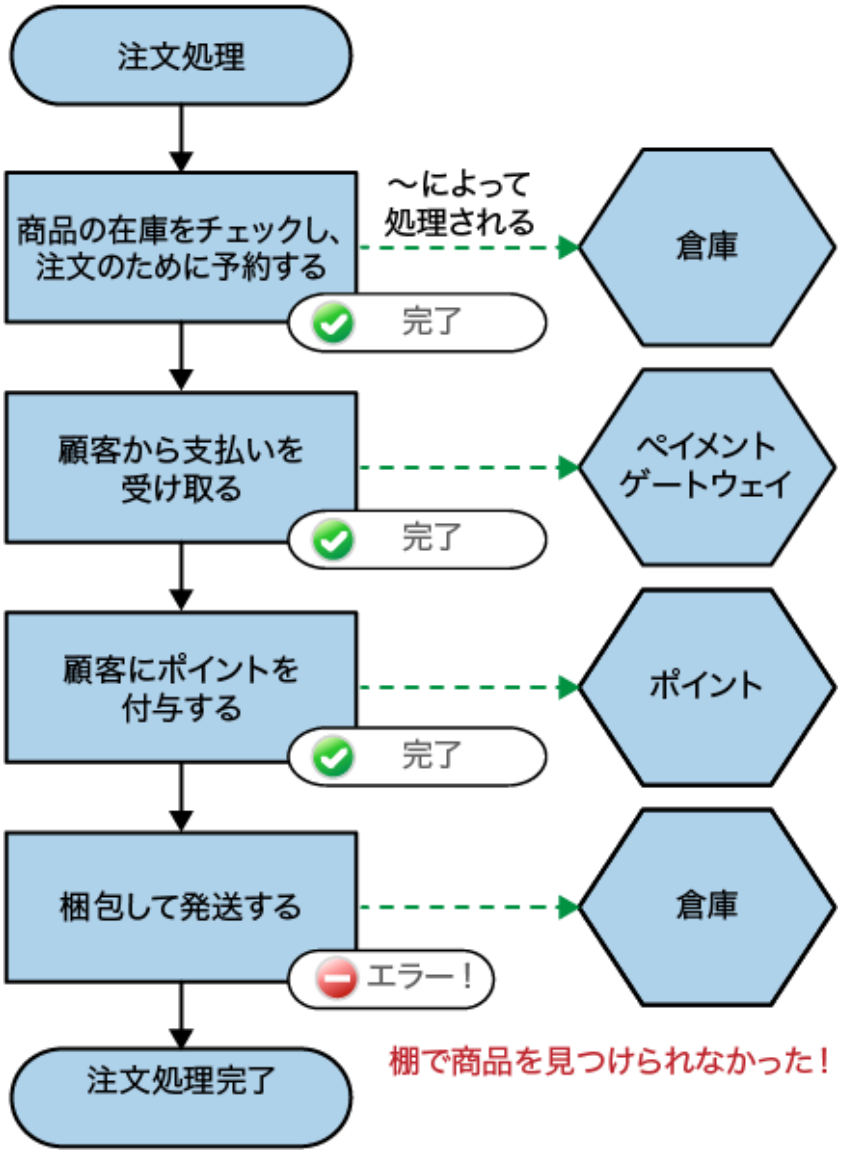


図4-51　商品を梱包しようとしたが、倉庫で見つからなかった

このとき、商品を取り寄せ注文とするのではなく、注文全体をロールバックすることにしたとしよう。問題は、既に支払いが行われ、注文に対してポイントが付与されていることだ。

もしこれらのステップの全てが単一のデータベーストランザクションで行われていたならば、単純なロールバックで全てがきれいになるだろう。しかし、注文処理の各ステップは異なるサービス呼び出しによって処理され、それぞれが異なるトランザクション範囲で動作している。操作全体のための単純な「ロールバック」は存在しない。

ロールバックを実装したい場合は、代わりに補償トランザクションを実装する必要がある。**補償トランザクション（Compensating Transaction）**とは、以前にコミットされたトランザクションを元に戻す操作だ。注文処理プロセスを

ロールバックするには、図4-52に示すように、既にコミットされているサーガの各ステップに対して補償トランザクションを実行することになる。

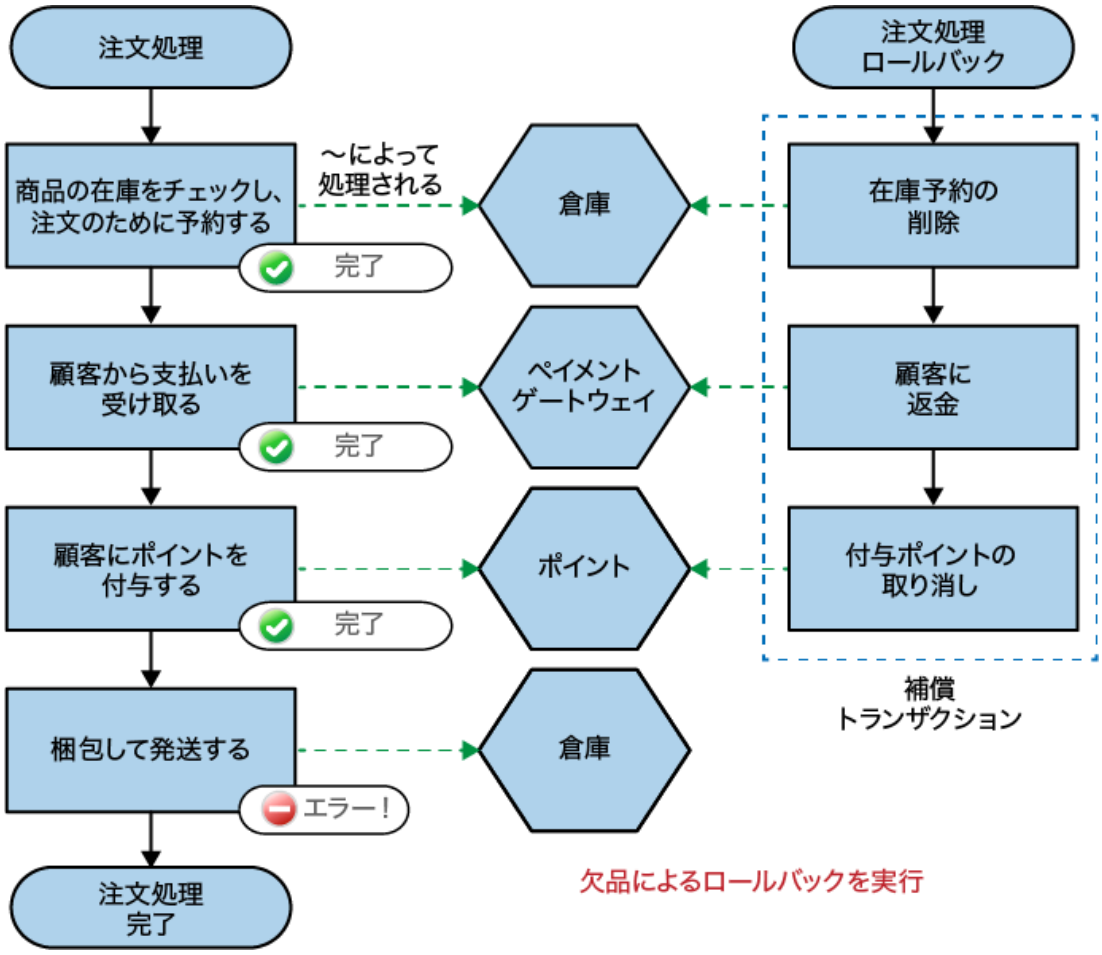


図4-52　サーガ全体のロールバックを実行する

補償トランザクションで重要なのは、データベースにおける通常のロールバックと全く同じ動作にはならないという事実だ。データベースのロールバックはコミット前に行われる。そのため、ロールバックの場合はトランザクションは全く起きていないかのようになる。一方で、サーガではこのようなトランザクションは実際に**起こる**。元のトランザクションで行われた変更を元に戻す新しいトランザクションを作成しているものの、時間を巻き戻して元のトランザクションが発生しなかったかのようにすることはできない。

トランザクションを常にきれいに戻すことはできないため、これらの補償トランザクションは「セマンティックロールバック（意味的なロールバック）」と呼ばれる。補償トランザクションは常に全てをクリーンアップすることはできないものの、サーガの文脈では十分なことを行う。業務プロセスの一つに、顧客に注文が処理中になったことを伝えるためにメールを送信することが含まれているとする。これをロールバックすることにした場合でも、電子メールを送っ

た事実を取り消すことはできない[†19]。その代わりに、補償トランザクションを実行することになるだろう。補償トランザクションでは、問題があったため注文がキャンセルされたことを知らせる2通目の電子メールを顧客に送信することになる。

[†19] 本当にできないんだ。私は試したことがある！

ロールバックされるサーガについての情報がシステムに残るのは、十分に適切だ。実際、それはとても重要な情報かもしれない。いろいろな理由から、この中止された注文の記録を何が起こったかの情報と共に注文サービスに記録しておきたい可能性があるからだ。

ロールバックを減らすために手順を並び変える

図4-52に示すサーガは、ステップの順序を変更することでロールバックのシナリオをよりシンプルにできる。具体的には、図4-53に示すように、注文が実際に発送された時にのみポイントを付与するようにする。このようにすれば、注文を梱包して発送しようとしていて問題が発生したとしても、ポイント付与のロールバックを心配しなくて済むようになる。プロセスの実行方法に手を加えるだけで、ロールバック作業を簡単にできることがある。失敗する可能性が最も高いステップを前倒しにして、プロセスを早めに失敗させる。そうすることで、下流のステップがそもそも実行されないようになるため、後から補償トランザクションを実行する必要はなくなる。

こうした変更が行えるなら、いくつかのステップの補償トランザクションを作成する必要性が回避されるため、運用はずっと楽になる。これは、補償トランザクションの実装が困難な場合に特に重要になる。プロセスの後半にステップを移動することで、そのステップをロールバックさせる必要がなくなるかもしれないと覚えておこう。

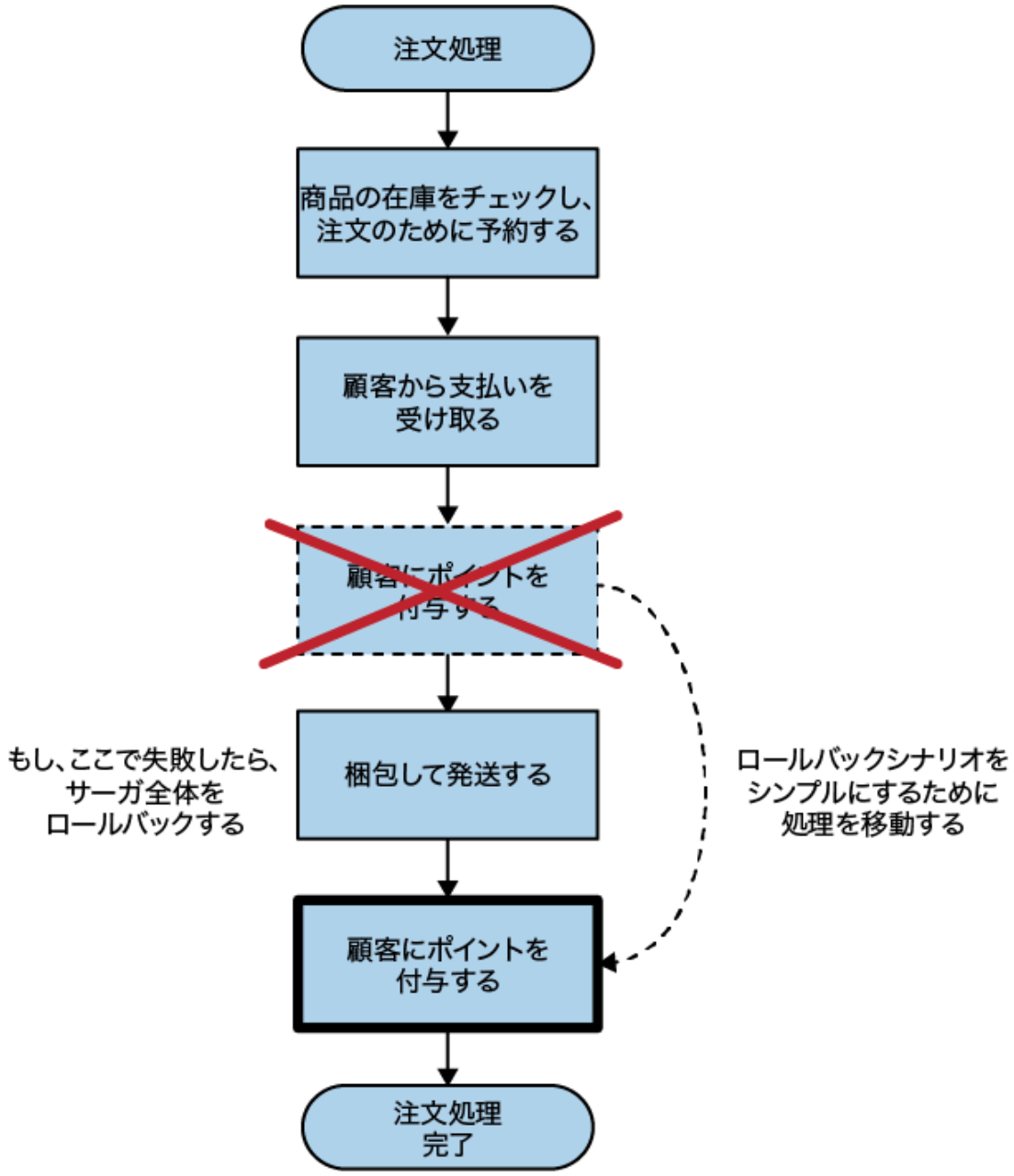


図4-53　サーガの中でステップを後回しにすることで、障害が発生した場合にロールバックしなければならないものを少なくできる

障害時の切り戻しと障害からの立ち直りを組み合わせる

障害回復モードを複数持つことは全くもって適切だ。障害の中にはロールバックが必要なものもあれば、障害からの立ち直りが必要なものもあるからだ。注文処理の場合であれば、顧客からお金を受け取って商品が梱包されていたら、残りのステップは荷物を発送するだけだ。何らかの理由で商品を発送できなかったとしても（配送業者のバンにその日の注文を受け取るだけのスペースがなかった場合など）、注文全体をロールバックするのはあまりに不自然だ。代わりに、発送をやり直すのが自然だろう。そして、それが失敗した場合には、状況を解決するために人の介入を必要とするはずだ。

4.16.2　サーガの実装

ここまでは、サーガがどう機能するかの論理的なモデルを見てきた。サーガ自体を実装する方法を考えるには、もう少し深く掘り下げてみる必要がある。サーガの実装には2つのスタイルがある。**オーケストレーションベースのサーガ**は、もともとのソリューション空間に忠実に従ったもので、主に中央集権的な調整と追跡に依存している。オーケストレーションベースのサーガは、**コレオグラフィベースのサーガ**と比較される。コレオグラフィベースのサーガは、中央集権的な調整の必要性を避け、よりゆるやかに結合されたモデルを好んで使用するものの、サーガの進行状況の追跡をより複雑にする可能性がある。

オーケストレーションベースのサーガ

オーケストレーションベースのサーガでは、中央のコーディネーター（以降は「**オーケストレーター**」と呼ぶことにする）が実行の順序を定義し、必要とされる補償アクションを実行する。オーケストレーションベースのサーガは、コマンドアンドコントロールのアプローチと考えられる。つまり、中央のオーケストレーターは、いつ何が起こるかをコントロールし、それによって、特定のサーガで何が起こっているかについて十分な可視性を保証する。

図4-50で示されている注文処理プロセスを例に、連携するサービスの集合として、この中央の調停プロセスがどのように機能するかを図4-54に示す。

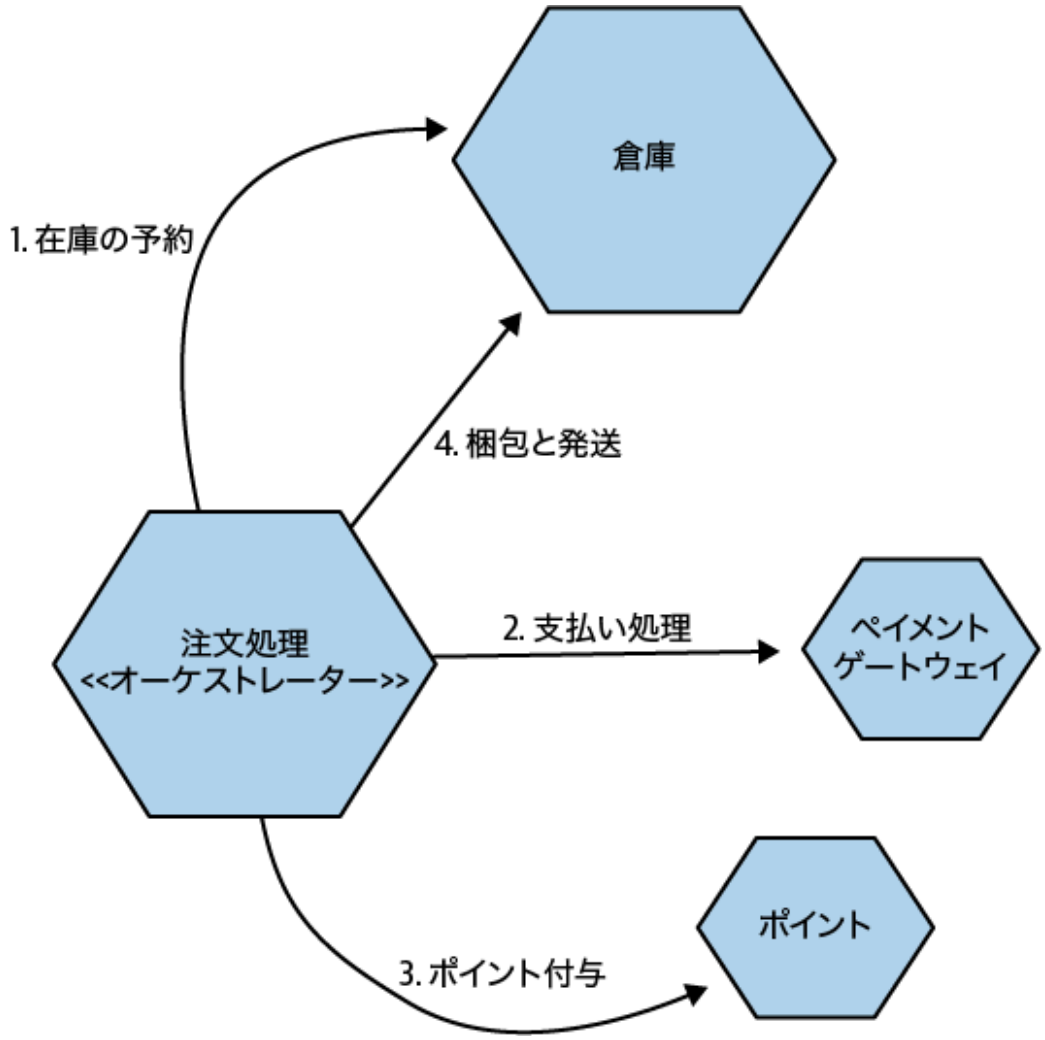


図4-54　注文処理プロセスを実装するために、オーケストレーションベースのサーガがどのように使用されるかの例

ここでは、オーケストレーターの役割を果たす中央の注文処理サービスが、注文処理プロセスを調整する。注文処理サービスは、操作を実行するためにどのようなサービスが必要かを知っており、それらのサービスにいつ呼び出しを行うかを決定する。それによって、呼び出しに失敗した場合に、結果として何をすべきかを決定できる。こうしたオーケストレーションベースの処理では、サービス間のリクエスト／レスポンス呼び出しを多用する傾向がある。注文処理サービスはサービス(ペイメントゲートウェイなど)にリクエストを送信し、リクエストが成功したかどうかとリクエストの結果を提供するレスポンスを期待する。

注文処理サービスの内部でビジネスプロセスを明示的にモデル化することは、とても有益だ。これにより、システムの一箇所を見るだけで、このプロセスがどのように動作するのかを理解できる。そして、システムのコア部分を伝えやすくなり、人員の増加も容易になる。

ただし、考慮すべきデメリットもいくつかある。最初に、その性質上、これはやや結合されたアプローチとなる。注文処理サービスは、関連する全てのサービスについて知る必要があり、結果として、1章で説明したドメイン結合をより強いものにする。これは本質的に悪いことではないものの、可能であればドメイン結合は最小限に抑えたいものだ。しかし、この例では、注文処理サービスは多くのことを知って制御する必要があるので、この形式の結合を避けることは難しい。

もう一つのより微妙な問題は、本来であればサービスに押し込まれるべきロジックが、代わりにオーケストレーターに吸収され始める可能性があるということだ。このようなことが起こり始めたら、サービスは、注文処理サービスのようなオーケストレーターからの命令をただ受けている、自身の振る舞いをほとんど持たない貧弱なサービスになっているかもしれない。こうしたオーケストレーションの各フローを構成するサービスを、独自のローカル状態と振る舞いを持つエンティティとして考えることは依然として重要だ。それらのサービスは独自のローカルな状態マシンを担当している。

ロジックを集中管理できる場所があると、ロジックはそこに集中管理されるようになってしまう！

オーケストレーションのフローが集中化しすぎないようにするには、フローごとに異なるサービスがオーケストレーターの役割を果たすようにする方法がある。注文を処理する注文処理サービス、返品と返金のプロセスを処理する返品サービス、倉庫に到着した新しい在庫を処理して棚に並べる商品入荷サービスなどがあるかもしれない。このようなモデルでは、倉庫サービスに機能を持たせておくことで、そうした全てのフローから機能を再利用することが容易になる。

BPMツール？

ビジネスプロセスモデリング（BPM）ツールは、何年も前から利用可能になっている。これらのツールは、多くの場合、開発者以外でも直感的なドラッグ＆ドロップでビジネスプロセスフローを定義できるように設計されている。このアイデアは、開発者がこれらのプロセスの構成要素を作成し、開発者以外がこれらの構成要素をより大きなプロセスフローに接続するというものだ。このようなツールの使用は、オーケストレーションベースのサーガを実装する方法として非常にうまくいっているようで、実際にプロセスのオーケストレーションを行うことはBPMツールの主なユースケースとなっている（逆に言えば、BPMツールを使うとオーケストレーションを採用しなければならなくなっている）。

過去の経験から、私はBPMツールが大嫌いになった。その主な理由は「開発者以外がビジネスプロセスを定義する」という中心的な考え方が、経験上、ほとんど現実的ではなかったからだ。開発者以外に向けたツールは結局開発者が使うことになり、多くの問題を抱えることになる。フローを変更するにはGUIを使用する必要があることが多く、作成したフローはバージョン管理が難しい（または不可能）場合があり、フロー自体がテストを考慮して設計されていない場合がある。

もし開発者がビジネスプロセスを実装するのであれば、開発者がよく分かっていて、ワークフローにも適しているツールを使用させよう。一般的には、これは、単にコードを使ってワークフローを実装することを意味する。ビジネスプロセスがどのように実装されたか、またはどのように動作しているかを可視化する必要がある場合には、コードがどのように動作するかをワークフローの視覚的な表現を使用して説明するよりも、コードからワークフローの視覚的な表現を投影させる方がはるかに簡単だ。

より開発者に優しいBPMツールを作成する努力もなされている。これらのツールに対する開発者からのフィードバックは賛否両論あるようだが、一部の人にとってはうまく機能しており、これらのフレームワークの改善が見受けられるのは良いことだ。もしこれらのツールをさらに探求する必要があると感じたら、Camunda[†20]やZeebe[†21]を見てみるとよいだろう。どちらもマイクロサービス開発者を対象としたオープンソースのオーケストレーションフレームワークだ。

[†20] https://camunda.com/

[†21] https://zeebe.io

コレオグラフィベースのサーガ

コレオグラフィベースのサーガは、複数の連携サービス間でサーガの運用責任を分散させることを目的としている。オーケストレーションベースのサーガが指揮と統制のアーキテクチャなら、コレオグラフィベースのサーガは信頼と検証のアーキテクチャだ。図4-55に示すように、コレオグラフィベースのサーガはサービス間のコラボレーションのためにイベントを多用する。

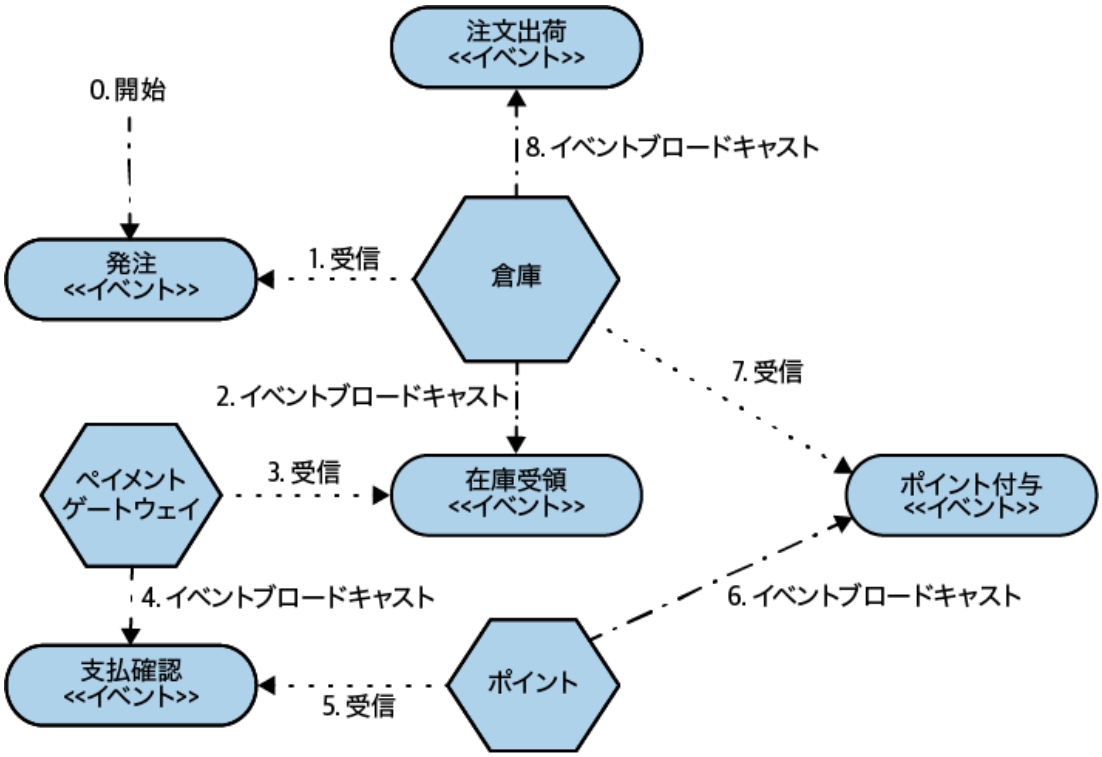


図4-55　注文処理を実装するためのコレオグラフィベースのサーガの例

ここではかなり多くのことが行われているので、より詳しく探っていこう。まず、これらのサービスは受信したイベントに反応している。概念的には、イベントはシステム内でブロードキャストされ、興味のあるサービスはそれを受信できる。サービス**に**イベントを送信する必要はない。イベントをただ発生させるだけだ。そうすれば、興味を持つサービスはイベントを受信し、それに応じて行動できる。この例では、倉庫サービスは、最初の発注イベントを受信すると、適切な在庫を確保し、完了したイベントを発信するのが役目だと分かって

いる。在庫を確保できなかった場合には、倉庫サービスは適切なイベント（おそらく在庫不足イベント）を発生させる必要があり、それによって注文が中止される可能性がある。

通常、信頼性のあるブロードキャストとイベントの配信を管理するには、何らかのメッセージブローカーを使用する。メッセージブローカーを用いると、複数のサービスが同じイベントに反応することやトピックの使用が可能になる。特定の種類のイベントに興味があるサービスは、イベントがどこから来たのかを気にすることなく特定のトピックをサブスクライブでき、ブローカーはトピックの持続性とそのトピック上のイベントがサブスクライバーに正常に配信されることを保証する。たとえば、発注イベントを待ち受け、それを好みの音楽データベースを構築するために利用するレコメンデーションサービスが考えられる。

前述のアーキテクチャでは、どのサービスも他のサービスのことを知らない。サービスは、あるイベントを受信したときに何をすべきかを知っている必要があるだけだ。本質的に、これはより結合の少ないアーキテクチャになる。プロセスの実装が分解され、4つのサービスに分散されているので、ロジックの集中化に関する懸念も回避できる（ロジックを集中化できる場所がなければ、集中化されることはない！）。

これの裏返しは、何が起こっているのかを把握するのが難しくなったということだ。オーケストレーションベースのサーガでは、プロセスはオーケストレーターによって明示的にモデル化されていた。さて、このようなアーキテクチャが提示されているとして、プロセスがどのようなものかのメンタルモデルをどのようにして構築するのだろうか？　そのためには、各サービスの動作を個別に見て、自分の頭の中で再構成しなければならない。ここで示したように、単純なビジネスプロセスであっても、それはとても複雑なプロセスとなる。

ビジネスプロセスの明示的な表現がないのは、それだけでもまずいことだ。それに加えて、サーガがどのような状態にあるのかを知る方法も欠いているため、必要なときに補償アクションを実行する機会も失われる可能性がある。補償アクションの実行責任を個々のサービスに押し付けることもできるが、基本的には、ある種の回復のためにサーガがどのような状態にあるかを知る方法が必要だ。サーガの状態を調べるための中心的な場所がないのは大きな問題となる。オーケストレーションベースのサーガであればそれは分かることだが、ここではどうやって解決するとよいだろうか？

これを行う最も簡単な方法の一つは、発行されたイベントを使用して、既存のシステムからサーガの状態に関するビューを投影することだ。サーガのためにユニークなIDを生成しておけば、このIDをサーガの一部として発行される全てのイベントに入れておける。これが**相関ID（Correlation ID：CID）**として知られているものだ。そうすれば、これら全てのイベントを収集するサービスを作ることで、各注文がどのような状態にあるかのビューを提供し、場合によっては他のサービスが自ら補償アクションを行えない場合にも注文処理の一部として問題を解決するためのアクションをプログラム的に実行できるかもしれない。

混合スタイル

オーケストレーションベースのサーガとコレオグラフィベースのサーガは、サーガがどのように実装されるかという点では正反対のアイデアのように思われるかもしれない。しかし、これらのモデルの組み合わせは容易に考えられる。システムの中には、いずれかのモデルにより自然に適合するビジネスプロセスがあるかもしれないし、混在したスタイルを持つ単一のサーガを持つものもあるかもしれない。注文処理のユースケースであれば、より巨大なコレオグラフィベースのサーガの一部として元のリクエストが行われた場合であっても、倉庫サービスの境界内では、パッケージの梱包と発送を管理する際にオーケストレーションベースのフローを使用することがある[†22]。

[†22] この本の範囲外ではあるが、Hector Garcia-MolinaとKenneth Salemは、より複雑なプロセスを実装するために複数のサーガを「入れ子にする」方法を探究している。このトピックの詳細については、Hector Garcia-Molina他 “Modeling Long-Running Activities as Nested Sagas” [13]を参照してほしい。

スタイルを混在させることにした場合でも、サーガの一部として何が起こったかを明確に理解することが重要だ。これがないと、障害モードの理解が複雑になり、障害からの復旧が困難になる。

コレオグラフィやオーケストレーションを使うべきか？

コレオグラフィベースのサーガを実装していると、あなたやあなたのチームにとって見慣れないアイデアが出てくるかもしれない。コレオグラフィベースの

サーガは一般的にイベント駆動型のコラボレーションを多用することを前提としているが、これは広く理解されていないからだ。しかし、私の経験では、サーガの進行状況を追跡することに関連した余分な複雑さは、ほとんどの場合、より疎結合なアーキテクチャを持つことに関連する利点には勝らない。

私の個人的な好みはさておき、オーケストレーションとコレオグラフィに関する私からの一般的なアドバイスは、一つのチームがサーガ全体の実装を所有している場合には、オーケストレーションベースのサーガを使用する方がとても安心できるということだ。このような状況では、より本質的に結合されたアーキテクチャの方が、チームの境界内での管理がはるかに容易になる。複数のチームが関与している場合には、より分解されたコレオグラフィベースのサーガの方を好んで使用する。より疎結合のアーキテクチャにより、チームがより独立して作業できるからだ。

4.16.3　サーガ vs 分散トランザクション

ここまでにかみ砕いて説明してきたように、分散トランザクションにはいくつかの重要な課題があり、特定の状況以外では、私は避ける傾向がある。分散システムの先駆者であるPat Hellandは、今日私たちが構築している種類のアプリケーションで分散トランザクションを実装する際の基本的な課題を次のように説明している。

> *ほとんどの分散トランザクションシステムでは、単一ノードの障害によってトランザクションのコミットが停止する。これにより、アプリケーションの詰まりが生じる。このようなシステムは、大きくなればなるほど、システムがダウンする可能性が高くなる。全てのエンジンを動作させる必要がある飛行機を飛行させる場合、エンジンを追加するにつれて飛行機の可用性が低下する。*
>
> *Pat Helland “Life Beyond Distributed Transactions”* [14]

私の経験では、ビジネスプロセスをサーガとして明示的にモデル化することで、分散トランザクションの課題の多くを回避できる。それと同時に、ビジネスプロセスをサーガとして明示的にモデル化することには、暗黙のうちにモデル化されていたかもしれないプロセスを、開発者にとってより明示的で明確なものにできるという利点もある。システムの中核となるビジネスプロセスを第一級のコンセプトにすることは、多くの利点をもたらす。

オーケストレーションやコレオグラフィの実装について、さまざまな実装の詳細とともに詳しく説明することは、本書の範囲外だ。『マイクロサービスアーキテクチャ』[25]の4章で説明しているが、このトピックの多くの側面を深く掘り下げたい場合は “Enterprise Integration Patterns”（Addison-Wesley）[16]もお勧めだ[†23]。

[†23] どちらの本でもサーガについて明示的には触れられていないものの、オーケストレーションとコレオグラフィは両方とも取り上げられている。“Enterprise Integration Patterns” の著者の経験について話すことはできないが、『マイクロサービスアーキテクチャ』を書いた当時は、私はまだサーガを知らなかった。

4.17　この章のまとめ

サービスの境界線が現れる接合部を見つけシステムを分解していく作業は、段階的なアプローチとなる。そのように、接合部を見つけたり、そもそもサービスを分割するためのコストを減らすように努めたりするのに長けていくことで、将来どんな要件が生じてもそれを満たせるように、システムを継続的に成長、進化させていけるようになる。見てきたとおり、中には骨の折れるものもあるし、対処しなければならない重大な問題が発生することもある。しかし、段階的に行っていけるという事実は、この作業を恐れる必要がないことを意味している。

サービスを分割することで、いくつかの新しい問題も発生した。次の章では、モノリスを分解する際に出てくるさまざまな課題について見ていく。しかし、心配する必要はない。これらの問題が発生したときに対処するのに役立つアイデアもたくさん紹介していく。